Exploratory Verification
in the age of
Probabilistic Code

Mourjo Sen

September 2026

$

```
$ mvn clean test
```

```
$ mvn clean test
```

```
…
tries = 187                      | # of calls to property
checks = 187                     | # of not rejected calls
generation = RANDOMIZED          | parameters are randomly generated
…

[ERROR] Tests run: 3, Failures: 1, Errors: 0, Skipped: 0
```

```
$ mvn clean test

 Invariant 'no-overlap' failed after the following actions: [




 ]

…
tries = 187                     | # of calls to property
checks = 187                    | # of not rejected calls
generation = RANDOMIZED         | parameters are randomly generated
…

[ERROR] Tests run: 3, Failures: 1, Errors: 0, Skipped: 0
```

```
$ mvn clean test

 Invariant 'no-overlap' failed after the following actions: [
     alice creates Meeting-1 in calendar default (UTC) from 10:00 to 10:01



 ]

…
tries = 187                      | # of calls to property
checks = 187                     | # of not rejected calls
generation = RANDOMIZED          | parameters are randomly generated
…

[ERROR] Tests run: 3, Failures: 1, Errors: 0, Skipped: 0
```

```
$ mvn clean test

 Invariant 'no-overlap' failed after the following actions: [
     alice creates Meeting-1 in calendar default (UTC) from 10:00 to 10:01
     bob creates Meeting-2 in calendar default (UTC) from 10:00 to 10:01

 ]

…
tries = 187                    | # of calls to property
checks = 187                   | # of not rejected calls
generation = RANDOMIZED        | parameters are randomly generated
…

[ERROR] Tests run: 3, Failures: 1, Errors: 0, Skipped: 0
```

```
$ mvn clean test

 Invariant 'no-overlap' failed after the following actions: [
     alice creates Meeting-1 in calendar default (UTC) from 10:00 to 10:01
     bob creates Meeting-2 in calendar default (UTC) from 10:00 to 10:01
     bob invites alice to Meeting-2

 ]

…
tries = 187                     | # of calls to property
checks = 187                    | # of not rejected calls
generation = RANDOMIZED         | parameters are randomly generated
…

[ERROR] Tests run: 3, Failures: 1, Errors: 0, Skipped: 0
```

```
$ mvn clean test

 Invariant 'no-overlap' failed after the following actions: [
     alice creates Meeting-1 in calendar default (UTC) from 10:00 to 10:01
     bob creates Meeting-2 in calendar default (UTC) from 10:00 to 10:01
     bob invites alice to Meeting-2
     alice accepts invitation to Meeting-2
 ]

…
tries = 187                      | # of calls to property
checks = 187                     | # of not rejected calls
generation = RANDOMIZED          | parameters are randomly generated
…

[ERROR] Tests run: 3, Failures: 1, Errors: 0, Skipped: 0
```

How did I miss that?

```
$ mvn clean test

Invariant 'no-overlap' failed after the following actions: [
    alice creates Meeting-1 in calendar default (UTC) from 10:00 to 10:01
    bob creates Meeting-2 in calendar default (UTC) from 10:00 to 10:01
    bob invites alice to Meeting-2
    alice accepts invitation to Meeting-2
]

…
tries = 187                      | # of calls to property
checks = 187                     | # of not rejected calls
generation = RANDOMIZED          | parameters are randomly generated
…

[ERROR] Tests run: 3, Failures: 1, Errors: 0, Skipped: 0
```

Namaste

- India → Netherlands
- Staff Engineer @ Booking.com
- Writing and speaking for 5 years

Everything's Changed

LLMs Write Faster Than We Can Read

LLMs Write Faster Than We Can Read

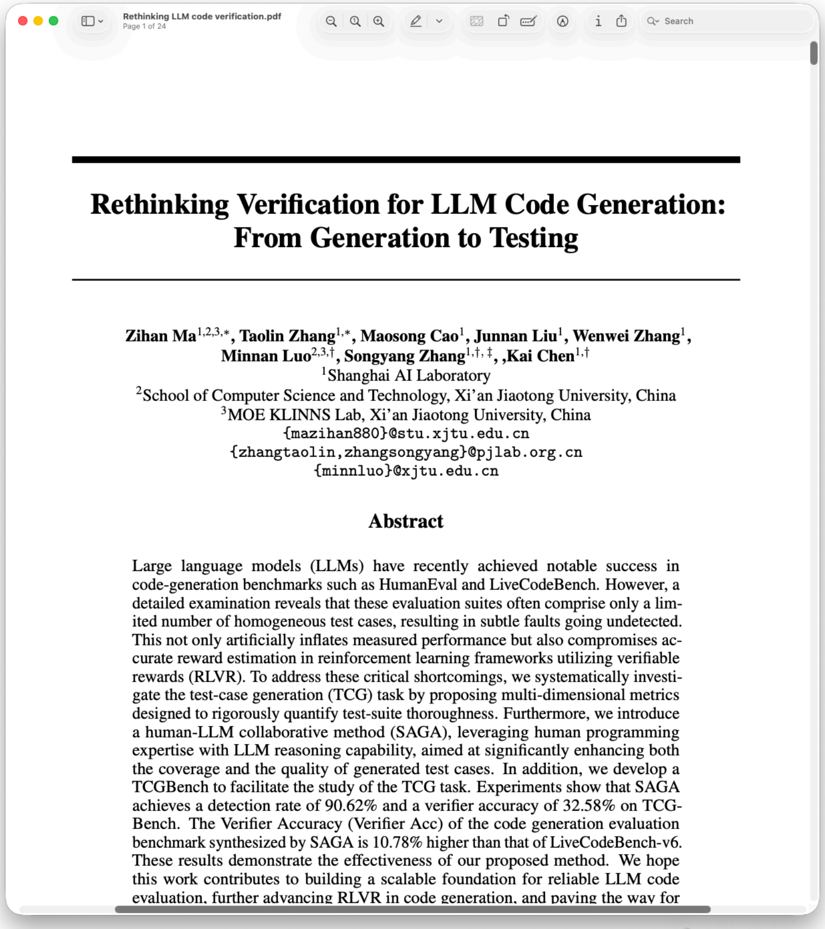
- Code generation is fast but probabilistic
  - Quality of software cannot be probabilistic

LLMs Write Faster Than We Can Read

- Code generation is fast but probabilistic
  - Quality of software cannot be probabilistic
- Use LLMs for testing software?
  - Because we cannot read at that speed

Are LLMs Good At Testing Software?

Are LLMs Good At Testing Software?



Rethinking Verification for LLM Code Generation: From Generation to Testing

Zihan Ma[1,2,3,*], **Taolin Zhang**[1,*], **Maosong Cao**[1], **Junnan Liu**[1], **Wenwei Zhang**[1], **Minnan Luo**[2,3,†], **Songyang Zhang**[1,†,‡], ,**Kai Chen**[1,†]
[1]Shanghai AI Laboratory
[2]School of Computer Science and Technology, Xi'an Jiaotong University, China
[3]MOE KLINNS Lab, Xi'an Jiaotong University, China
{mazihan880}@stu.xjtu.edu.cn
{zhangtaolin,zhangsongyang}@pjlab.org.cn
{minnluo}@xjtu.edu.cn

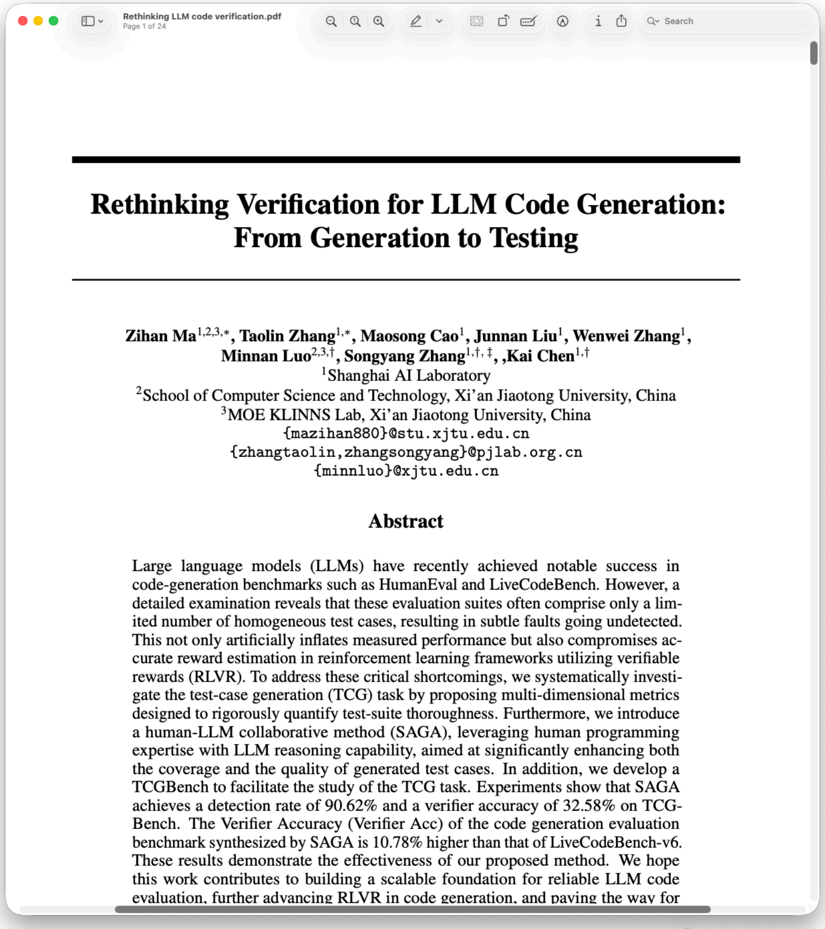
Abstract

Large language models (LLMs) have recently achieved notable success in code-generation benchmarks such as HumanEval and LiveCodeBench. However, a detailed examination reveals that these evaluation suites often comprise only a limited number of homogeneous test cases, resulting in subtle faults going undetected. This not only artificially inflates measured performance but also compromises accurate reward estimation in reinforcement learning frameworks utilizing verifiable rewards (RLVR). To address these critical shortcomings, we systematically investigate the test-case generation (TCG) task by proposing multi-dimensional metrics designed to rigorously quantify test-suite thoroughness. Furthermore, we introduce a human-LLM collaborative method (SAGA), leveraging human programming expertise with LLM reasoning capability, aimed at significantly enhancing both the coverage and the quality of generated test cases. In addition, we develop a TCGBench to facilitate the study of the TCG task. Experiments show that SAGA achieves a detection rate of 90.62% and a verifier accuracy of 32.58% on TCGBench. The Verifier Accuracy (Verifier Acc) of the code generation evaluation benchmark synthesized by SAGA is 10.78% higher than that of LiveCodeBench-v6. These results demonstrate the effectiveness of our proposed method. We hope this work contributes to building a scalable foundation for reliable LLM code evaluation, further advancing RLVR in code generation, and paving the way for

Source: https://arxiv.org/abs/2507.06920

Are LLMs Good At Testing Software?

- LLMs struggle to catch bugs
  - ~60% chance of identifying a buggy solution



Rethinking Verification for LLM Code Generation: From Generation to Testing

Zihan Ma[1,2,3,*], Taolin Zhang[1,*], Maosong Cao[1], Junnan Liu[1], Wenwei Zhang[1], Minnan Luo[2,3,†], Songyang Zhang[1,†,‡], ,Kai Chen[1,†]

[1]Shanghai AI Laboratory

[2]School of Computer Science and Technology, Xi'an Jiaotong University, China

[3]MOE KLINNS Lab, Xi'an Jiaotong University, China

{mazihan880}@stu.xjtu.edu.cn

{zhangtaolin,zhangsongyang}@pjlab.org.cn

{minnluo}@xjtu.edu.cn

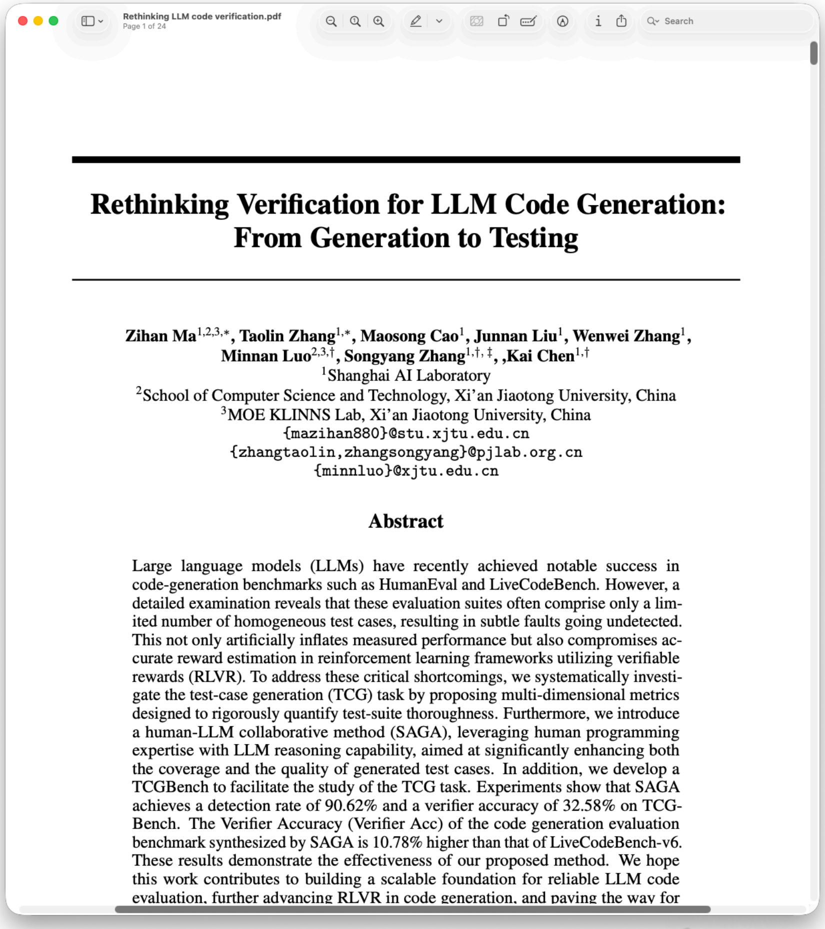
Abstract

Large language models (LLMs) have recently achieved notable success in code-generation benchmarks such as HumanEval and LiveCodeBench. However, a detailed examination reveals that these evaluation suites often comprise only a limited number of homogeneous test cases, resulting in subtle faults going undetected. This not only artificially inflates measured performance but also compromises accurate reward estimation in reinforcement learning frameworks utilizing verifiable rewards (RLVR). To address these critical shortcomings, we systematically investigate the test-case generation (TCG) task by proposing multi-dimensional metrics designed to rigorously quantify test-suite thoroughness. Furthermore, we introduce a human-LLM collaborative method (SAGA), leveraging human programming expertise with LLM reasoning capability, aimed at significantly enhancing both the coverage and the quality of generated test cases. In addition, we develop a TCGBench to facilitate the study of the TCG task. Experiments show that SAGA achieves a detection rate of 90.62% and a verifier accuracy of 32.58% on TCG-Bench. The Verifier Accuracy (Verifier Acc) of the code generation evaluation benchmark synthesized by SAGA is 10.78% higher than that of LiveCodeBench-v6. These results demonstrate the effectiveness of our proposed method. We hope this work contributes to building a scalable foundation for reliable LLM code evaluation, further advancing RLVR in code generation, and paving the way for

Source: https://arxiv.org/abs/2507.06920

Are LLMs Good At Testing Software?

- LLMs struggle to catch bugs
  - ~60% chance of identifying a buggy solution
- Inflated self-testing success
  - LLM-written tests have a high success rate against LLM-written source



Rethinking Verification for LLM Code Generation: From Generation to Testing

Zihan Ma[1,2,3,*], Taolin Zhang[1,*], Maosong Cao[1], Junnan Liu[1], Wenwei Zhang[1], Minnan Luo[2,3,†], Songyang Zhang[1,†,‡], ,Kai Chen[1,†]

[1]Shanghai AI Laboratory

[2]School of Computer Science and Technology, Xi'an Jiaotong University, China

[3]MOE KLINNS Lab, Xi'an Jiaotong University, China

{mazihan880}@stu.xjtu.edu.cn

{zhangtaolin,zhangsongyang}@pjlab.org.cn

{minnluo}@xjtu.edu.cn

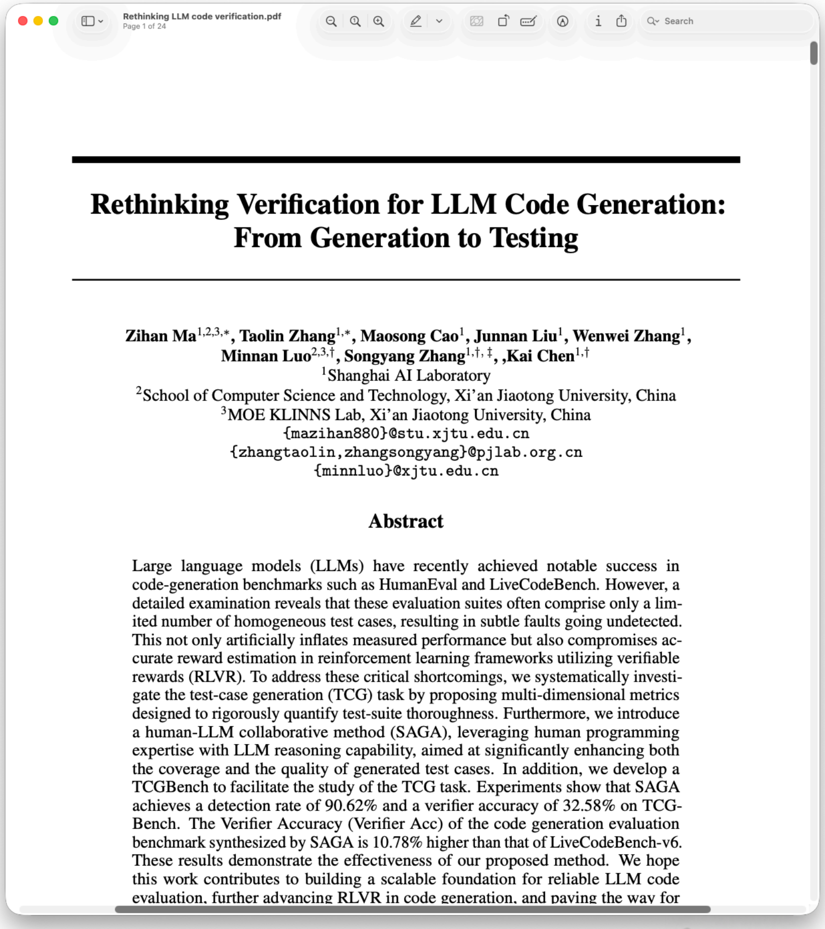
Abstract

Large language models (LLMs) have recently achieved notable success in code-generation benchmarks such as HumanEval and LiveCodeBench. However, a detailed examination reveals that these evaluation suites often comprise only a limited number of homogeneous test cases, resulting in subtle faults going undetected. This not only artificially inflates measured performance but also compromises accurate reward estimation in reinforcement learning frameworks utilizing verifiable rewards (RLVR). To address these critical shortcomings, we systematically investigate the test-case generation (TCG) task by proposing multi-dimensional metrics designed to rigorously quantify test-suite thoroughness. Furthermore, we introduce a human-LLM collaborative method (SAGA), leveraging human programming expertise with LLM reasoning capability, aimed at significantly enhancing both the coverage and the quality of generated test cases. In addition, we develop a TCGBench to facilitate the study of the TCG task. Experiments show that SAGA achieves a detection rate of 90.62% and a verifier accuracy of 32.58% on TCG-Bench. The Verifier Accuracy (Verifier Acc) of the code generation evaluation benchmark synthesized by SAGA is 10.78% higher than that of LiveCodeBench-v6. These results demonstrate the effectiveness of our proposed method. We hope this work contributes to building a scalable foundation for reliable LLM code evaluation, further advancing RLVR in code generation, and paving the way for

Source: https://arxiv.org/abs/2507.06920

Are LLMs Good At Testing Software?

- LLMs struggle to catch bugs
  - ~60% chance of identifying a buggy solution
- Inflated self-testing success
  - LLM-written tests have a high success rate against LLM-written source
- More tests doesn't catch more bugs
  - Detection of bugs don't improve after a point



Rethinking LLM code verification.pdf

Page 1 of 24

Search

Rethinking Verification for LLM Code Generation: From Generation to Testing

Zihan Ma[1,2,3,*], Taolin Zhang[1,*], Maosong Cao[1], Junnan Liu[1], Wenwei Zhang[1], Minnan Luo[2,3,†], Songyang Zhang[1,†,‡], ,Kai Chen[1,†]
[1]Shanghai AI Laboratory
[2]School of Computer Science and Technology, Xi'an Jiaotong University, China
[3]MOE KLINNS Lab, Xi'an Jiaotong University, China
{mazihan880}@stu.xjtu.edu.cn
{zhangtaolin,zhangsongyang}@pjlab.org.cn
{minnluo}@xjtu.edu.cn

Abstract

Large language models (LLMs) have recently achieved notable success in code-generation benchmarks such as HumanEval and LiveCodeBench. However, a detailed examination reveals that these evaluation suites often comprise only a limited number of homogeneous test cases, resulting in subtle faults going undetected. This not only artificially inflates measured performance but also compromises accurate reward estimation in reinforcement learning frameworks utilizing verifiable rewards (RLVR). To address these critical shortcomings, we systematically investigate the test-case generation (TCG) task by proposing multi-dimensional metrics designed to rigorously quantify test-suite thoroughness. Furthermore, we introduce a human-LLM collaborative method (SAGA), leveraging human programming expertise with LLM reasoning capability, aimed at significantly enhancing both the coverage and the quality of generated test cases. In addition, we develop a TCGBench to facilitate the study of the TCG task. Experiments show that SAGA achieves a detection rate of 90.62% and a verifier accuracy of 32.58% on TCG-Bench. The Verifier Accuracy (Verifier Acc) of the code generation evaluation benchmark synthesized by SAGA is 10.78% higher than that of LiveCodeBench-v6. These results demonstrate the effectiveness of our proposed method. We hope this work contributes to building a scalable foundation for reliable LLM code evaluation, further advancing RLVR in code generation, and paving the way for

Source: https://arxiv.org/abs/2507.06920

Speed At Odds With Quality

Speed At Odds With Quality

- Code generation is fast
    - AI writes code faster than we can read it

Speed At Odds With Quality

- Code generation is fast
  - AI writes code faster than we can read it
- But it is also probabilistic
  - If we can't read it, we can't verify its correctness

Speed At Odds With Quality

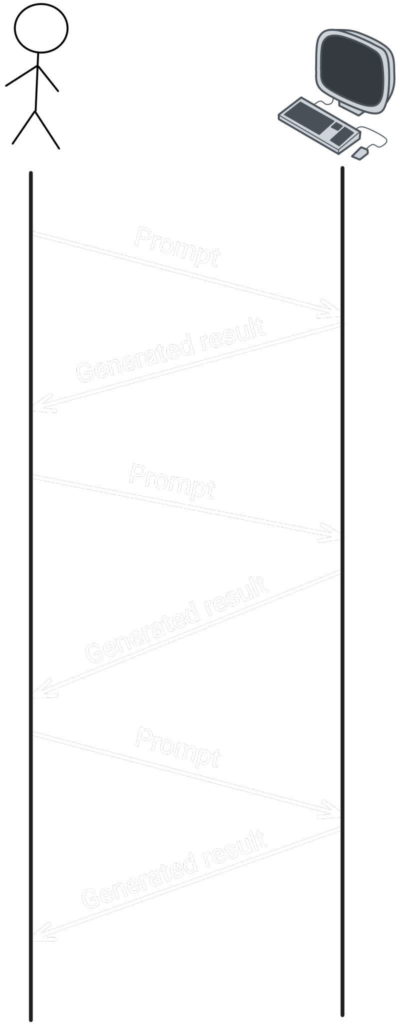
- Code generation is fast
  - AI writes code faster than we can read it
- But it is also probabilistic
  - If we can’t read it, we can’t verify its correctness
  - LLMs are not good at testing software

Speed At Odds With Quality

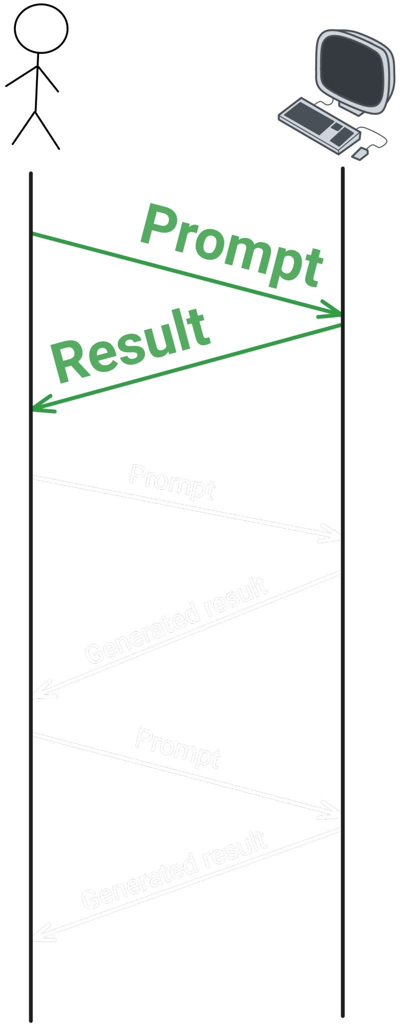
- Code generation is fast
  - AI writes code faster than we can read it
- But it is also probabilistic
  - If we can't read it, we can't verify its correctness
  - LLMs are not good at testing software
- Human comprehension
  - Our ability to understand and make meaning from information

Playing To Our Strengths

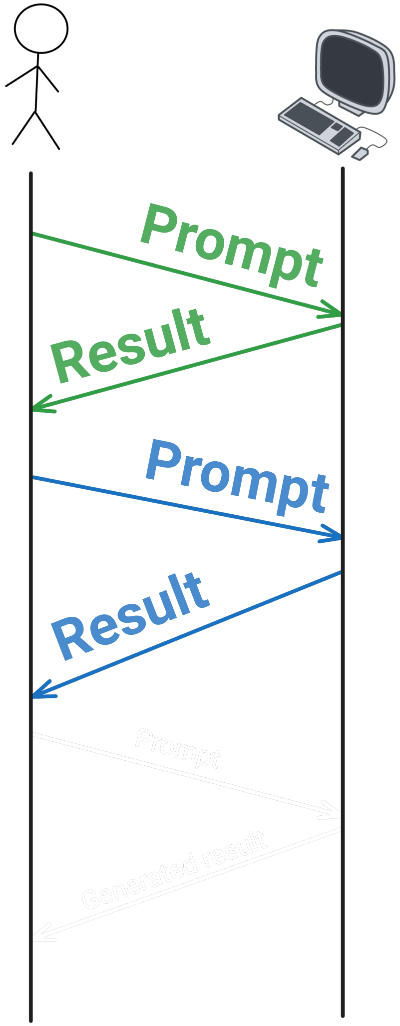
Playing To Our Strengths



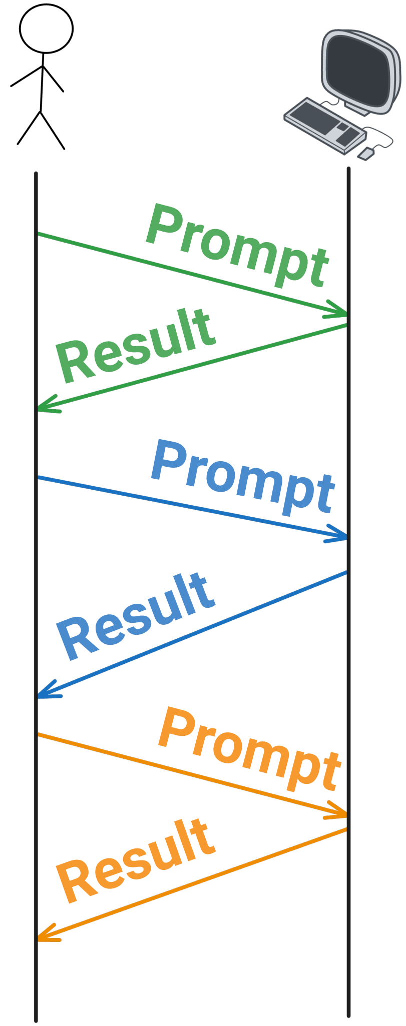
Playing To Our Strengths



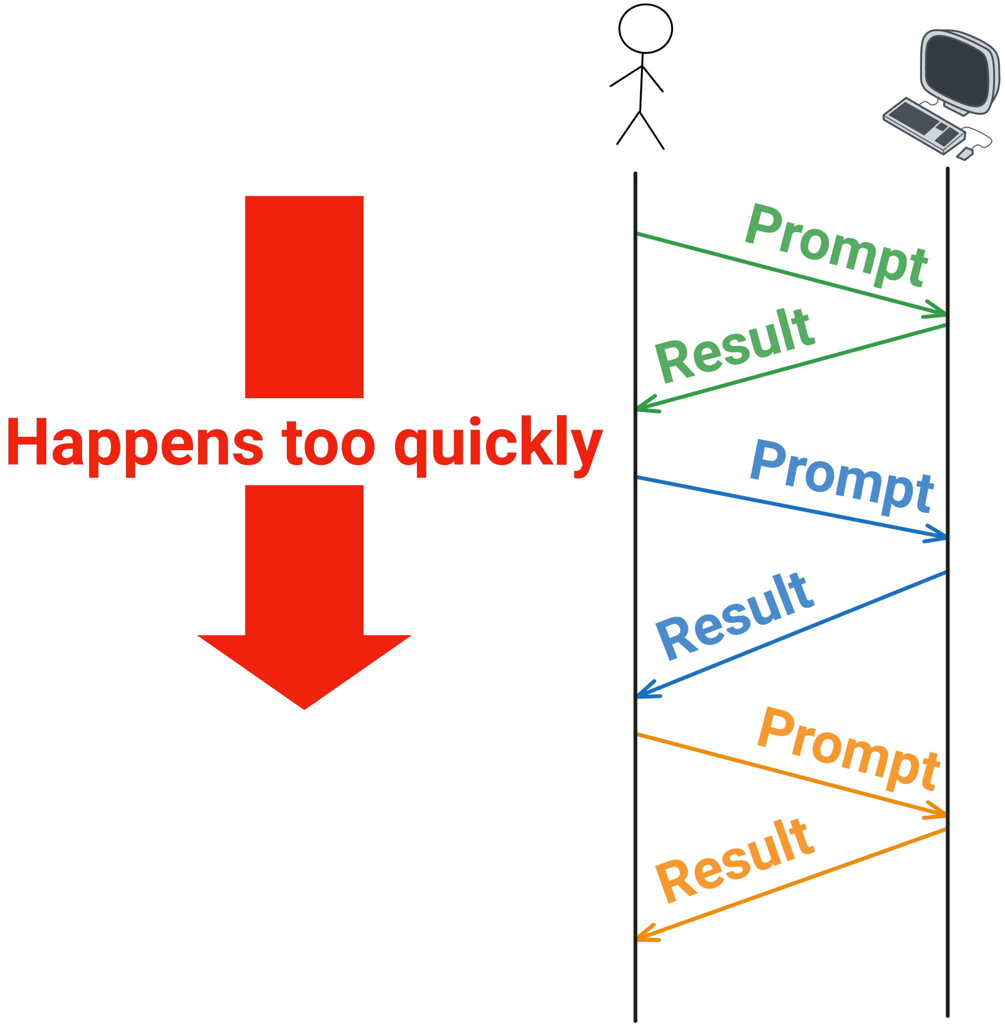
Playing To Our Strengths



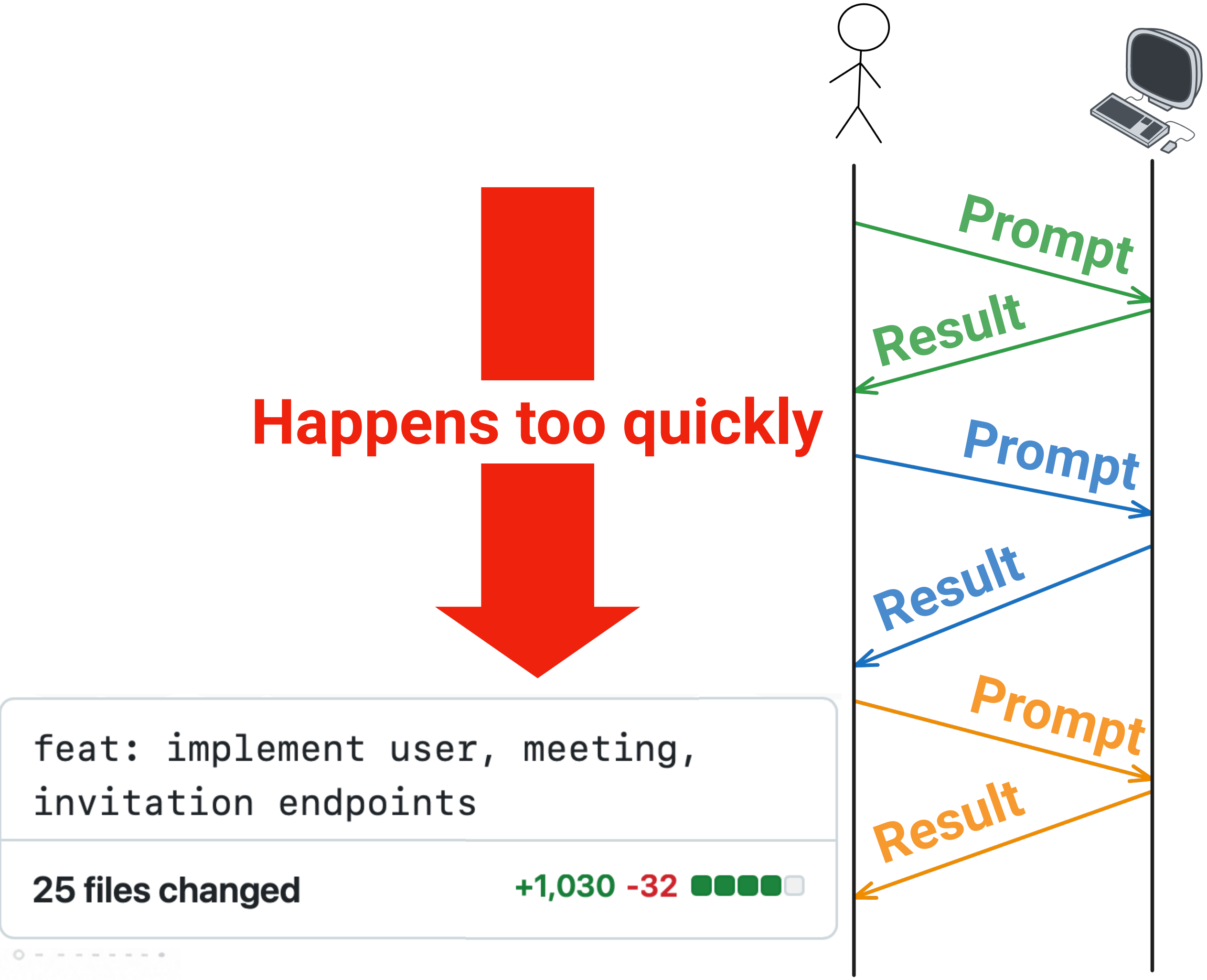
Playing To Our Strengths



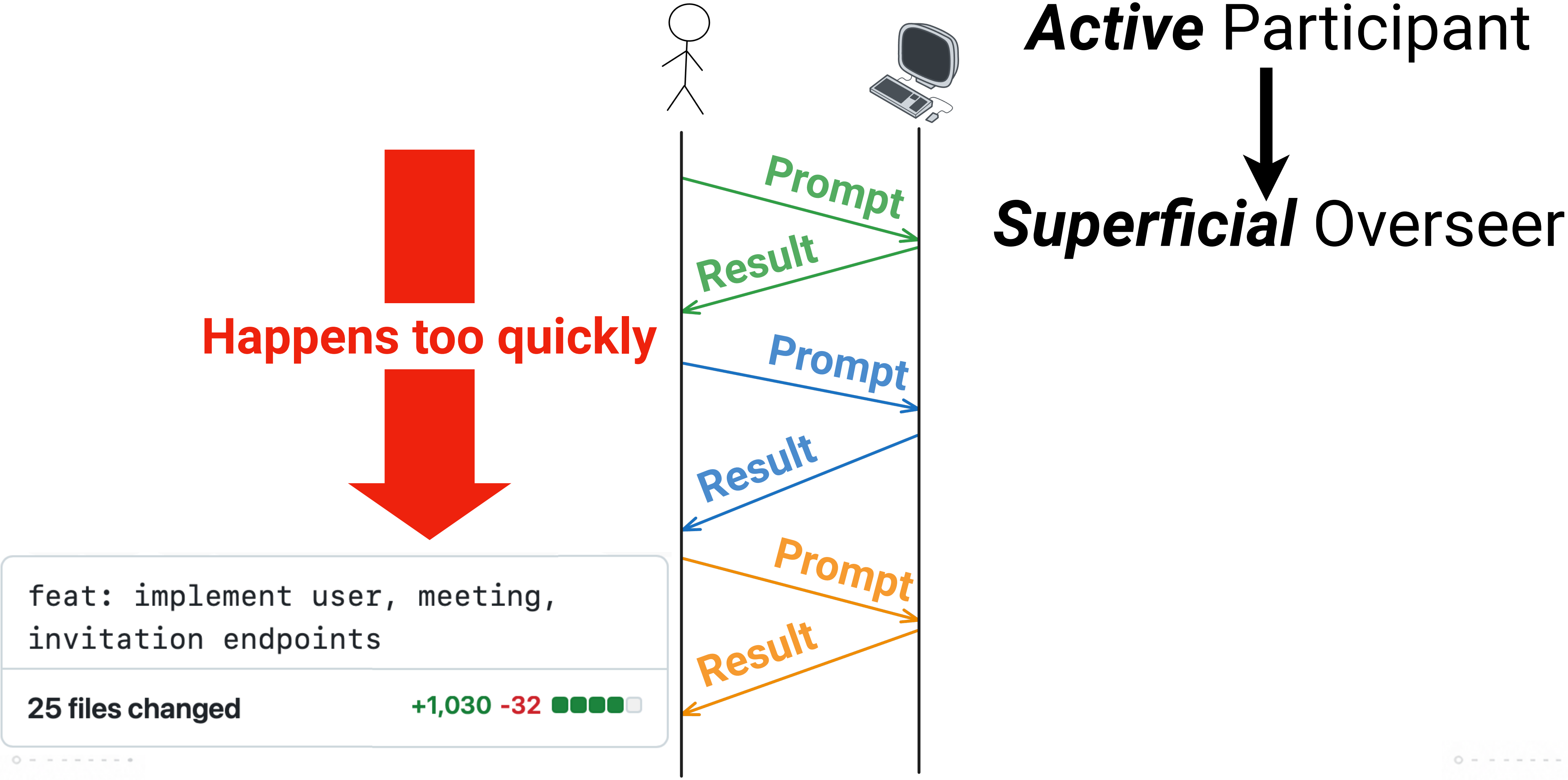
Playing To Our Strengths



Playing To Our Strengths

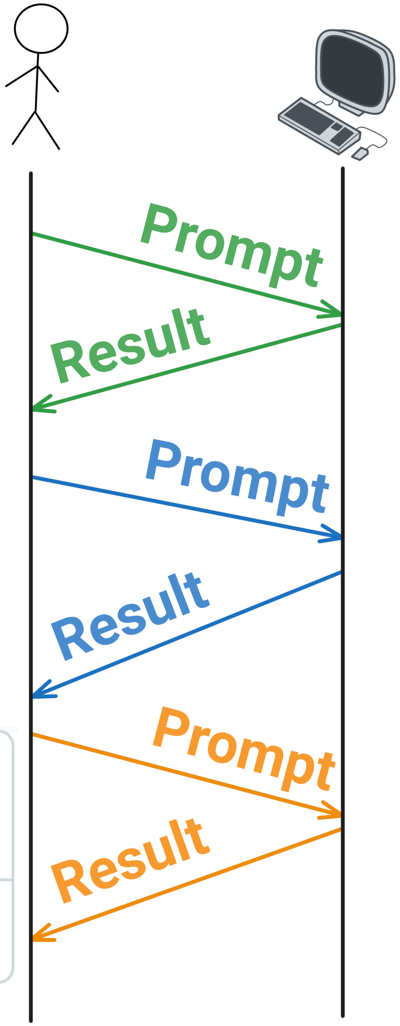


Playing To Our Strengths



Playing To Our Strengths

Happens too quickly




```
feat: implement user, meeting,
invitation endpoints
```

25 files changed +1,030 -32

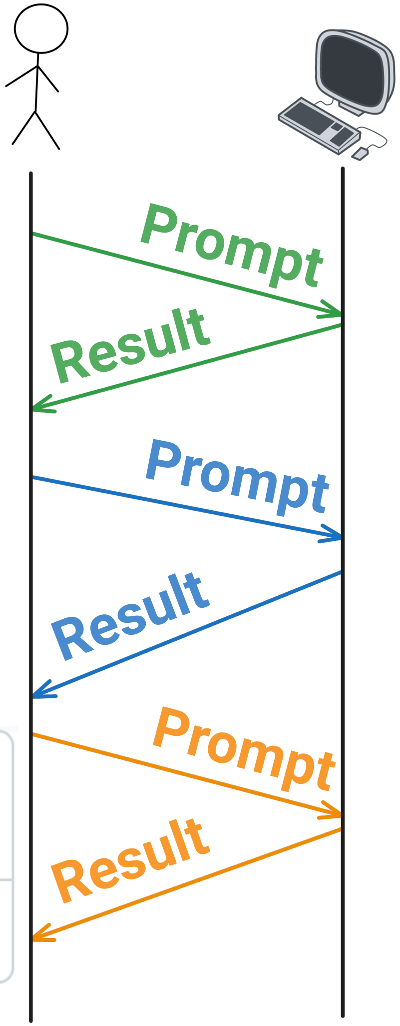
Active **Participant**

↓

Superficial **Overseer**

Overseeing does not **engage** our **cognitive** processes

Playing To Our Strengths



Happens too quickly

```
feat: implement user, meeting,
invitation endpoints
```

25 files changed +1,030 -32

Active Participant

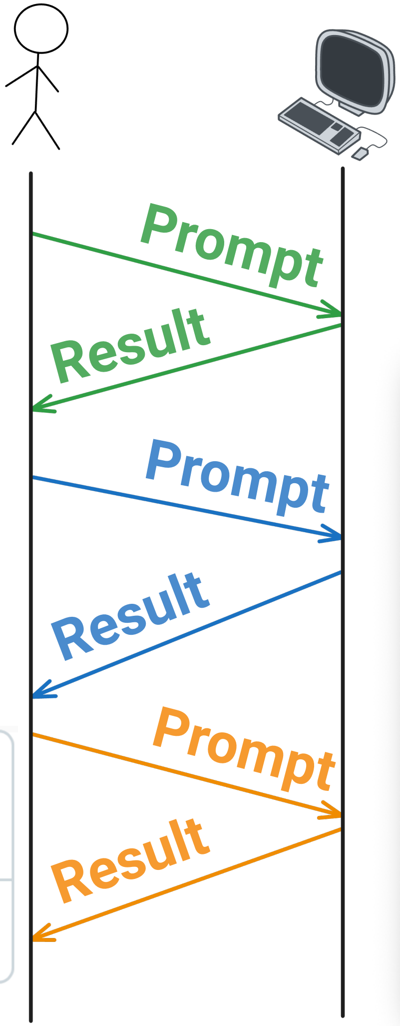
Superficial Overseer

Overseeing does not **engage** our **cognitive** processes

How did I miss that?

Playing To Our Strengths

Happens too quickly

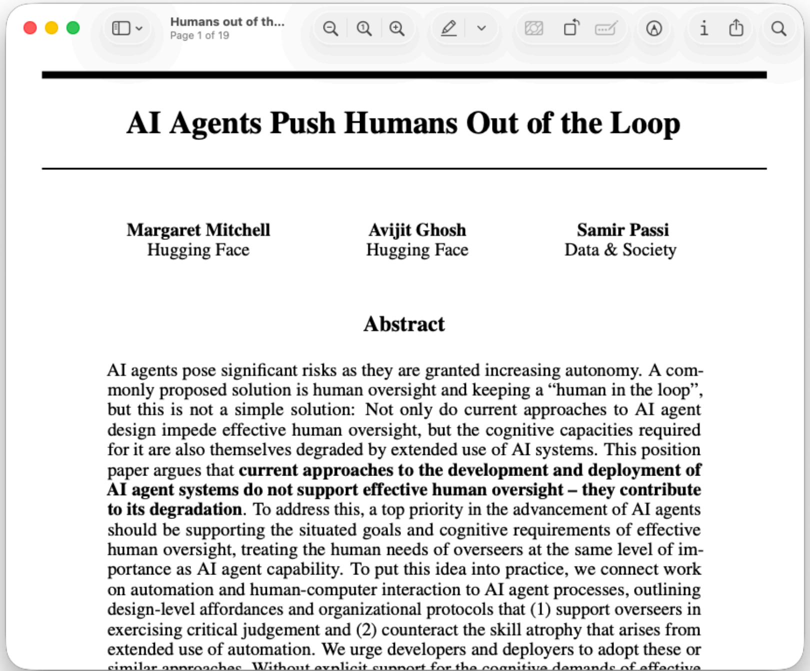



```
feat: implement user, meeting,
invitation endpoints
```

25 files changed +1,030 -32

Active **Participant**

Superficial **Overseer**



Humans out of th...
Page 1 of 19

AI Agents Push Humans Out of the Loop

Margaret Mitchell
Hugging Face

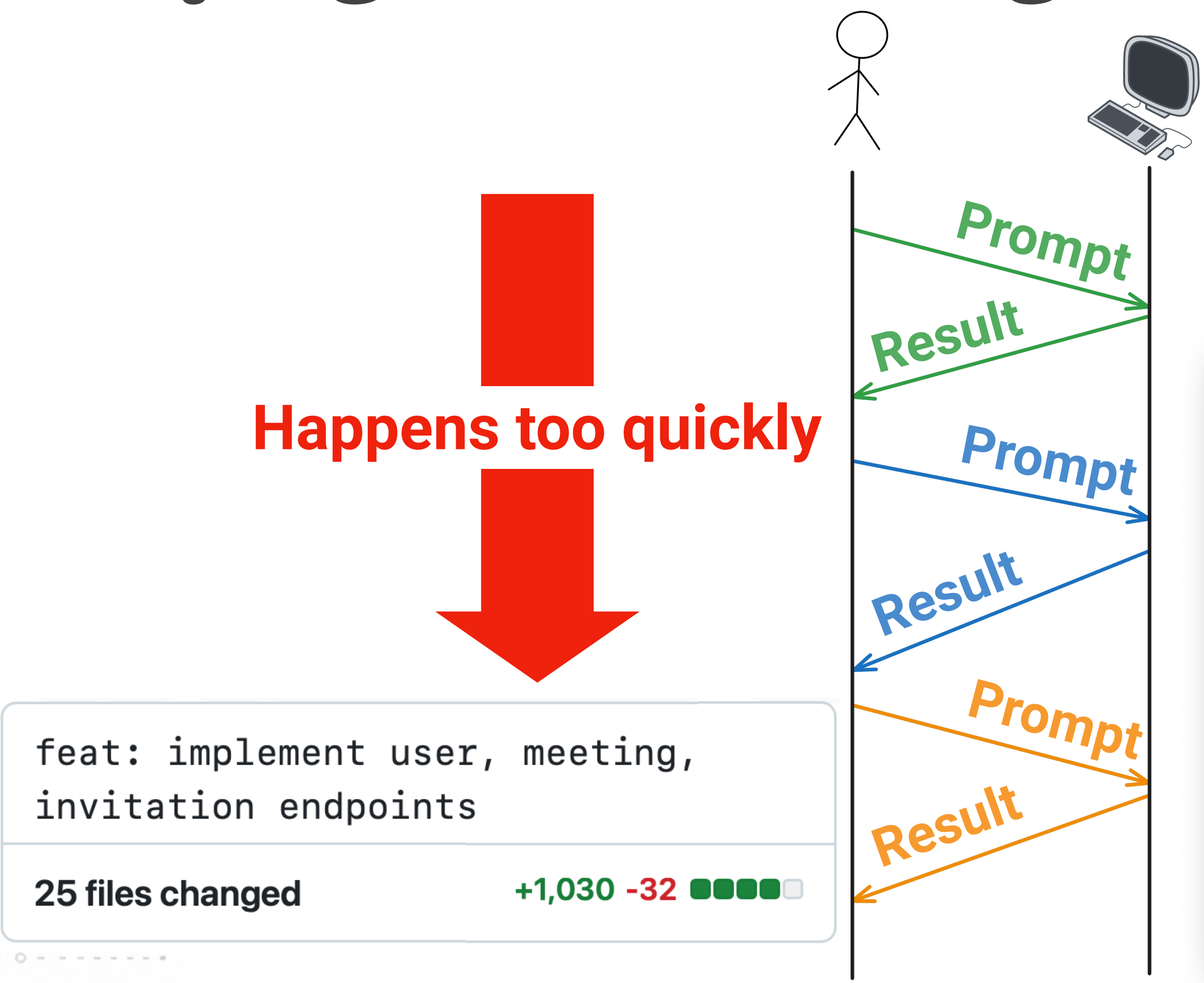
Avijit Ghosh
Hugging Face

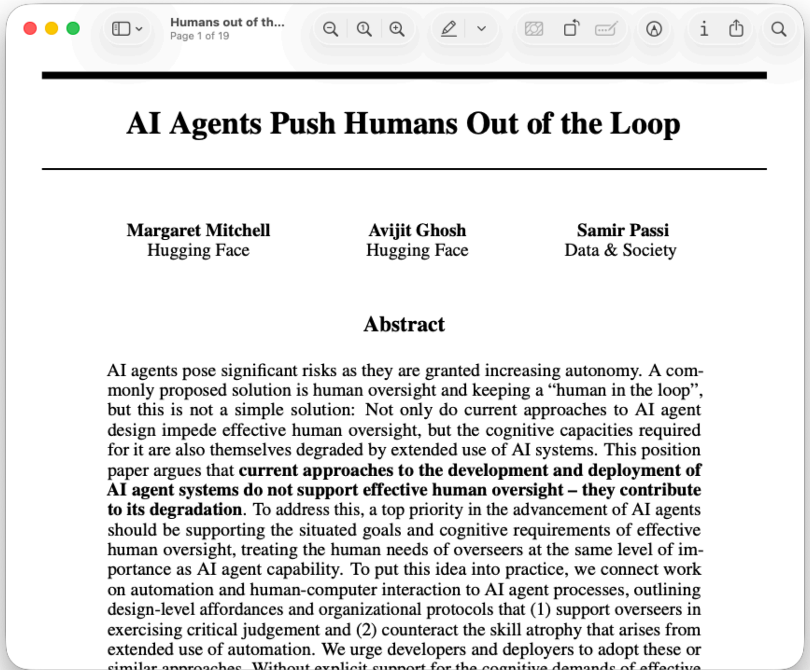
Samir Passi
Data & Society

Abstract

AI agents pose significant risks as they are granted increasing autonomy. A commonly proposed solution is human oversight and keeping a "human in the loop", but this is not a simple solution: Not only do current approaches to AI agent design impede effective human oversight, but the cognitive capacities required for it are also themselves degraded by extended use of AI systems. This position paper argues that **current approaches to the development and deployment of AI agent systems do not support effective human oversight – they contribute to its degradation**. To address this, a top priority in the advancement of AI agents should be supporting the situated goals and cognitive requirements of effective human oversight, treating the human needs of overseers at the same level of importance as AI agent capability. To put this idea into practice, we connect work on automation and human-computer interaction to AI agent processes, outlining design-level affordances and organizational protocols that (1) support overseers in exercising critical judgement and (2) counteract the skill atrophy that arises from extended use of automation. We urge developers and deployers to adopt these or similar approaches. Without explicit support for the cognitive demands of effective

Playing To Our Strengths





AI Agents Push Humans Out of the Loop

Margaret Mitchell
Hugging Face

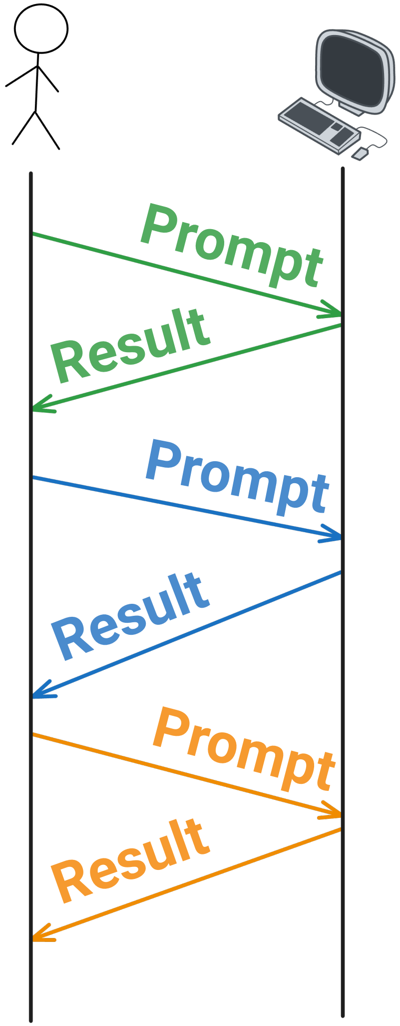
Avijit Ghosh
Hugging Face

Samir Passi
Data & Society

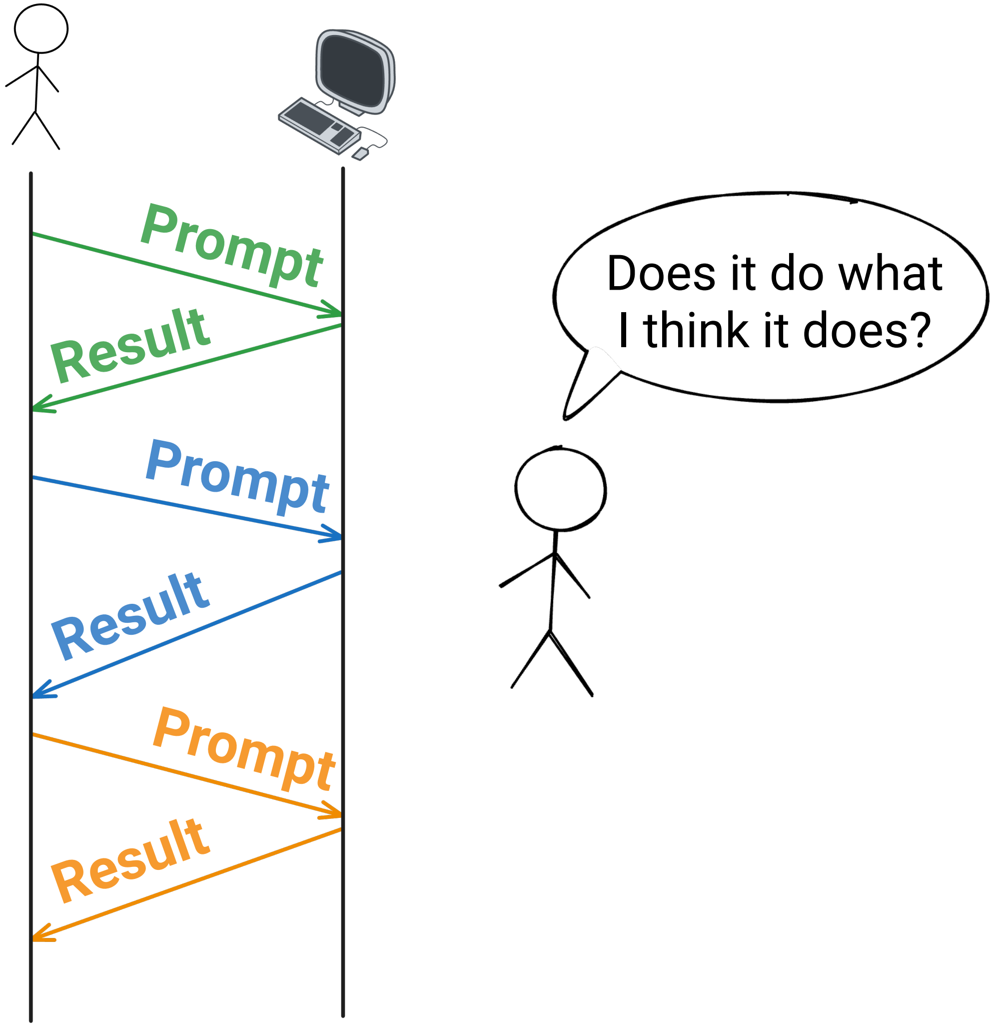
Abstract

AI agents pose significant risks as they are granted increasing autonomy. A commonly proposed solution is human oversight and keeping a "human in the loop", but this is not a simple solution: Not only do current approaches to AI agent design impede effective human oversight, but the cognitive capacities required for it are also themselves degraded by extended use of AI systems. This position paper argues that **current approaches to the development and deployment of AI agent systems do not support effective human oversight – they contribute to its degradation**. To address this, a top priority in the advancement of AI agents should be supporting the situated goals and cognitive requirements of effective human oversight, treating the human needs of overseers at the same level of importance as AI agent capability. To put this idea into practice, we connect work on automation and human-computer interaction to AI agent processes, outlining design-level affordances and organizational protocols that (1) support overseers in exercising critical judgement and (2) counteract the skill atrophy that arises from extended use of automation. We urge developers and deployers to adopt these or

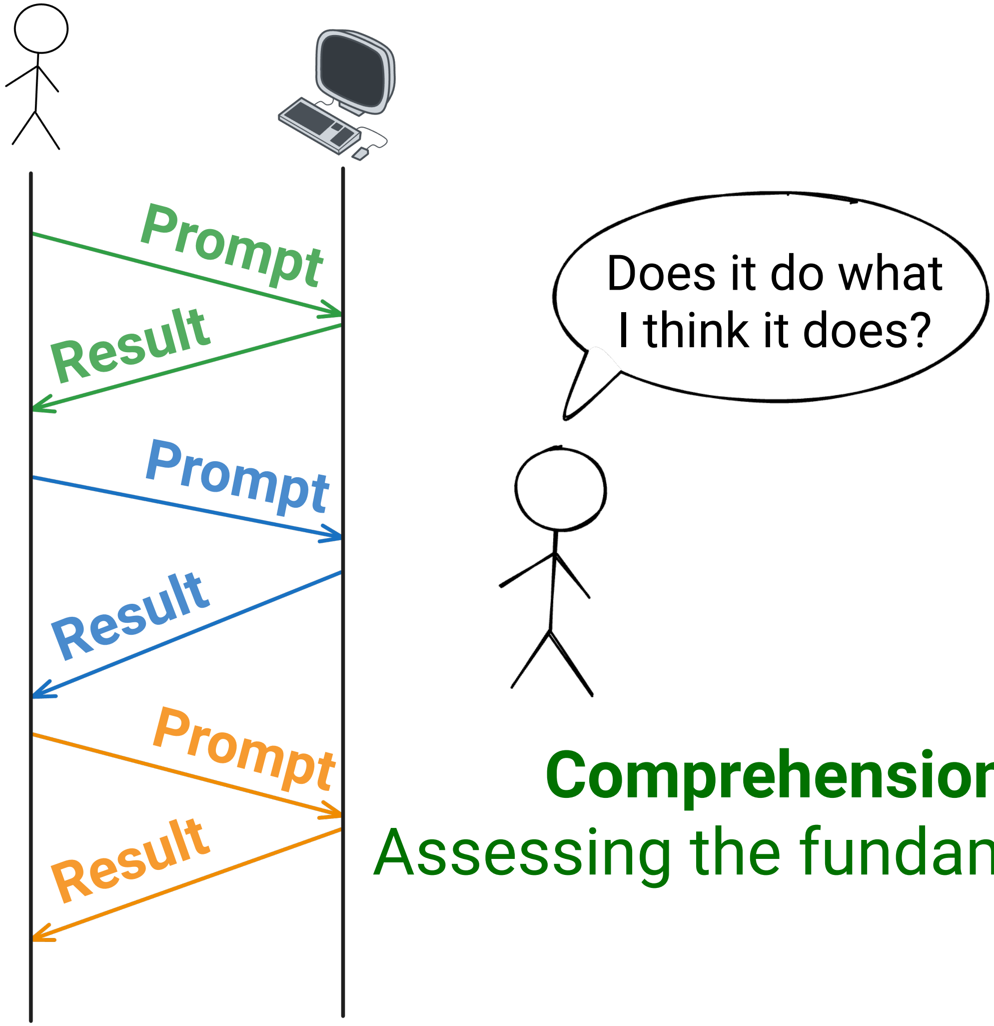
Playing To Our Strengths



Playing To Our Strengths



Playing To Our Strengths



Comprehension:
Assessing the fundamentals

Embedding Comprehension

into Software Engineering

Fundamental Properties

Fundamental Properties

- A user never receives stack traces

Fundamental Properties

- A user never receives stack traces
- Every element in a sorted list <= the next element

Fundamental Properties

- A user never receives stack traces
- Every element in a sorted list <= the next element
- One person cannot be in two meetings at the same time

Fundamental Properties

- A user never receives stack traces
- Every element in a sorted list <= the next element
- One person cannot be in two meetings at the same time
- The amount of money remains the same after account transfers

Fundamental Properties of Addition

Fundamental Properties of Addition

- Zero is the additive identity
  - $a + 0 = a$

Fundamental Properties of Addition

- Zero is the additive identity
  - $a + 0 = a$
- Adding something to its inverse gives back the identity
  - $a + (-a) = 0$

Fundamental Properties of Addition

- Zero is the additive identity
  - a + 0 = a
- Adding something to its inverse gives back the identity
  - a + (-a) = 0
- Addition is commutative
  - a + b = b + a

Fundamental Properties of Addition

- Zero is the additive identity
  - $a + 0 = a$
- Adding something to its inverse gives back the identity
  - $a + (-a) = 0$
- Addition is commutative
  - $a + b = b + a$
- Addition is associative
  - $(a + b) + c = a + (b + c)$

Testing the Properties of Addition

Testing the Properties of Addition

```
assertEquals(a, add(a, 0), "Additive Identity");
```

Testing the Properties of Addition

```
assertEquals(a, add(a, 0), "Additive Identity");
assertEquals(0, add(a, -a), "Additive Inverse");
```

Testing the Properties of Addition

```
assertEquals(a, add(a, 0), "Additive Identity");
assertEquals(0, add(a, -a), "Additive Inverse");
assertEquals(add(a, b), add(b, a), "Commutativity");
```

Testing the Properties of Addition

```
assertEquals(a, add(a, 0), "Additive Identity");
assertEquals(0, add(a, -a), "Additive Inverse");
assertEquals(add(a, b), add(b, a), "Commutativity");
assertEquals(add(1, add(a, b)), add(add(1, a), b), "Associativity");
```

Testing the Properties of Addition

```
                                @ForAll double a, @ForAll double b
assertEquals(a, add(a, 0), "Additive Identity");
assertEquals(0, add(a, -a), "Additive Inverse");
assertEquals(add(a, b), add(b, a), "Commutativity");
assertEquals(add(1, add(a, b)), add(add(1, a), b), "Associativity");
```

Testing the Properties of Addition

```java
void propertyBasedAdditionTest(@ForAll double a, @ForAll double b) {
   assertEquals(a, add(a, 0), "Additive Identity");
   assertEquals(0, add(a, -a), "Additive Inverse");
   assertEquals(add(a, b), add(b, a), "Commutativity");
   assertEquals(add(1, add(a, b)), add(add(1, a), b), "Associativity");
}
```

Testing the Properties of Addition

```
@Property
void propertyBasedAdditionTest(@ForAll double a, @ForAll double b) {
   assertEquals(a, add(a, 0), "Additive Identity");
   assertEquals(0, add(a, -a), "Additive Inverse");
   assertEquals(add(a, b), add(b, a), "Commutativity");
   assertEquals(add(1, add(a, b)), add(add(1, a), b), "Associativity");
}
```

Testing the Properties of Addition

```
@Property
void propertyBasedAdditionTest(@ForAll double a, @ForAll double b) {
   assertEquals(a, add(a, 0), "Additive Identity");
   assertEquals(0, add(a, -a), "Additive Inverse");
   assertEquals(add(a, b), add(b, a), "Commutativity");
   assertEquals(add(1, add(a, b)), add(add(1, a), b), "Associativity");
}

@Test
void exampleBasedAdditionTest() {
   assertEquals(1.5, add(1.5, 0));
   assertEquals(-101.8, add(-100, -1.8));
   assertEquals(99, add(-1, 100));
   assertEquals(230.5, add(130.5, 100));
}
```

Testing the Properties of Addition

```java
@Property
void propertyBasedAdditionTest(@ForAll double a, @ForAll double b) {
   assertEquals(a, add(a, 0), "Additive Identity");
   assertEquals(0, add(a, -a), "Additive Inverse");
   assertEquals(add(a, b), add(b, a), "Commutativity");
   assertEquals(add(1, add(a, b)), add(add(1, a), b), "Associativity");
}

@Test
void exampleBasedAdditionTest() {
   assertEquals(1.5, add(1.5, 0));
   assertEquals(-101.8, add(-100, -1.8));
   assertEquals(99, add(-1, 100));
   assertEquals(230.5, add(130.5, 100));
}
```

Sampled values are tested

Testing the Properties of Addition

```
@Property
void propertyBasedAdditionTest(@ForAll double a, @ForAll double b) {
    assertEquals(a, add(a, 0), "Additive Identity");
    assertEquals(0, add(a, -a), "Additive Inverse");
    assertEquals(add(a, b), add(b, a), "Commutativity");
    assertEquals(add(1, add(a, b)), add(add(1, a), b), "Associativity");
}

@Test
void exampleBasedAdditionTest() {
    assertEquals(1.5, add(1.5, 0));
    assertEquals(-101.8, add(-100, -1.8));
    assertEquals(99, add(-1, 100));
    assertEquals(230.5, add(130.5, 100));
}
```

The problem space is explored

Sampled values are tested

Testing the Properties of Addition

```
@Property
void propertyBasedAdditionTest(@ForAll double a, @ForAll double b) {
   assertEquals(a, add(a, 0), "Additive Identity");
   assertEquals(0, add(a, -a), "Additive Inverse");
   assertEquals(add(a, b), add(b, a), "Commutativity");
   assertEquals(add(1, add(a, b)), add(add(1, a), b), "Associativity");
}

@Test
void exampleBasedAdditionTest() {
   assertEquals(1.5, add(1.5, 0));
   assertEquals(-101.8, add(-100, -1.8));
   assertEquals(99, add(-1, 100));
   assertEquals(230.5, add(130.5, 100));
}
```

Verifies for multiple values of ***a*** and ***b***

Testing the Properties of Addition

```
@Property
void propertyBasedAdditionTest(@ForAll double a, @ForAll double b) {
    assertEquals(a, add(a, 0), "Additive Identity");
    assertEquals(0, add(a, -a), "Additive Inverse");
    assertEquals(add(a, b), add(b, a), "Commutativity");
    assertEquals(add(1, add(a, b)), add(add(1, a), b), "Associativity");
}
```

Verifies for multiple values of ***a*** and ***b***

```
@Test
void exampleBasedAdditionTest() {
    assertEquals(1.5, add(1.5, 0));
    assertEquals(-101.8, add(-100, -1.8));
    assertEquals(99, add(-1, 100));
    assertEquals(230.5, add(130.5, 100));
}
```

Only verifies for 1.5 and 0

Testing the Properties of Addition

```java
@Property
void propertyBasedAdditionTest(@ForAll double a, @ForAll double b) {
    assertEquals(a, add(a, 0), "Additive Identity");
    assertEquals(0, add(a, -a), "Additive Inverse");
    assertEquals(add(a, b), add(b, a), "Commutativity");
    assertEquals(add(1, add(a, b)), add(add(1, a), b), "Associativity");
}

@Test
void exampleBasedAdditionTest() {
    assertEquals(1.5, add(1.5, 0));
    assertEquals(-101.8, add(-100, -1.8));
    assertEquals(99, add(-1, 100));
    assertEquals(230.5, add(130.5, 100));
}
```

Testing the Properties of Addition

```
@Property
void proper
    assertEq
    assertEq
    assertEq
    assertEq
}

@Test
void exampl
    assertEq
    assertEq
    assertEq
    assertEq
}
```

```
                               |-----------jqwik-----------
tries = 285                    | # of calls to property
checks = 285                   | # of not rejected calls
generation = RANDOMIZED        | parameters are randomly generated
...

Original Sample
---------------
  a: 15.6
  b: -1.49

  Original Error
  --------------
  org.opentest4j.AssertionFailedError:
    Associativity ⟹ expected: <15.11> but was: <15.110000000000001>
```

Testing the Properties of Addition

```
@Property
void proper
    assertEq
    assertEq
    assertEq
    assertEq
}

@Test
void exampl
    assertEq
    assertEq
    assertEq
    assertEq
}
```

```
                              |-----------jqwik-----------
tries = 285                   | # of calls to property
checks = 285                  | # of not rejected calls
generation = RANDOMIZED       | parameters are randomly generated
...

Original Sample
---------------
  a: 15.6
  b: -1.49

  Original Error
  --------------
  org.opentest4j.AssertionFailedError:
    Associativity ==> expected: <15.11> but was: <15.110000000000001>
```

Testing the Properties of Addition

```
@Property
void proper
    assertEq
    assertEq
    assertEq
    assertEq
}

@Test
void exampl
    assertEq
    assertEq
    assertEq
    assertEq
}
```

```
                              |-----------jqwik-----------
tries = 285                   | # of calls to property
checks = 285                  | # of not rejected calls
generation = RANDOMIZED       | parameters are randomly generated
...

Original Sample
---------------
  a: 15.6
  b: -1.49

  Original Error
  --------------
  org.opentest4j.AssertionFailedError:
    Associativity ⟹ expected: <15.11> but was: <15.110000000000001>
```

Testing the Properties of Addition

```
@Property
void proper
    assertEq
    assertEq
    assertEq
    assertEq
}

@Test
void exampl
    assertEq
    assertEq
    assertEq
    assertEq
}
```

```
                                       |-----------jqwik-----------
tries = 285                            | # of calls to property
checks = 285                           | # of not rejected calls
generation = RANDOMIZED                | parameters are randomly generated
...

Original Sample
---------------
  a: 15.6
  b: -1.49

  Original Error
  --------------
  org.opentest4j.AssertionFailedError:
    Associativity ==> expected: <15.11> but was: <15.110000000000001>
```

```
jshell>
```

Testing the Properties of Addition

```
@Property
void proper
    assertEq
    assertEq
    assertEq
    assertEq
}

@Test
void exampl
    assertEq
    assertEq
    assertEq
    assertEq
}
```

```
                                     |-----------jqwik-----------
tries = 285                          | # of calls to property
checks = 285                         | # of not rejected calls
generation = RANDOMIZED              | parameters are randomly generated
...

Original Sample
---------------
  a: 15.6
  b: -1.49

  Original Error
  --------------
  org.opentest4j.AssertionFailedError:
    Associativity ==> expected: <15.11> but was: <15.110000000000001>
```

```
jshell> 1 + (15.6 + -1.49)
$1 ==> 15.11
```

Testing the Properties of Addition

```
@Property
void proper
    assertEq
    assertEq
    assertEq
    assertEq
}

@Test
void exampl
    assertEq
    assertEq
    assertEq
    assertEq
}
```

```
                              |-----------jqwik-----------
tries = 285                   | # of calls to property
checks = 285                  | # of not rejected calls
generation = RANDOMIZED       | parameters are randomly generated
...

Original Sample
---------------
  a: 15.6
  b: -1.49

  Original Error
  --------------
  org.opentest4j.AssertionFailedError:
    Associativity ==> expected: <15.11> but was: <15.110000000000001>
```

```
jshell> 1 + (15.6 + -1.49)
$1 ==> 15.11

jshell> (1 + 15.6) + -1.49
$2 ==> 15.110000000000001
```

Testing the Properties of Addition

```
@Property
void proper
    assertEq
    assertEq
    assertEq
    assertEq
}

@Test
void exampl
    assertEq
    assertEq
    assertEq
    assertEq
}
```

```
                                 |-----------jqwik-----------
tries = 285                      | # of calls to property
checks = 285                     | # of not rejected calls
generation = RANDOMIZED          | parameters are randomly generated
...

Original Sample
---------------
  a: 15.6
  b: -1.49

  Original Error
  --------------
  org.opentest4j.AssertionFailedError:
    Associativity ==> expected: <15.11> but was: <15.110000000000001>
```

```
jshell> 1 + (15.6 + -1.49)
$1 ==> 15.11

jshell> (1 + 15.6) + -1.49
$2 ==> 15.110000000000001

jshell> 15.6d == 15.6f
$3 ==> false
```

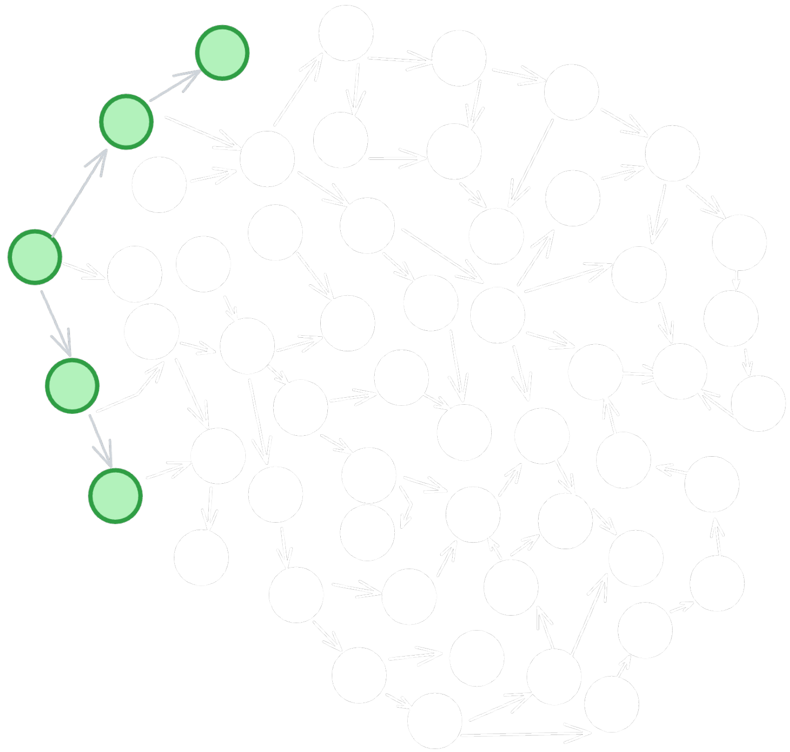
Testing the Properties of Addition

```java
@Property
void propertyBasedAdditionTest(@ForAll double a, @ForAll double b) {
    assertEquals(a, add(a, 0), "Additive Identity");
    assertEquals(0, add(a, -a), "Additive Inverse");
    assertEquals(add(a, b), add(b, a), "Commutativity");
    assertEquals(add(1, add(a, b)), add(add(1, a), b), "Associativity");
}
```

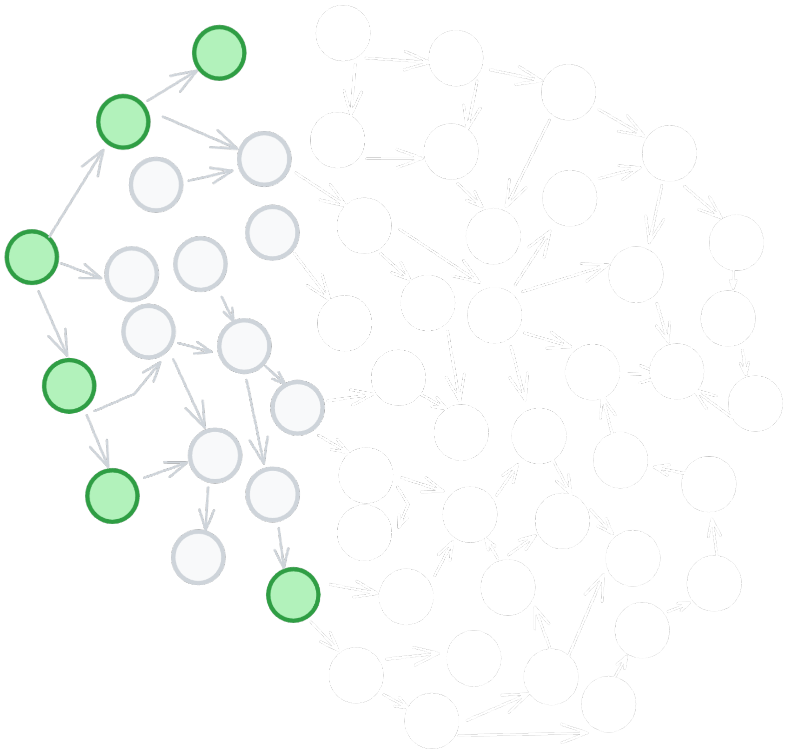

```java
@Test
void exampleBasedAdditionTest() {
    assertEquals(1.5, add(1.5, 0));
    assertEquals(-101.8, add(-100, -1.8));
    assertEquals(99, add(-1, 100));
    assertEquals(230.5, add(130.5, 100));
}
```

Examples → Exploration

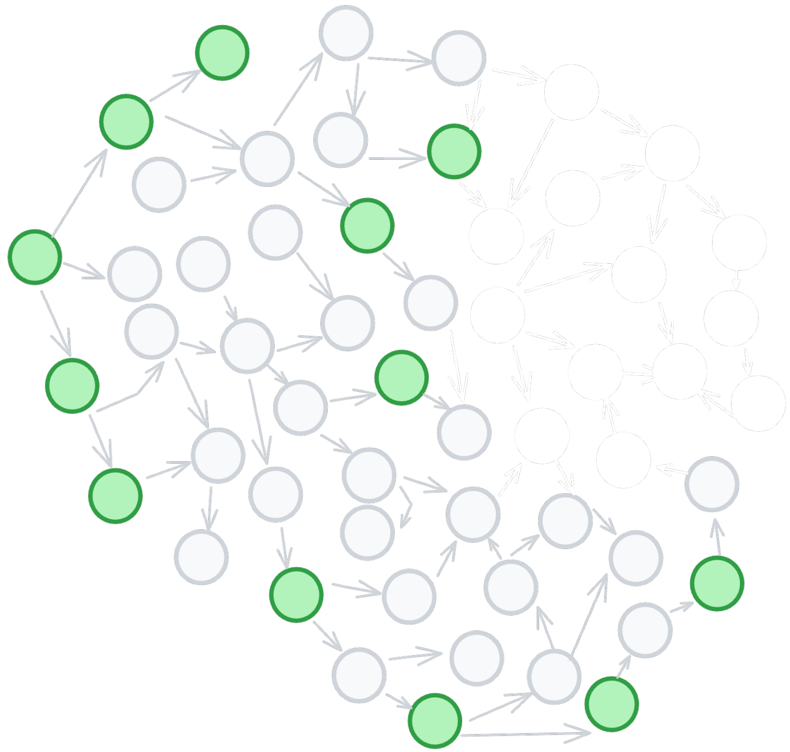
Examples → Exploration



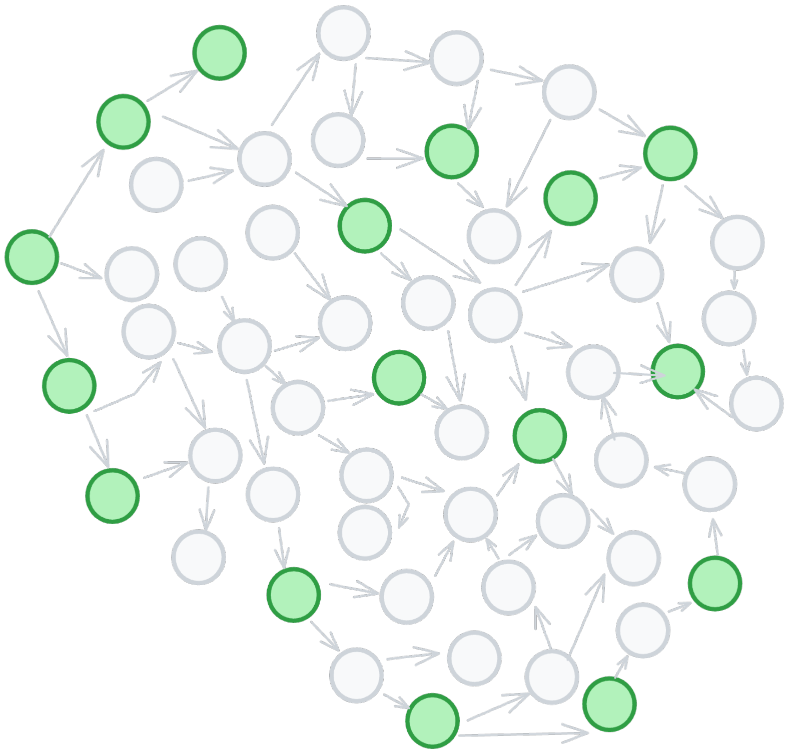
Examples → Exploration



Examples → Exploration

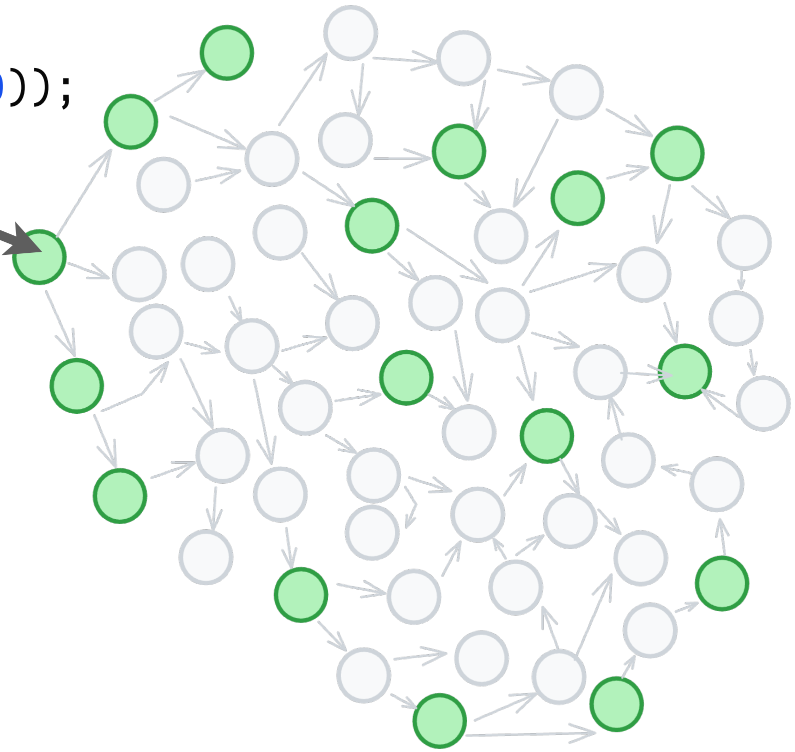


Examples → Exploration



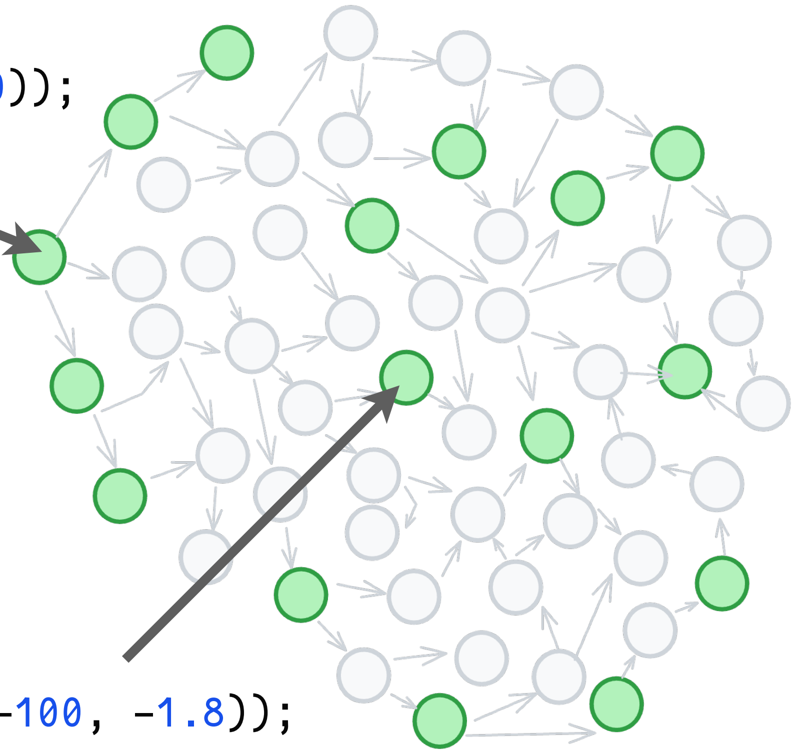
Examples → Exploration

```
assertEquals(99, add(-1, 100));
```



Examples → Exploration

```
assertEquals(99, add(-1, 100));
```

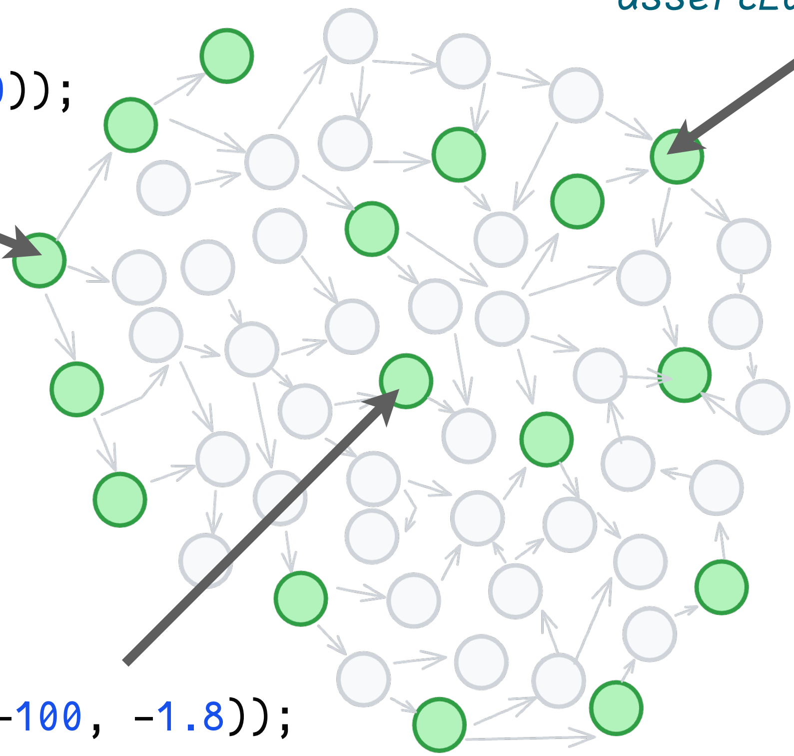


```
assertEquals(-101.8, add(-100, -1.8));
```

Examples → Exploration

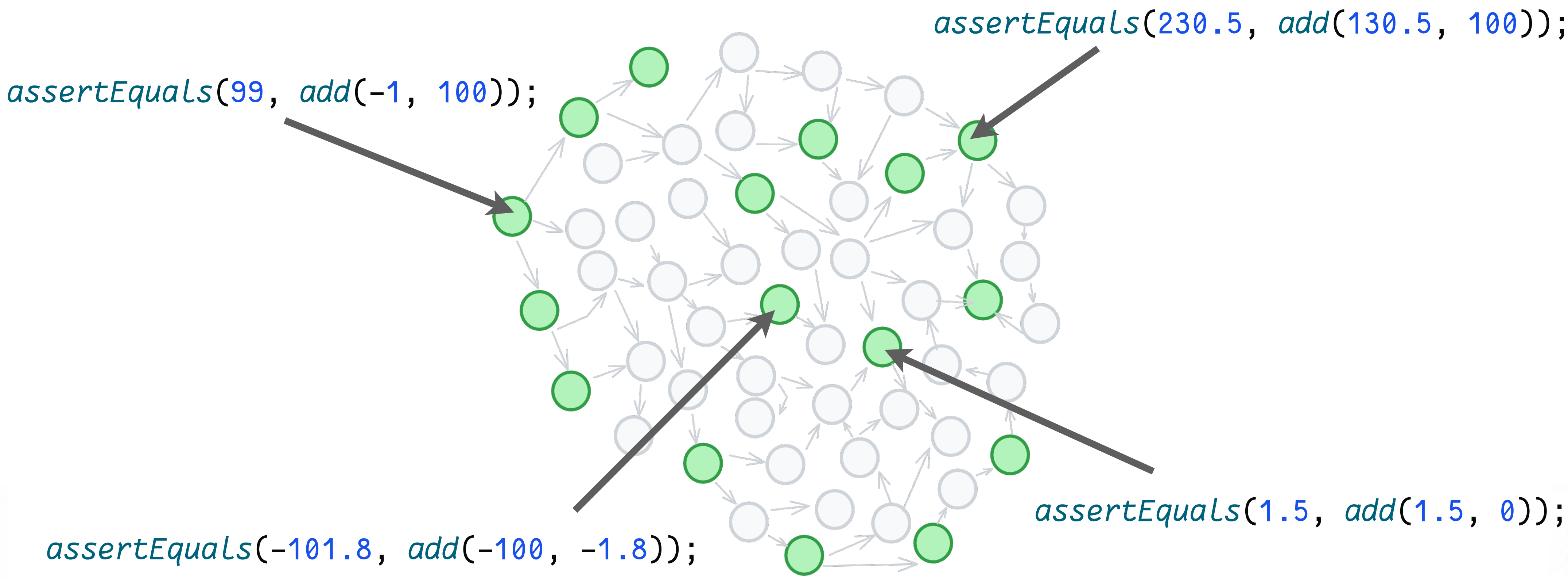
```
assertEquals(230.5, add(130.5, 100));
```

```
assertEquals(99, add(-1, 100));
```



```
assertEquals(-101.8, add(-100, -1.8));
```

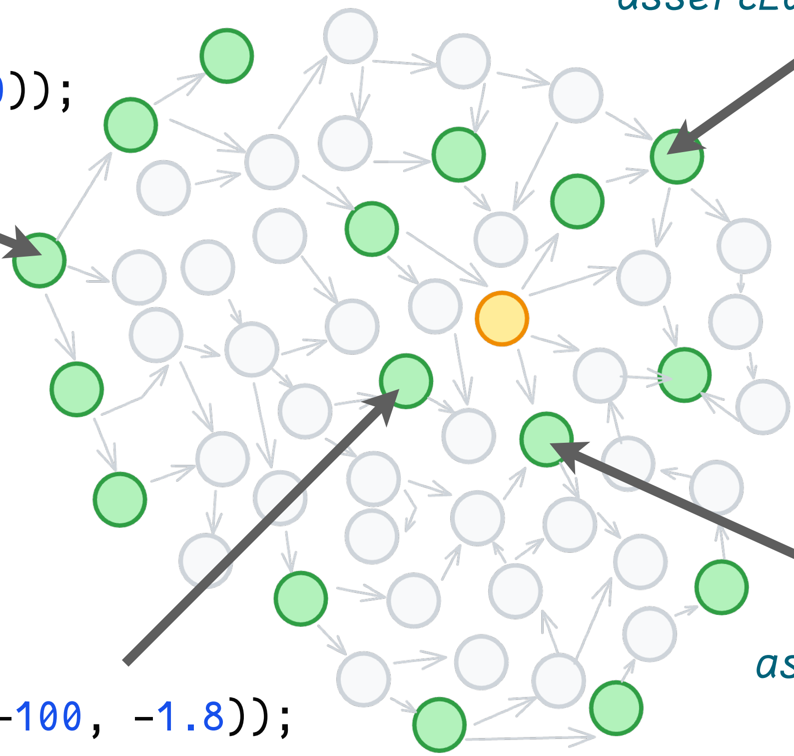
Examples → Exploration



Examples → Exploration

```
assertEquals(230.5, add(130.5, 100));
```

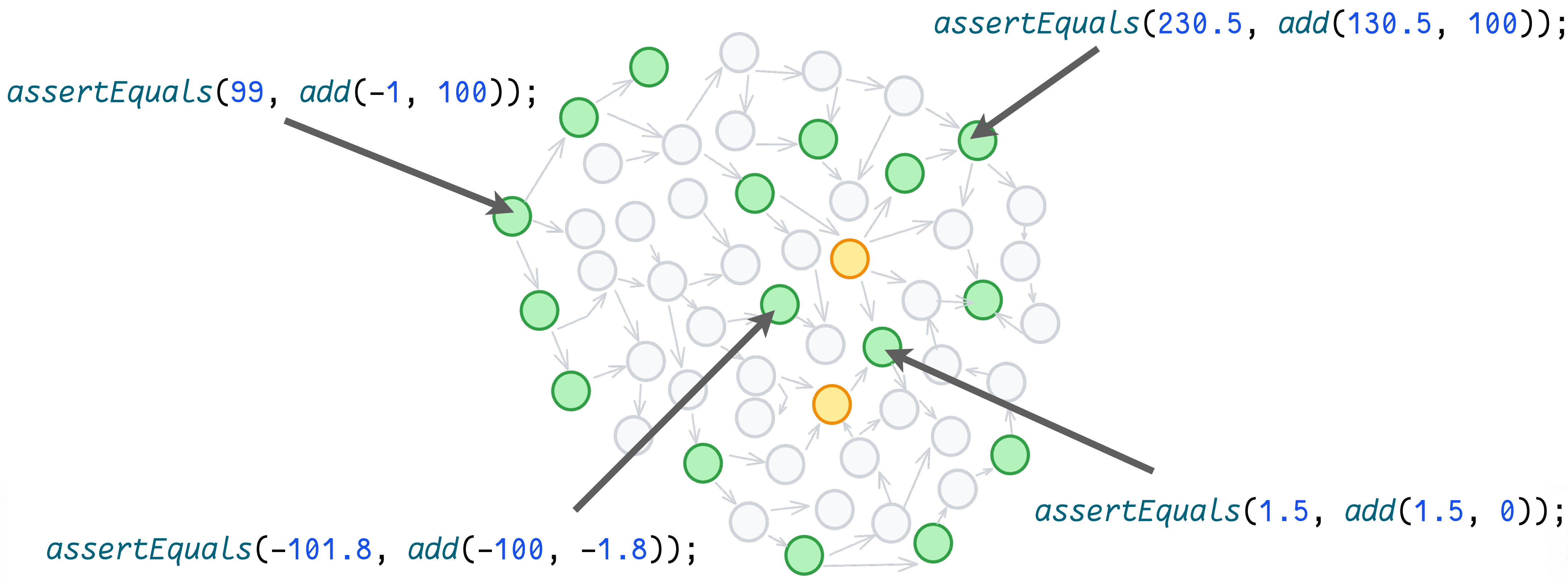
```
assertEquals(99, add(-1, 100));
```



```
assertEquals(1.5, add(1.5, 0));
```

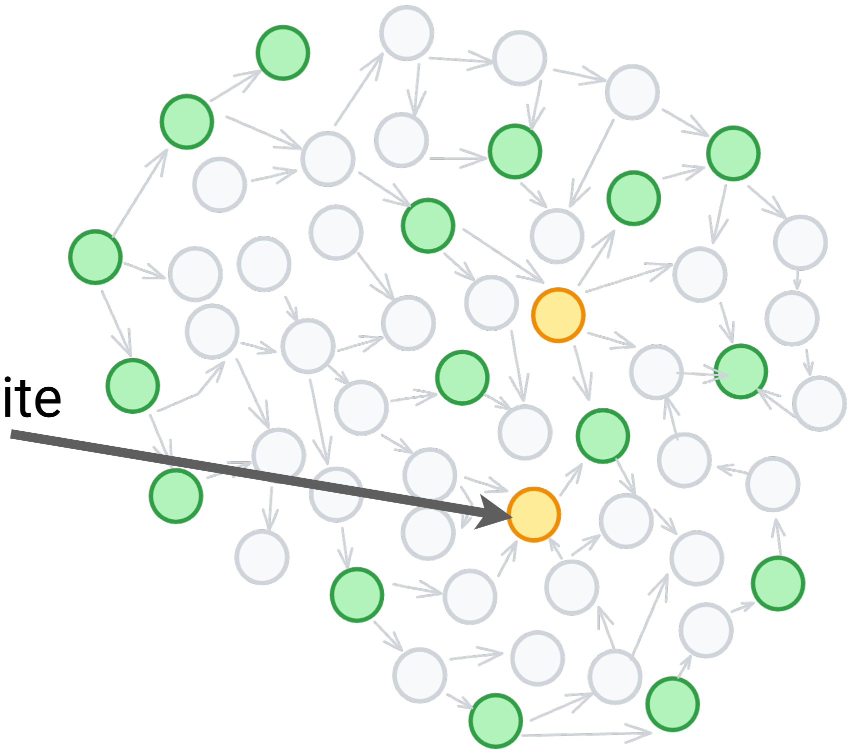
```
assertEquals(-101.8, add(-100, -1.8));
```

Examples → Exploration



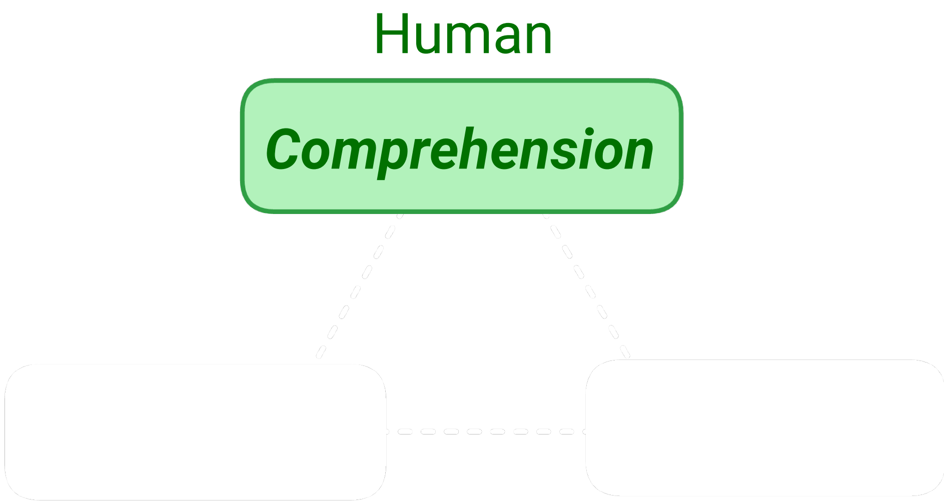
Examples → Exploration

What if the LLM didn't write
a test case for this?

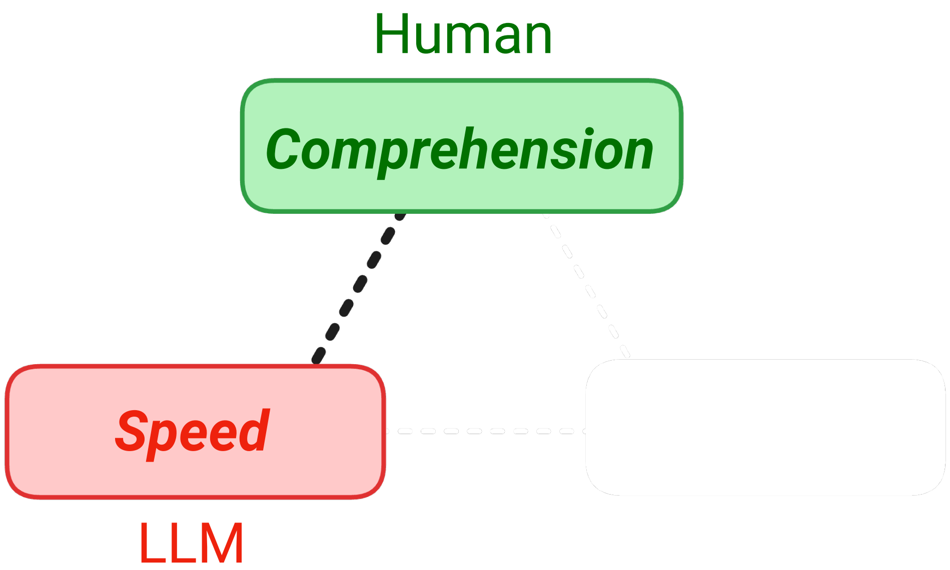


Examples → Exploration

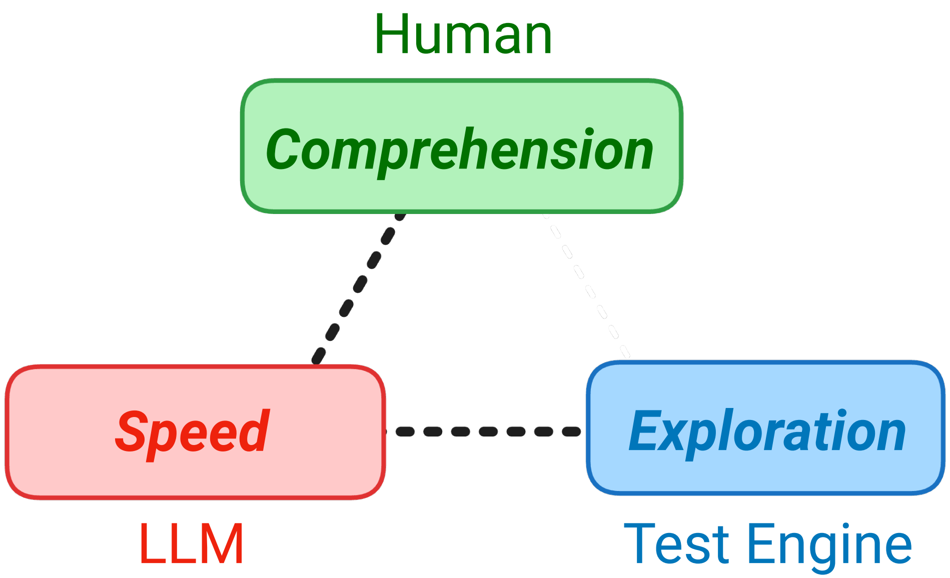
Examples → Exploration



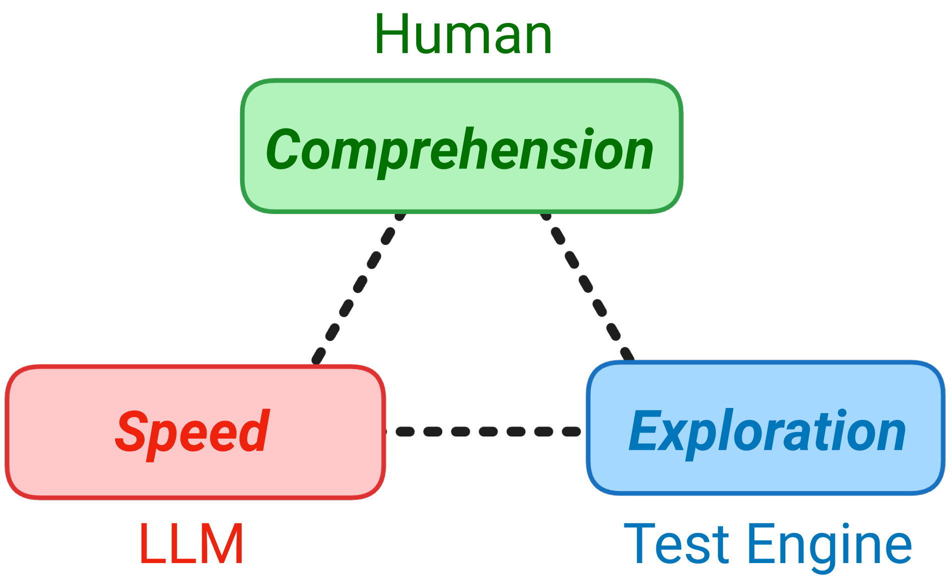
Examples → Exploration



Examples → Exploration



Examples → Exploration

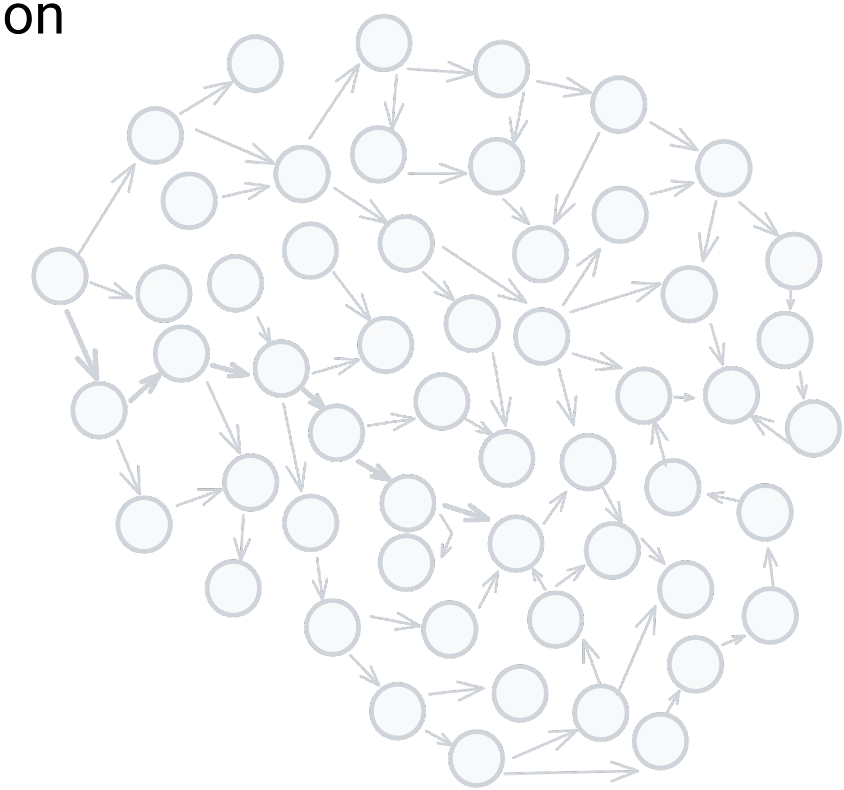


Examples → Exploration

LLMs generate a solution

Examples → Exploration

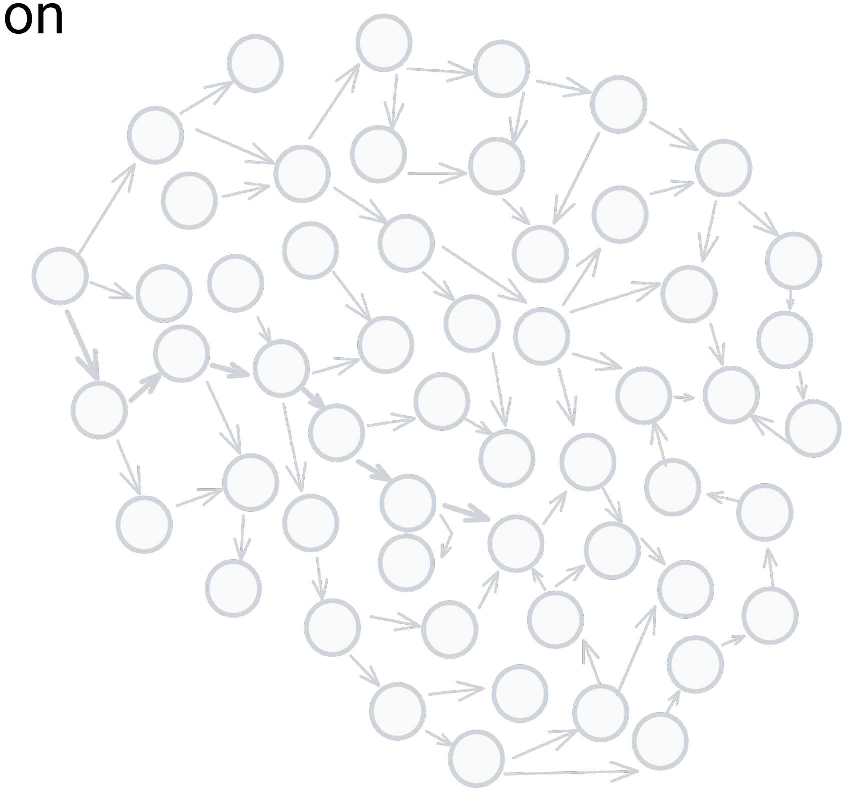
LLMs generate a solution



Examples → Exploration

LLMs generate a solution

Humans define what
must always hold true

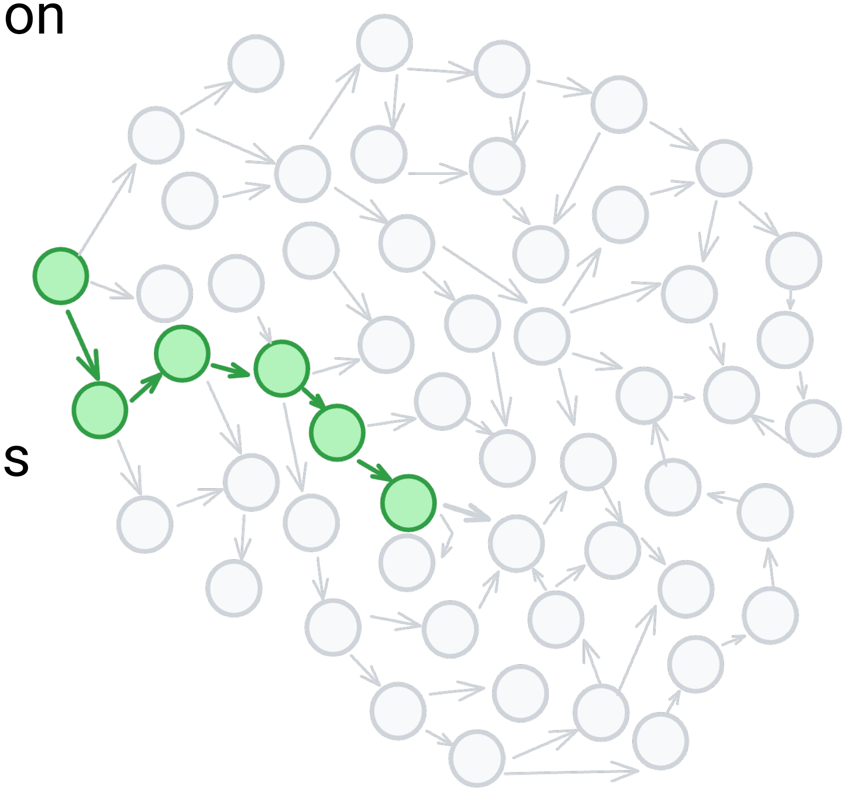


Examples → Exploration

LLMs generate a solution

Humans define what must always hold true

The test engine explores the problem space

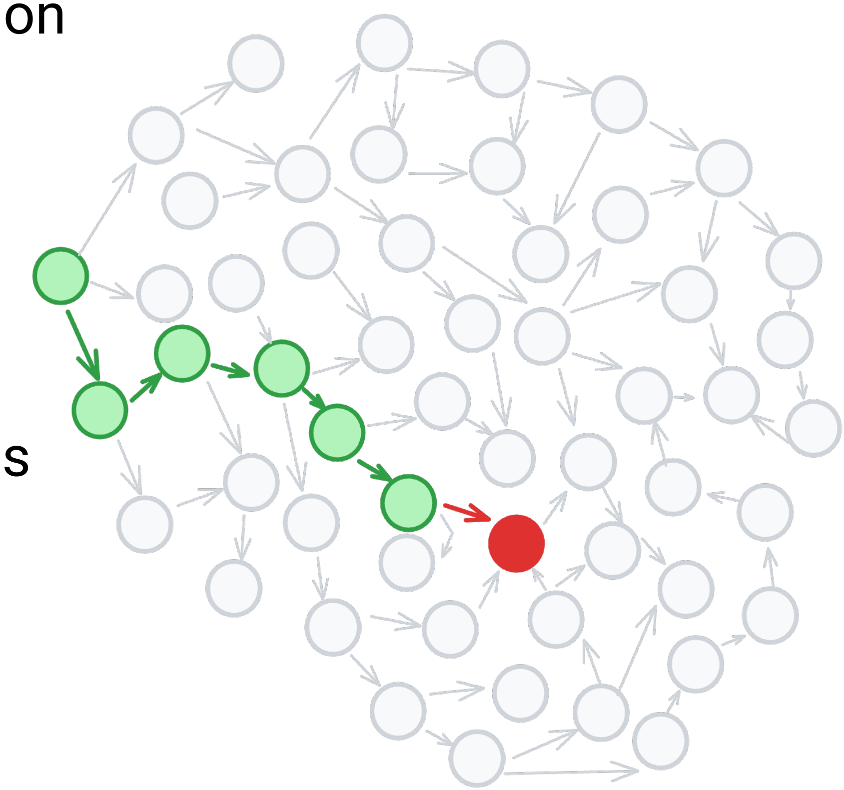


Examples → Exploration

LLMs generate a solution

Humans define what must always hold true

The test engine explores the problem space



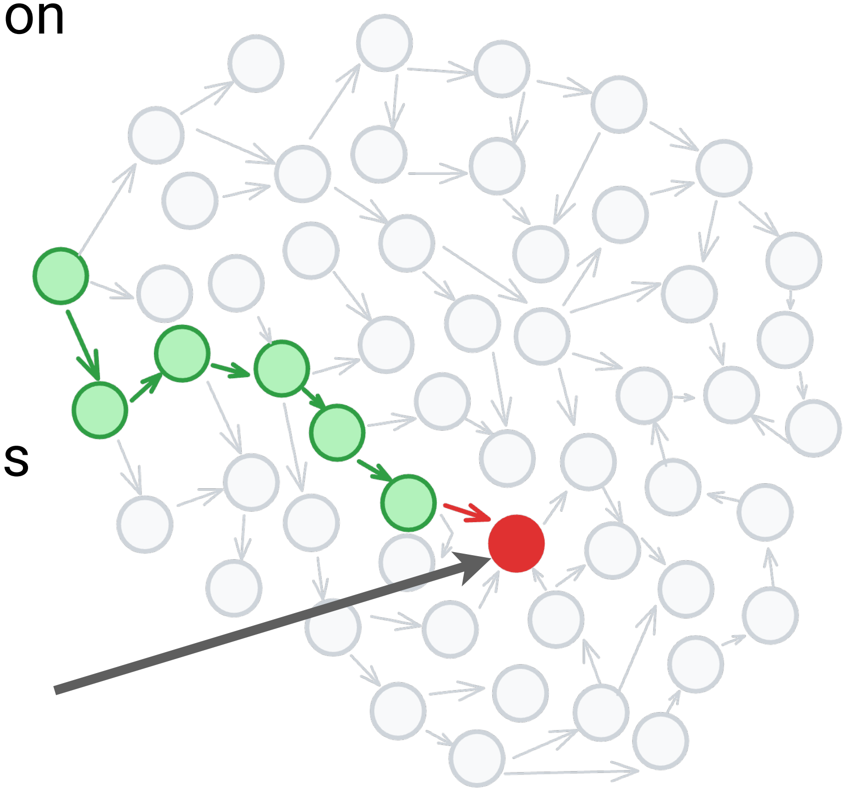
Examples → Exploration

LLMs generate a solution

Humans define what must always hold true

The test engine explores the problem space

The test engine finds a counter example



Failure

Failure → Shrinking

Failure → Shrinking → Understanding

Failure → Shrinking → Understanding

```
Original Sample
---------------
  integers: [-778383, 138, -958125, -9, -2125, 1430, -2, 16, 1, -50110, -91381,
             249644599, -5995, 2396, -1997, 24, 23, -121416, -868149, -38934,
             8045, -373, 147465, -3, -58, 514, -177, 97, 1386116950, 2953, 3804]
  Expecting actual:
    1386116950
  to be less than or equal to:
    3804
```

Failure → Shrinking → Understanding

```
Original Sample
---------------
  integers: [-778383, 138, -958125, -9, -2125, 1430, -2, 16, 1, -50110, -91381,
             249644599, -5995, 2396, -1997, 24, 23, -121416, -868149, -38934,
             8045, -373, 147465, -3, -58, 514, -177, 97, 1386116950, 2953, 3804]
  Expecting actual:
    1386116950
  to be less than or equal to:
    3804
```

Failure → Shrinking → Understanding

```
Shrunk Sample (11 steps)
------------------------
  integers: [0, -1]
  Expecting actual:
    0
  to be less than or equal to:
    -1

Original Sample
---------------
  integers: [-778383, 138, -958125, -9, -2125, 1430, -2, 16, 1, -50110, -91381,
             249644599, -5995, 2396, -1997, 24, 23, -121416, -868149, -38934,
             8045, -373, 147465, -3, -58, 514, -177, 97, 1386116950, 2953, 3804]
  Expecting actual:
    1386116950
  to be less than or equal to:
    3804
```

Failure → Shrinking → Understanding

```
Shrunk Sample (11 steps)
------------------------
  integers: [0, -1]
  Expecting actual:
    0
  to be less than or equal to:
    -1
```

```
for (int i = 0; i < n - 1; i++) {
   for (int j = 0; j < n - i - 2; j++) {
       if (arr.get(j) > arr.get(j + 1)) {
           int temp = arr.get(j);
           arr.set(j, arr.get(j + 1));
           arr.set(j + 1, temp);
       }
   }
}
```

Failure → Shrinking → Understanding

```
Shrunk Sample (11 steps)
------------------------
  integers: [0, -1]
  Expecting actual:
    0
  to be less than or equal to:
    -1
```

```
for (int i = 0; i < n - 1; i++) {
    for (int j = 0; j < n - i - 2; j++) {
        if (arr.get(j) > arr.get(j + 1)) {
            int temp = arr.get(j);
            arr.set(j, arr.get(j + 1));
            arr.set(j + 1, temp);
        }
    }
}
```

Last item never gets sorted

Failure → Shrinking → Understanding

```
Shrunk Sample (11 steps)
------------------------
  integers: [0, -1]
  Expecting actual:
    0
  to be less than or equal to:
    -1
```

```java
for (int i = 0; i < n - 1; i++) {
    for (int j = 0; j < n - i - 2; j++) {
        if (arr.get(j) > arr.get(j + 1)) {
            int temp = arr.get(j);
            arr.set(j, arr.get(j + 1));
            arr.set(j + 1, temp);
        }
    }
}
```

Last item never gets sorted

Shrunk samples are easier to **understand**

How Did I Miss That?

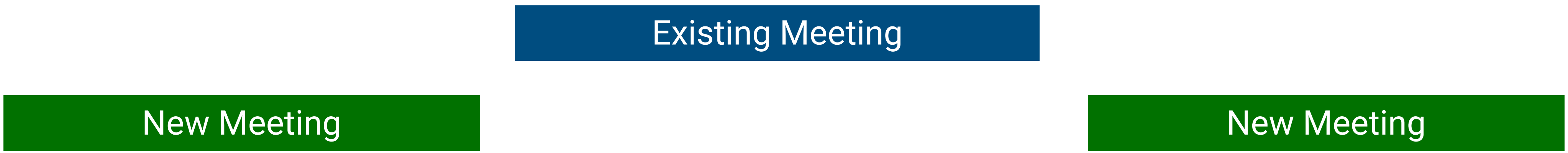
Meeting Scheduling

Meeting Scheduling

Existing Meeting

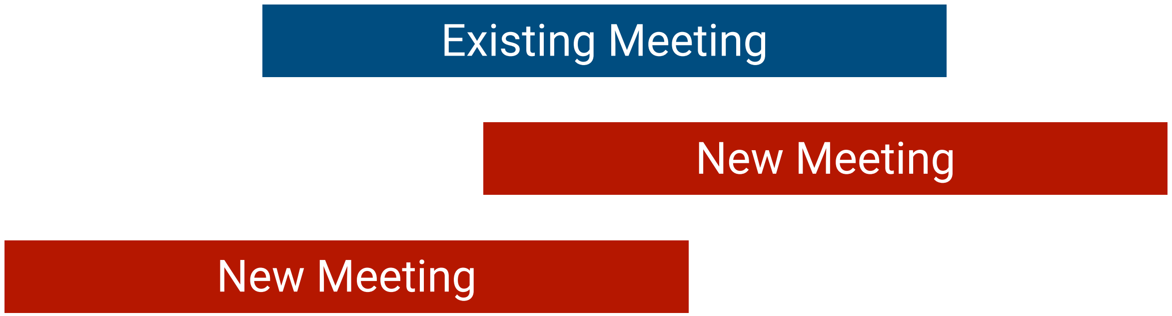
Meeting Scheduling

Meeting Scheduling

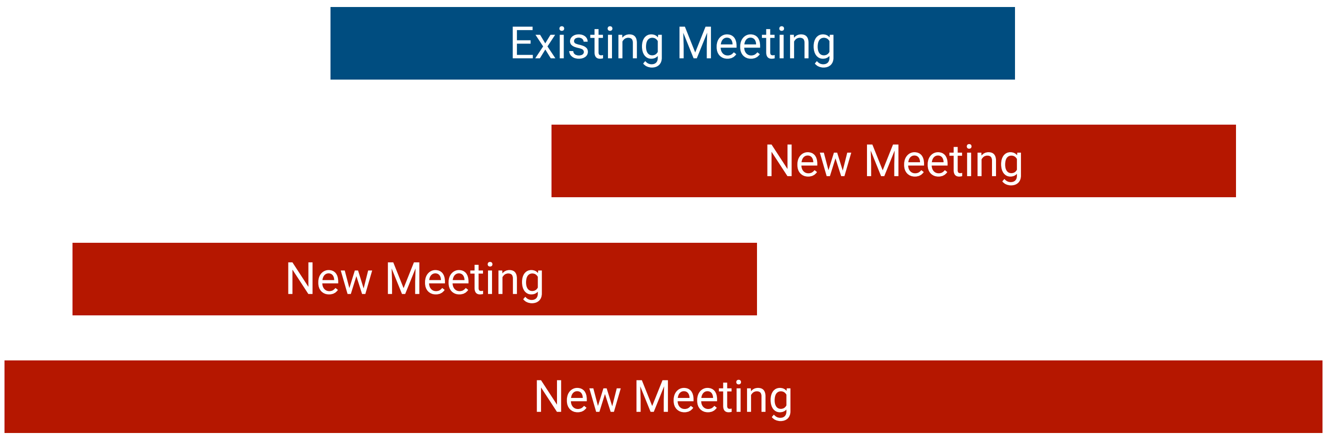


Meeting Scheduling

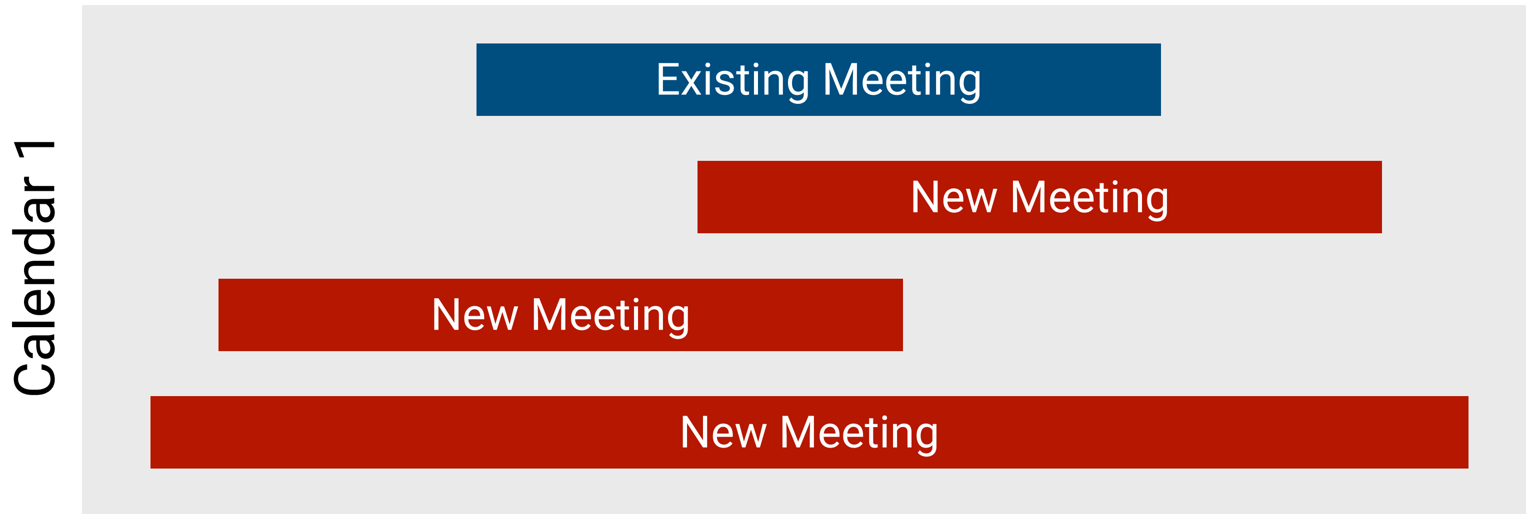
Meeting Scheduling



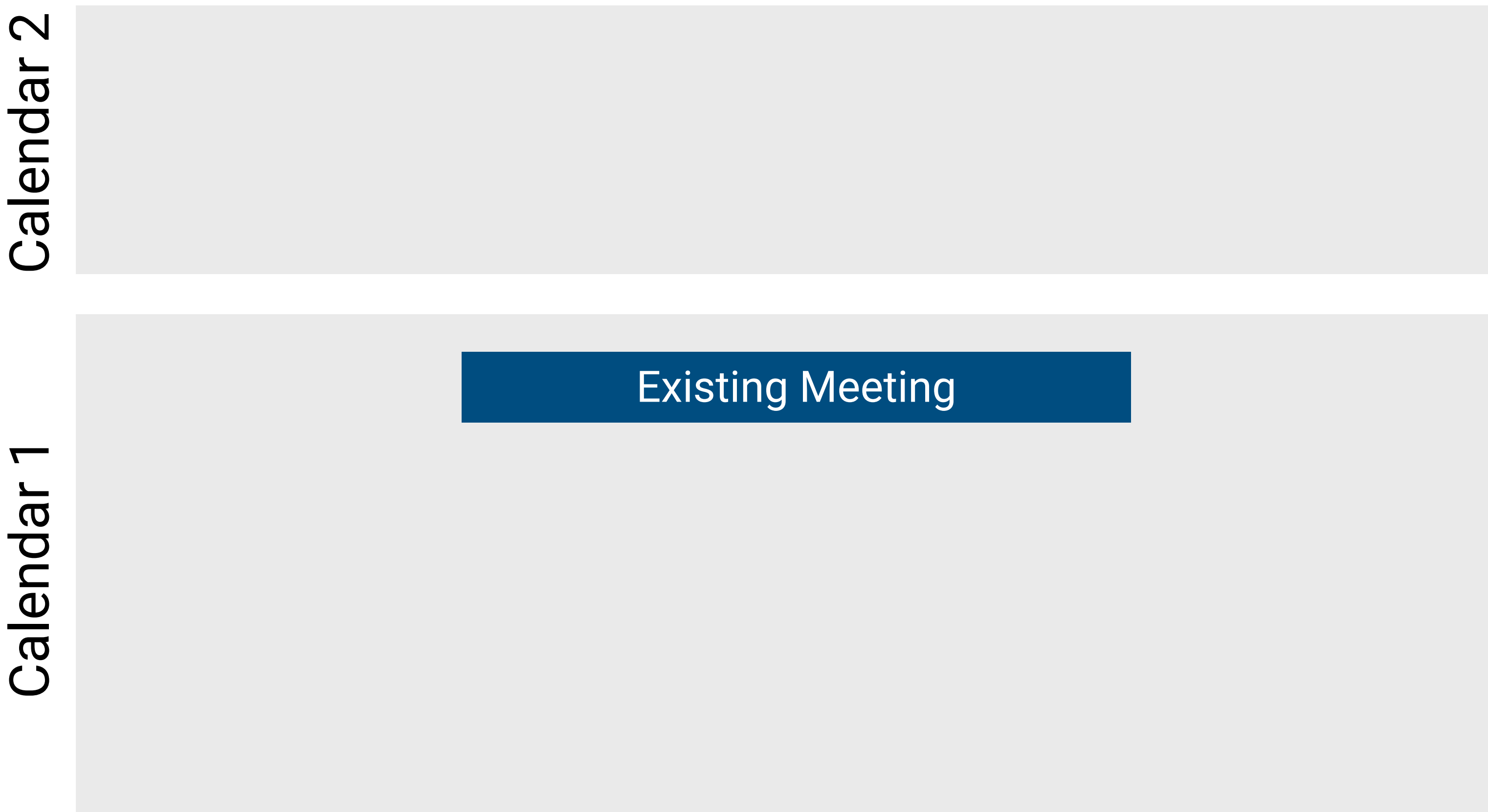
Meeting Scheduling



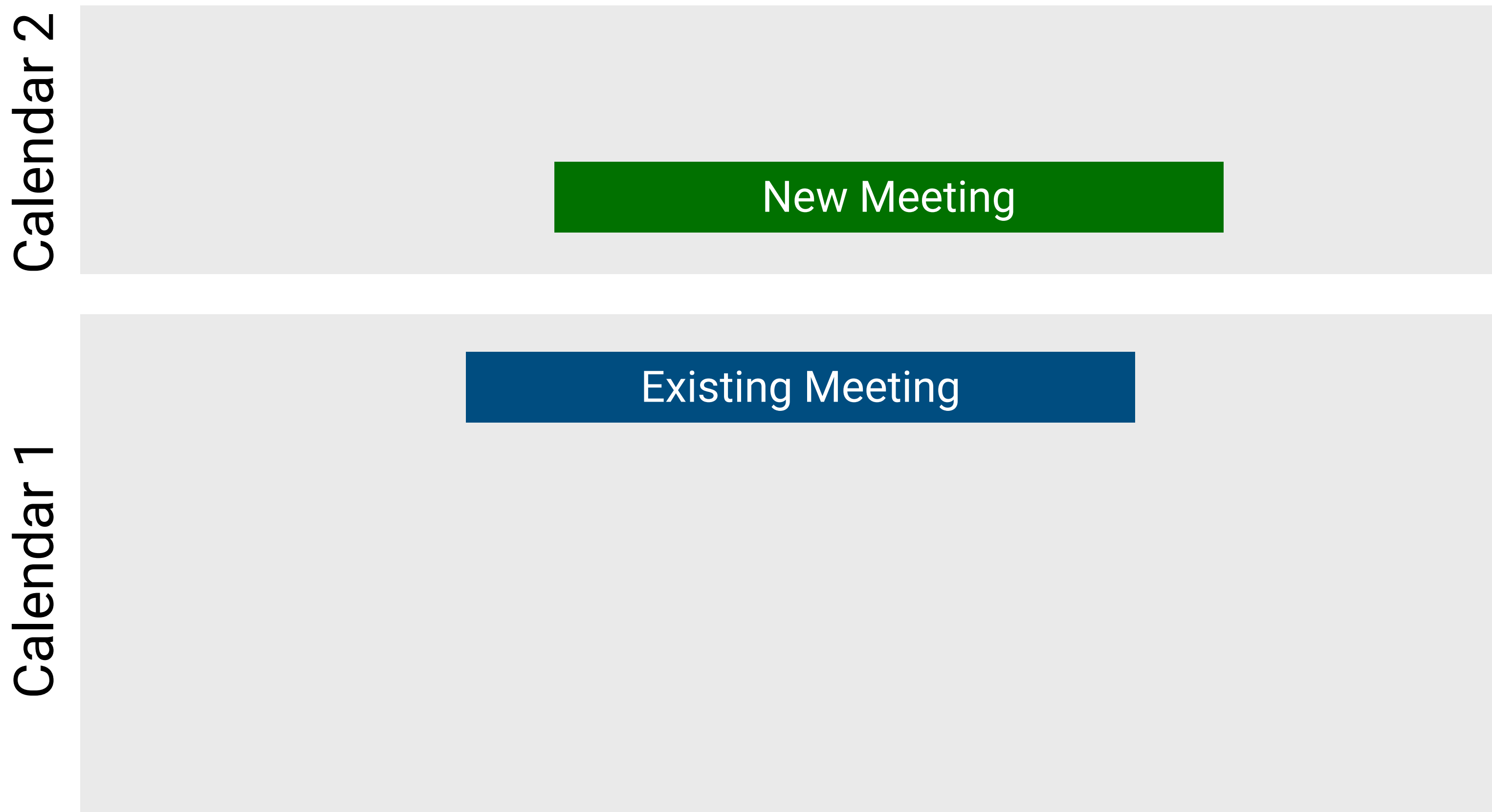
Meeting Scheduling



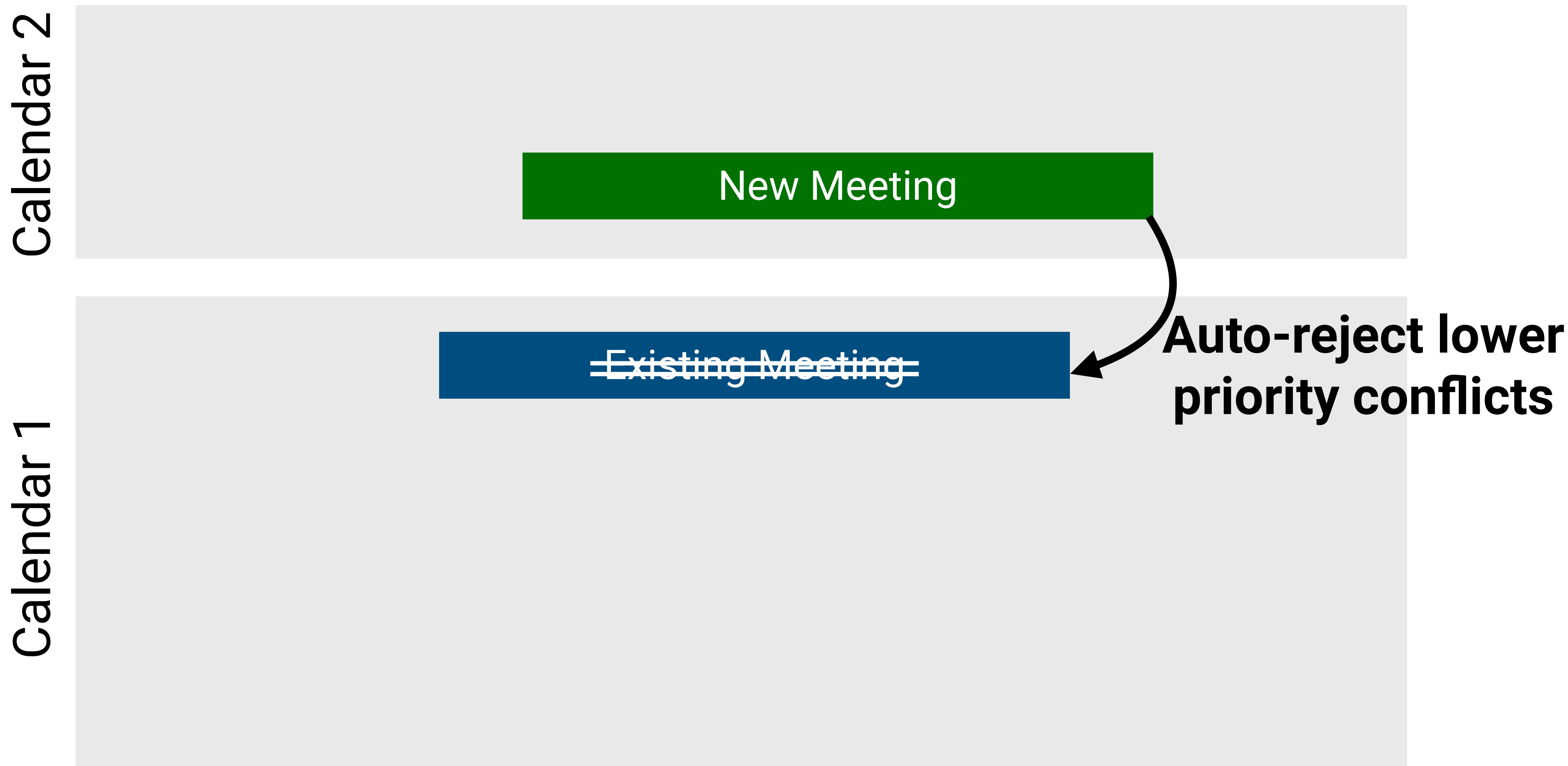
Meeting Scheduling



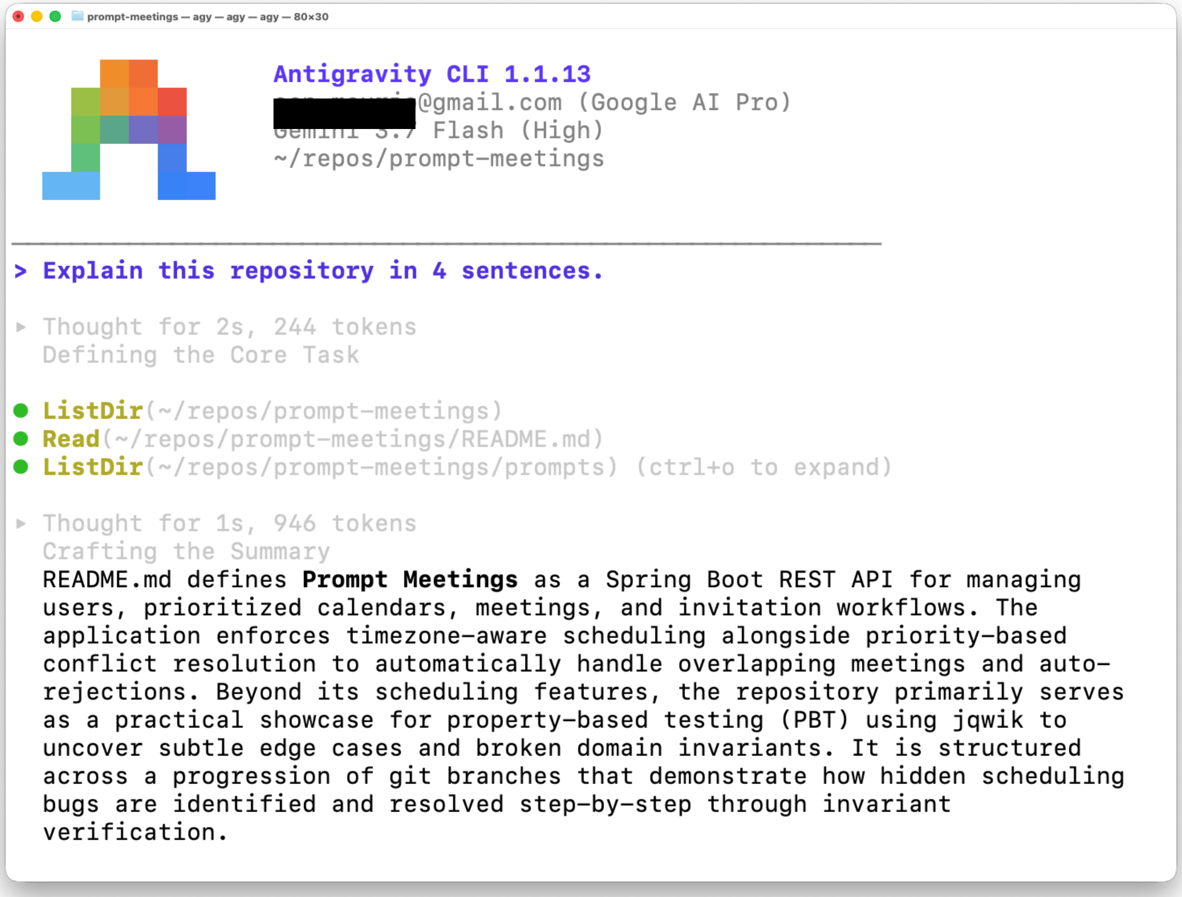
Meeting Scheduling



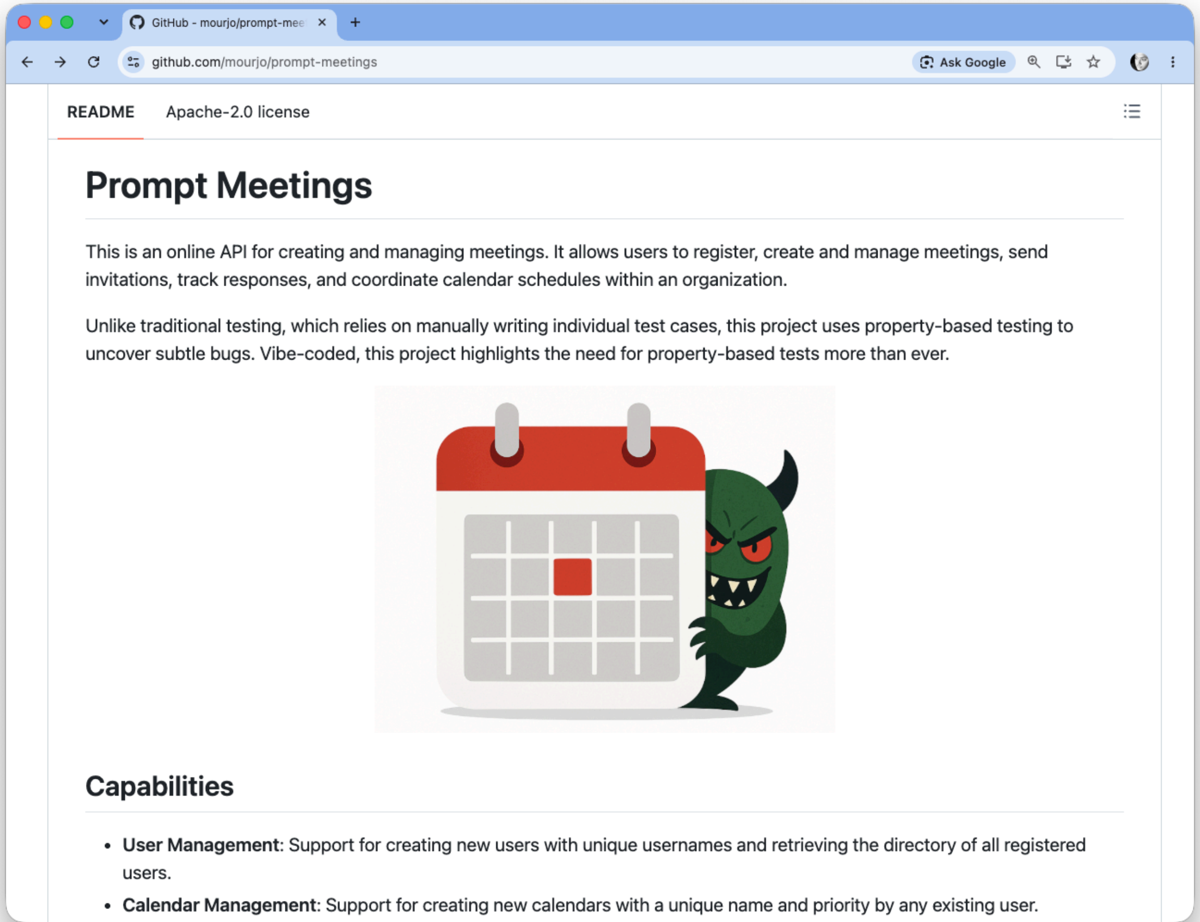
Meeting Scheduling



Meeting Scheduling, Vibe Coded



Meeting Scheduling, Vibe Coded



Source: github.com/mourjo/prompt-meetings

Meeting Scheduling, Vibe Coded

Meeting Scheduling, Vibe Coded

> *This application called "prompt-meetings" is designed to be used for creating/managing meetings. create the following endpoints:*
> *- create a user with a unique username*
> *- view all users in the system*
> *- create a meeting with a start and end time*
> *...*

Meeting Scheduling, Vibe Coded

> *I want to introduce now a concept of calendars. A calendar contains meetings. Every meeting must be part of a calendar. Calendars have priorities denoted by a floating point number. Migrate the existing meetings to a default calendar. Update the database schema. Update the create-meeting endpoints...*

Meeting Scheduling, Vibe Coded

> *I want to introduce now a concept of calendars. A calendar contains meetings. Every meeting must be part of a calendar. Calendars have priorities denoted by a floating point number. Migrate the existing meetings to a default calendar. Update the database schema. Update the create-meeting endpoints...*

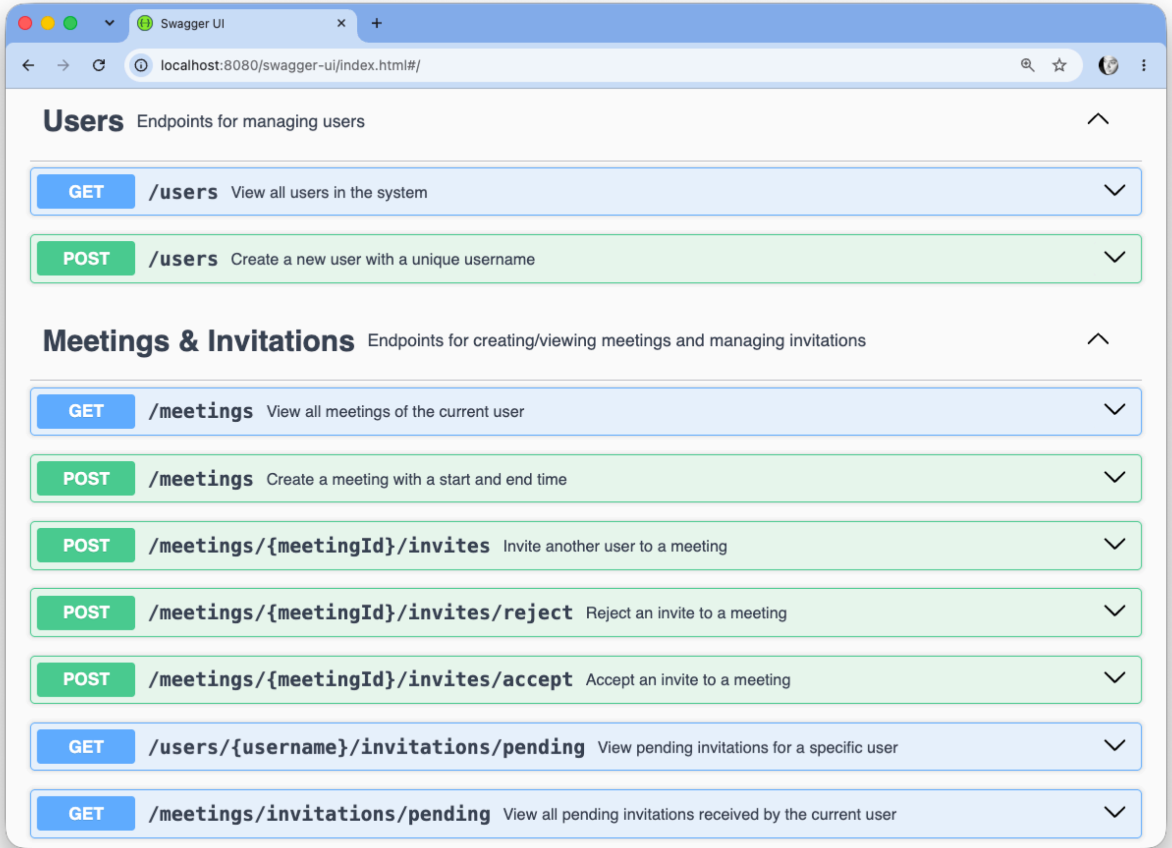
Meeting Scheduling, Vibe Coded

> *Now I want to use the priority of calendars – when a meeting is being created, check if any of my meetings are conflicting with this. If the conflicting meeting is originating from a calendar of a lower priority, reject that meeting. If the conflict is coming from a calendar of the same priority or higher, do not allow its creation...*

Meeting Scheduling, Vibe Coded

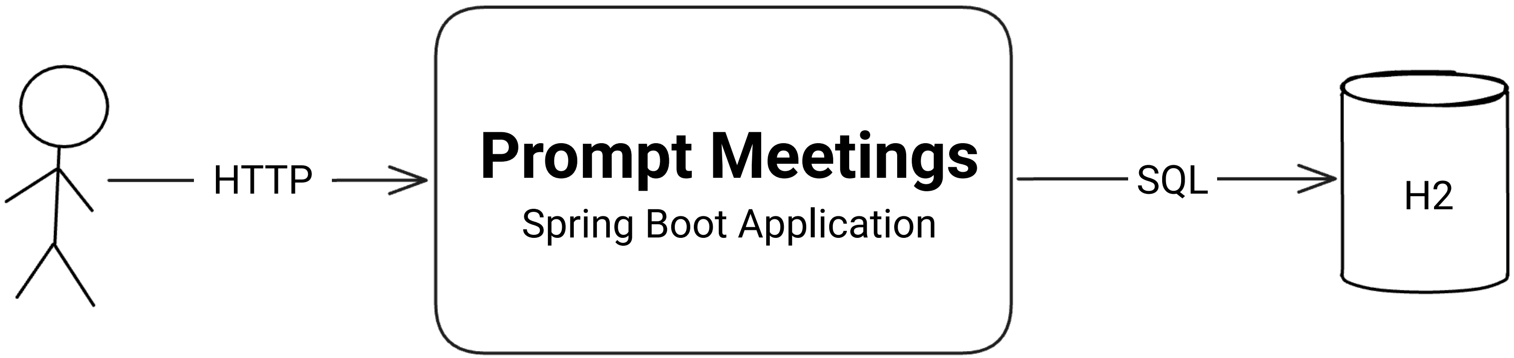
```
$ mvn clean test
[INFO] -------------------------------------------------------
[INFO]  T E S T S
[INFO] -------------------------------------------------------
...
[INFO] Results:
[INFO] Tests run: 2, Failures: 0, Errors: 0, Skipped: 0
[INFO] ------------------------------------------------------------------------
[INFO] BUILD SUCCESS
[INFO] ------------------------------------------------------------------------
[INFO] Total time:  6.349 s
[INFO] Finished at: 2026-08-16T17:54:28+02:00
[INFO] ------------------------------------------------------------------------
```

Meeting Scheduling, Vibe Coded



Source: github.com/mourjo/prompt-meetings

Meeting Scheduling, Vibe Coded



Source: github.com/mourjo/prompt-meetings

Meeting Scheduling, Vibe Coded

> *I want to add property based tests using jqwik. Use version 1.9.3. Use the spring boot extension.*
> *- There should be 4 users in the system (this should be a constant, do this in the setup)*
> *- There should be 3 calendars of priorities default, medium, high, this should be a constant, do this in the setup...*

Meeting Scheduling, Vibe Coded

> *...the property based test should perform sequences of actions on behalf of users*
> *- create-meeting*
> *- invite someone else to a meeting*
> *- accept an invitation*
> *- reject an invitation*

Meeting Scheduling, Vibe Coded

> *...the invariant should always hold true: no user can ever part of two meetings at the same time – no sequence of actions should ever allow this.*

Meeting Scheduling, Vibe Coded

```
# ~/repos/prompt-meetings → mvn clean test
[INFO] Scanning for projects...
...
[INFO]
[INFO] -------------------------------------------------------
[INFO]  T E S T S
[INFO] -------------------------------------------------------
[INFO]
[INFO] Results:
[INFO]
[ERROR] Failures:
[ERROR]   MeetingSchedulerPropertyTests.noOverlappingMeetingsForAnyUser:90
>lambda$noOverlappingMeetingsForAnyUser$0:80 Invariant 'no-overlap' failed after the following actions: [
    alice creates Meeting-0 in calendar default (UTC) from 10:00 to 10:01
    alice creates Meeting-1 in calendar medium (UTC) from 10:00 to 10:01
    alice accepts invitation to meeting Meeting-0
]
final state: me.mourjo.prompt.meetings.MeetingSchedulerPropertyTests$CalendarMeetingsState@7fdfe501
User charlie is in overlapping meetings: Meeting-0 [10:00 - 10:01] and Meeting-1 [10:00 - 10:01]
[INFO]
[ERROR] Tests run: 3, Failures: 1, Errors: 0, Skipped: 0
[INFO]
[INFO] ------------------------------------------------------------------------
[INFO] BUILD FAILURE
```

Meeting Scheduling, Vibe Coded

```
Invariant 'no-overlap' failed after the following actions: [
    alice creates Meeting-0 in calendar default (UTC) from 10:00 to 10:01
    alice creates Meeting-1 in calendar medium (UTC) from 10:00 to 10:01
    alice accepts invitation to meeting Meeting-0
]
```

Meeting Scheduling, Vibe Coded

```
Invariant 'no-overlap' failed after the following actions: [
    charlie creates Meeting-0 in calendar high (UTC) from 10:41 to 10:49
    alice creates Meeting-1 in calendar medium (UTC) from 13:49 to 13:50
    bob creates Meeting-2 in calendar default (UTC) from 10:44 to 11:04
    charlie creates Meeting-3 in calendar high (UTC) from 11:29 to 12:28
    dan creates Meeting-4 in calendar medium (UTC) from 10:05 to 10:25
    bob creates Meeting-5 in calendar default (UTC) from 10:19 to 10:37
    alice rejects invitation to Meeting-2
    dan accepts invitation to meeting Meeting-2
    dan invites alice to Meeting-2
    alice invites charlie to Meeting-1
    alice rejects invitation to Meeting-0
    charlie creates Meeting-6 in calendar medium (UTC) from 15:40 to 16:09
    dan rejects invitation to Meeting-3
    charlie accepts invitation to meeting Meeting-6
    dan creates Meeting-7 in calendar high (UTC) from 10:10 to 10:14
    alice invites dan to Meeting-4
    charlie creates Meeting-8 in calendar high (UTC) from 13:15 to 14:01
    dan invites bob to Meeting-9
    charlie invites alice to Meeting-4
    bob creates Meeting-9 in calendar default (UTC) from 10:07 to 10:10
    dan accepts invitation to meeting Meeting-4
]
```

Meeting Scheduling, Vibe Coded

```
Invariant 'no-overlap' failed after the following actions: [
    charlie creates Meeting-0 in calendar high (UTC) from 10:41 to 10:49
    alice creates Meeting-1 in calendar medium (UTC) from 13:49 to 13:50
    bob creates Meeting-2 in calendar default (UTC) from 10:44 to 11:04
    charlie creates Meeting-3 in calendar high (UTC) from 11:29 to 12:28
    dan creates Meeting-4 in calendar medium (UTC) from 10:05 to 10:25
    bob creates Meeting-5 in calendar default (UTC) from 10:19 to 10:37
    alice rejects invitation to Meeting-2
    dan accepts invitation to meeting Meeting-2
    dan invites alice to Meeting-2
    alice invites charlie to Meeting-1
    alice rejects invitation to Meeting-0
    charlie creates Meeting-6 in calendar medium (UTC) from 15:40 to 16:09
    dan rejects invitation to Meeting-3
    charlie accepts invitation to meeting Meeting-6
    dan creates Meeting-7 in calendar high (UTC) from 10:10 to 10:14
    alice invites dan to Meeting-4
    charlie creates Meeting-8 in calendar high (UTC) from 13:15 to 14:01
    dan invites bob to Meeting-9
    charlie invites alice to Meeting-4
    bob creates Meeting-9 in calendar default (UTC) from 10:07 to 10:10
    dan accepts invitation to meeting Meeting-4
]
```

Meeting Scheduling, Vibe Coded

```
Invariant 'no-overlap' failed after the following actions: [
    charlie creates Meeting-0 in calendar high (UTC) from 10:41 to 10:49
    alice creates Meeting-1 in calendar medium (UTC) from 13:49 to 13:50
    bob creates Meeting-2 in calendar default (UTC) from 10:44 to 11:04
    charlie creates Meeting-3 in calendar high (UTC) from 11:29 to 12:28
    dan creates Meeting-4 in calendar medium (UTC) from 10:05 to 10:25
    bob creates Meeting-5 in calendar default (UTC) from 10:19 to 10:37
    alice rejects invitation to Meeting-2
    dan accepts invitation to meeting Meeting-2
    dan invites alice to Meeting-2
    alice invites charlie to Meeting-1
    alice rejects invitation to Meeting-0
    charlie creates Meeting-6 in calendar medium (UTC) from 15:40 to 16:09
    dan rejects invitation to Meeting-3
    charlie accepts invitation to meeting Meeting-6
    dan creates Meeting-7 in calendar high (UTC) from 10:10 to 10:14
    alice invites dan to Meeting-4
    charlie creates Meeting-8 in calendar high (UTC) from 13:15 to 14:01
    dan invites bob to Meeting-9
    charlie invites alice to Meeting-4
    bob creates Meeting-9 in calendar default (UTC) from 10:07 to 10:10
    dan accepts invitation to meeting Meeting-4
]
```

Meeting Scheduling, Vibe Coded

```
Invariant 'no-overlap' failed after the following actions: [
    charlie creates Meeting-0 in calendar high (UTC) from 10:41 to 10:49
    alice creates Meeting-1 in calendar medium (UTC) from 13:49 to 13:50
    bob creates Meeting-2 in calendar default (UTC) from 10:44 to 11:04
    charlie creates Meeting-3 in calendar high (UTC) from 11:29 to 12:28
    dan creates Meeting-4 in calendar medium (UTC) from 10:05 to 10:25
    bob creates Meeting-5 in calendar default (UTC) from 10:19 to 10:37
    alice rejects invitation to Meeting-2
    dan accepts invitation to meeting Meeting-2
    dan invites alice to Meeting-2
    alice invites charlie to Meeting-1
    alice rejects invitation to Meeting-0
    charlie creates Meeting-6 in calendar medium (UTC) from 15:40 to 16:09
    dan rejects invitation to Meeting-3
    charlie accepts invitation to meeting Meeting-6
    dan creates Meeting-7 in calendar high (UTC) from 10:10 to 10:14
    alice invites dan to Meeting-4
    charlie creates Meeting-8 in calendar high (UTC) from 13:15 to 14:01
    dan invites bob to Meeting-9
    charlie invites alice to Meeting-4
    bob creates Meeting-9 in calendar default (UTC) from 10:07 to 10:10
    dan accepts invitation to meeting Meeting-4
]
```

Meeting Scheduling, Vibe Coded

@Property

Meeting Scheduling, Vibe Coded

```
@Property
                                        @ForAll("actions") ActionChain<CalendarState> chain
```

Meeting Scheduling, Vibe Coded

```
@Property
@ForAll("actions") ActionChain<CalendarState> chain
```

```
@Provide
Arbitrary<ActionChain<CalendarState>> actions() {
    return ActionChain.startWith(CalendarMeetingsState::new)
        .withAction(new CreateMeetingAction())
        .withAction(new InviteUserAction())
        .withAction(new AcceptInviteAction())
        .withAction(new RejectMeetingAction());
}
```

Meeting Scheduling, Vibe Coded

```
@Property
                                        @ForAll("actions") ActionChain<CalendarState> chain
```

Meeting Scheduling, Vibe Coded

```
@Property
void noOverlappingMeetingsForAnyUser(@ForAll("actions") ActionChain<CalendarState> chain) {
```

Meeting Scheduling, Vibe Coded

```
@Property
void noOverlappingMeetingsForAnyUser(@ForAll("actions") ActionChain<CalendarState> chain) {
    chain.withInvariant("no-overlap", sut -> {









    }).run();
```

Meeting Scheduling, Vibe Coded

```
@Property
void noOverlappingMeetingsForAnyUser(@ForAll("actions") ActionChain<CalendarState> chain) {
    chain.withInvariant("no-overlap", sut -> {

        for (String user : USERS) {








        }
    }).run();
```

Meeting Scheduling, Vibe Coded

```java
@Property
void noOverlappingMeetingsForAnyUser(@ForAll("actions") ActionChain<CalendarState> chain) {
    chain.withInvariant("no-overlap", sut -> {

        for (String user : USERS) {
            var meetings = meetingService.getMyMeetings(user).stream()
                .filter(m -> "ACCEPTED".equalsIgnoreCase(m.userStatus()))
                .toList();









        }
    }).run();
```

Meeting Scheduling, Vibe Coded

```
@Property
void noOverlappingMeetingsForAnyUser(@ForAll("actions") ActionChain<CalendarState> chain) {
    chain.withInvariant("no-overlap", sut -> {

        for (String user : USERS) {
            var meetings = meetingService.getMyMeetings(user).stream()
                .filter(m -> "ACCEPTED".equalsIgnoreCase(m.userStatus()))
                .toList();

            for (int i = 0; i < meetings.size(); i++) {
                for (int j = i + 1; j < meetings.size(); j++) {




                }
            }
        }
    }).run();
```

Meeting Scheduling, Vibe Coded

```java
@Property
void noOverlappingMeetingsForAnyUser(@ForAll("actions") ActionChain<CalendarState> chain) {
    chain.withInvariant("no-overlap", sut -> {

        for (String user : USERS) {
            var meetings = meetingService.getMyMeetings(user).stream()
                .filter(m -> "ACCEPTED".equalsIgnoreCase(m.userStatus()))
                .toList();

            for (int i = 0; i < meetings.size(); i++) {
                for (int j = i + 1; j < meetings.size(); j++) {
                    if (overlaps(meetings.get(i), meetings.get(j))) {
                        throw new AssertionError(...);
                    }
                }
            }
        }
    }).run();
```

Meeting Scheduling, Vibe Coded

```
[ERROR]   Invariant 'no-overlap' failed after the following actions: [
    charlie creates Meeting-0 in calendar default (UTC) from 10:00 to 10:01
    charlie creates Meeting-1 in calendar medium (UTC) from 10:00 to 10:01
    charlie accepts invitation to meeting Meeting-0
]
```

Meeting Scheduling, Vibe Coded

```
[ERROR]   Invariant 'no-overlap' failed after the following actions: [
    charlie creates Meeting-0 in calendar default (UTC) from 10:00 to 10:01
    charlie creates Meeting-1 in calendar medium (UTC) from 10:00 to 10:01
    charlie accepts invitation to meeting Meeting-0
]
```

Charlie creates Meeting-0

Meeting Scheduling, Vibe Coded

```
[ERROR]   Invariant 'no-overlap' failed after the following actions: [
    charlie creates Meeting-0 in calendar default (UTC) from 10:00 to 10:01
    charlie creates Meeting-1 in calendar medium (UTC) from 10:00 to 10:01
    charlie accepts invitation to meeting Meeting-0
]
```

Charlie creates Meeting-0
Charlie creates an overlapping meeting in a higher priority calendar

Meeting Scheduling, Vibe Coded

```
[ERROR]   Invariant 'no-overlap' failed after the following actions: [
    charlie creates Meeting-0 in calendar default (UTC) from 10:00 to 10:01
    charlie creates Meeting-1 in calendar medium (UTC) from 10:00 to 10:01
    charlie accepts invitation to meeting Meeting-0
]
```

Charlie creates Meeting-0
Charlie creates an overlapping meeting in a higher priority calendar
Charlie accepts Meeting-0

Meeting Scheduling, Vibe Coded

> *When accepting a meeting ensure that the invitation is in pending state - ie, do not allow acceptance of a meeting which is already accepted or rejected.*

Meeting Scheduling, Vibe Coded

```
[ERROR]   Invariant 'no-overlap' failed after the following actions: [
    charlie creates Meeting-0 in calendar default (UTC) from 10:00 to 10:01
    alice creates Meeting-1 in calendar default (UTC) from 10:00 to 10:01
    alice invites charlie to Meeting-1
    charlie accepts invitation to meeting Meeting-1
]
```

Meeting Scheduling, Vibe Coded

```
[ERROR]   Invariant 'no-overlap' failed after the following actions: [
    charlie creates Meeting-0 in calendar default (UTC) from 10:00 to 10:01
    alice creates Meeting-1 in calendar default (UTC) from 10:00 to 10:01
    alice invites charlie to Meeting-1
    charlie accepts invitation to meeting Meeting-1
]
```

Charlie creates a Meeting-0

Meeting Scheduling, Vibe Coded

```
[ERROR]   Invariant 'no-overlap' failed after the following actions: [
    charlie creates Meeting-0 in calendar default (UTC) from 10:00 to 10:01
    alice creates Meeting-1 in calendar default (UTC) from 10:00 to 10:01
    alice invites charlie to Meeting-1
    charlie accepts invitation to meeting Meeting-1
]
```

Charlie creates a Meeting-0
Alice creates an overlapping meeting

Meeting Scheduling, Vibe Coded

```
[ERROR]   Invariant 'no-overlap' failed after the following actions: [
    charlie creates Meeting-0 in calendar default (UTC) from 10:00 to 10:01
    alice creates Meeting-1 in calendar default (UTC) from 10:00 to 10:01
    alice invites charlie to Meeting-1
    charlie accepts invitation to meeting Meeting-1
]
```

Charlie creates a Meeting-0
Alice creates an overlapping meeting
Alice invites Charlie to their meeting

Meeting Scheduling, Vibe Coded

```
[ERROR]   Invariant 'no-overlap' failed after the following actions: [
    charlie creates Meeting-0 in calendar default (UTC) from 10:00 to 10:01
    alice creates Meeting-1 in calendar default (UTC) from 10:00 to 10:01
    alice invites charlie to Meeting-1
    charlie accepts invitation to meeting Meeting-1
]
```

Charlie creates a Meeting-0
Alice creates an overlapping meeting
Alice invites Charlie to their meeting
Charlie accepts the invitation

Meeting Scheduling, Vibe Coded

```
[ERROR]   Invariant 'no-overlap' failed after the following actions: [
    charlie creates Meeting-0 in calendar default (UTC) from 10:00 to 10:01
    alice creates Meeting-1 in calendar default (UTC) from 10:00 to 10:01
    alice invites charlie to Meeting-1
    charlie accepts invitation to meeting Meeting-1
]
```

```
[ERROR]   Invariant 'no-overlap' failed after the following actions: [
    charlie creates Meeting-0 in calendar default (UTC) from 10:00 to 10:01
    charlie creates Meeting-1 in calendar medium (UTC) from 10:00 to 10:01
    charlie accepts invitation to meeting Meeting-0
]
```

Meeting Scheduling, Vibe Coded

[

```
charlie creates Meeting-0 in calendar default (UTC) from 10:00 to 10:01
alice creates Meeting-1 in calendar default (UTC) from 10:00 to 10:01
alice invites charlie to Meeting-1
charlie accepts invitation to meeting Meeting-1
```

] Four actions

Three actions

[

```
charlie creates Meeting-0 in calendar default (UTC) from 10:00 to 10:01
charlie creates Meeting-1 in calendar medium (UTC) from 10:00 to 10:01
charlie accepts invitation to meeting Meeting-0
```

]

Meeting Scheduling, Vibe Coded

```
[
  charlie creates Meeting-0 in calendar default (UTC) from 10:00 to 10:01
  alice creates Meeting-1 in calendar default (UTC) from 10:00 to 10:01
  alice invites charlie to Meeting-1
  charlie accepts invitation to meeting Meeting-1
]
```

Two users

One user

```
[
  charlie creates Meeting-0 in calendar default (UTC) from 10:00 to 10:01
  charlie creates Meeting-1 in calendar medium (UTC) from 10:00 to 10:01
  charlie accepts invitation to meeting Meeting-0
]
```

Meeting Scheduling, Vibe Coded

```
[
    charlie creates Meeting-0 in calendar default (UTC) from 10:00 to 10:01
    alice creates Meeting-1 in calendar default (UTC) from 10:00 to 10:01
    alice invites charlie to Meeting-1
    charlie accepts invitation to meeting Meeting-1
]
```

One calendar

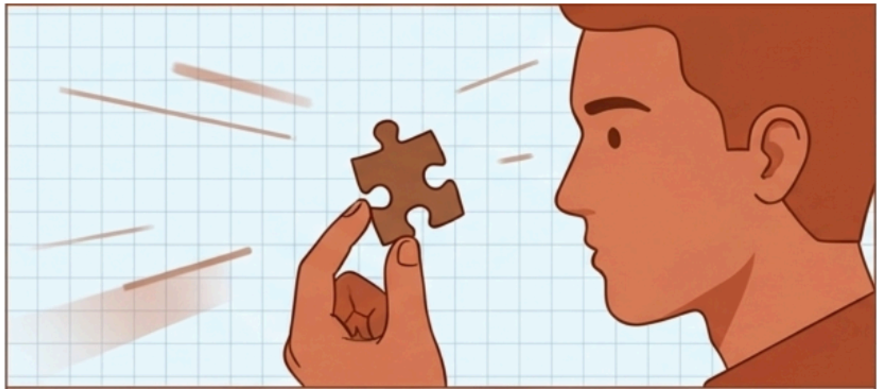
Two calendars

```
[
    charlie creates Meeting-0 in calendar default (UTC) from 10:00 to 10:01
    charlie creates Meeting-1 in calendar medium (UTC) from 10:00 to 10:01
    charlie accepts invitation to meeting Meeting-0
]
```

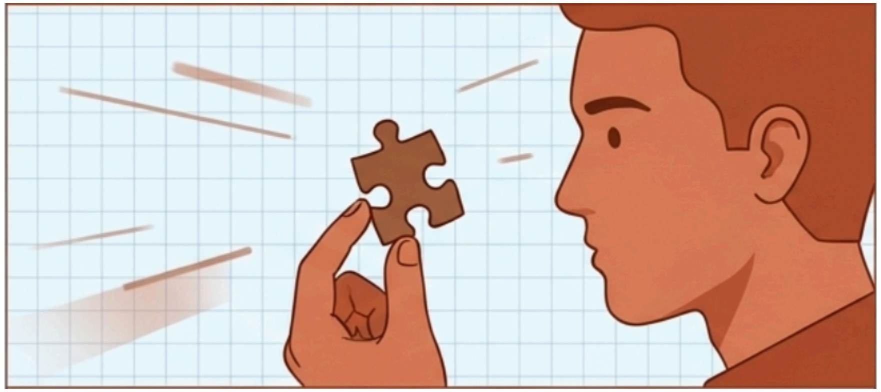
More Than Just Testing

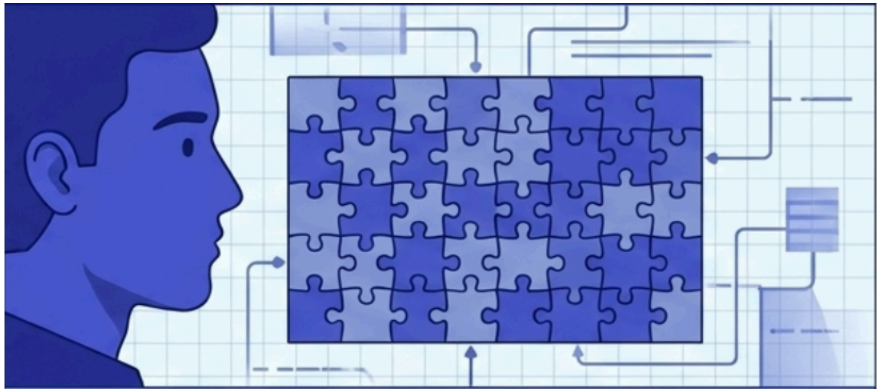
“For This” → “For All”

“For This” → “For All”



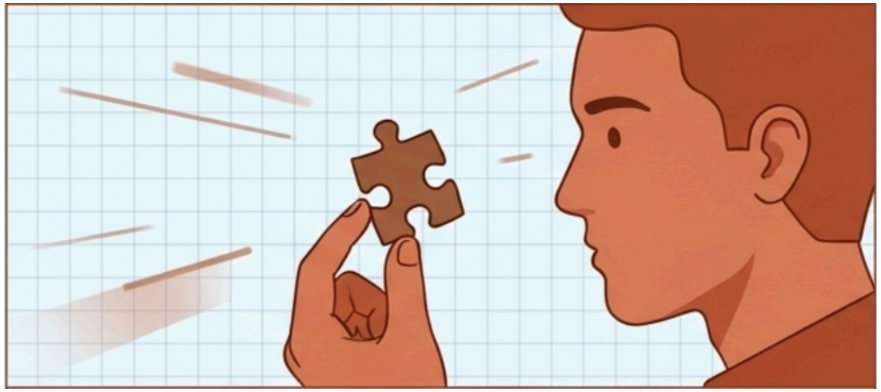
“For This” → “For All”

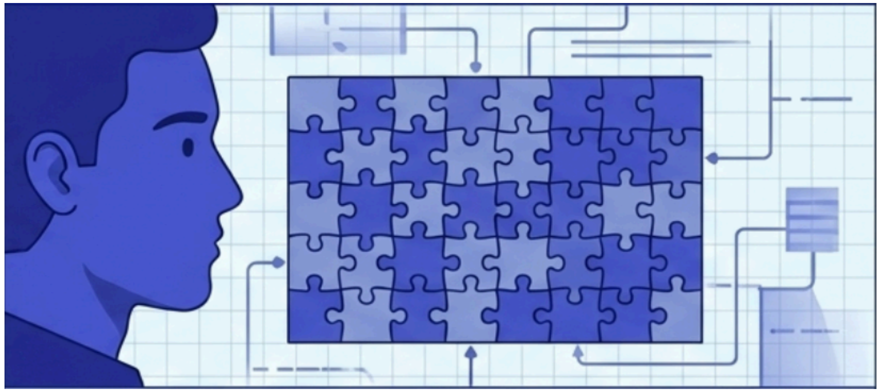




“For This” → “For All”

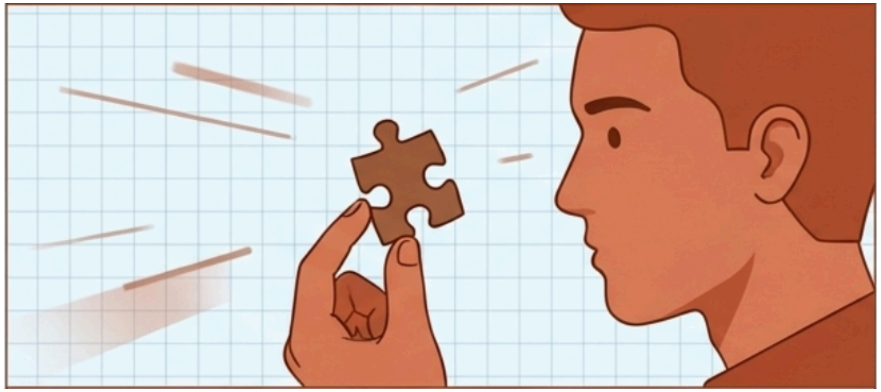
- Defining what must always hold true

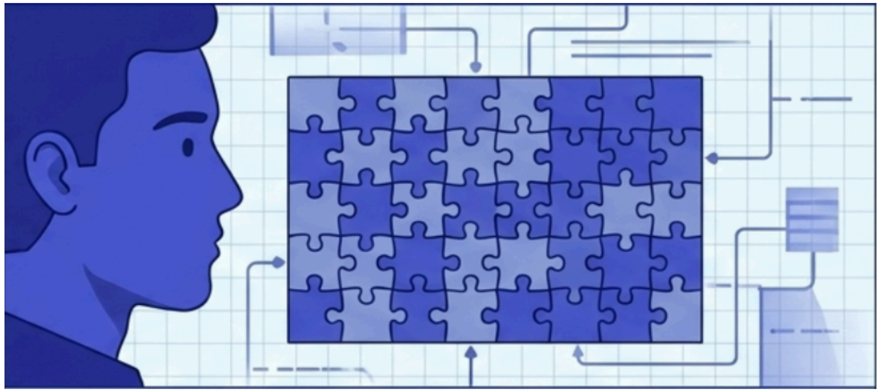




“For This” → “For All”

- Defining what must always hold true
- Understanding counter examples





“For This” → “For All”

- Defining what must always hold true
- Understanding counter examples

“For This” → “For All”

- Defining what must always hold true
- Understanding counter examples

> *Add a new property based test which ensures this invariant:* ***for every auto rejected meeting, there should be at least one higher priority meeting that overlaps with this meeting and is accepted*** *by the current user.*

“For This” → “For All”

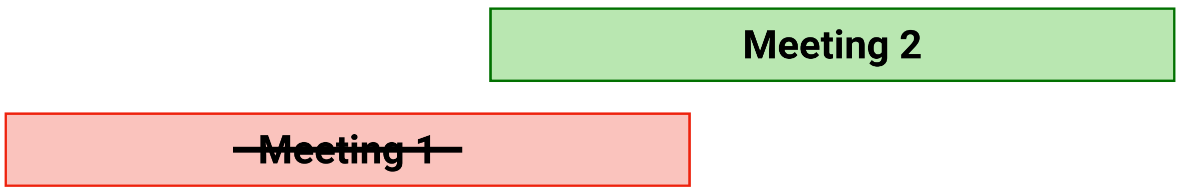
- Defining what must always hold true
- Understanding counter examples

> “*Add a new property based test which ensures this invariant:* ***for every auto rejected meeting, there should be at least one higher priority meeting that overlaps with this meeting and is accepted*** *by the current user.*”

“For This” → “For All”

- Defining what must always hold true
- Understanding counter examples

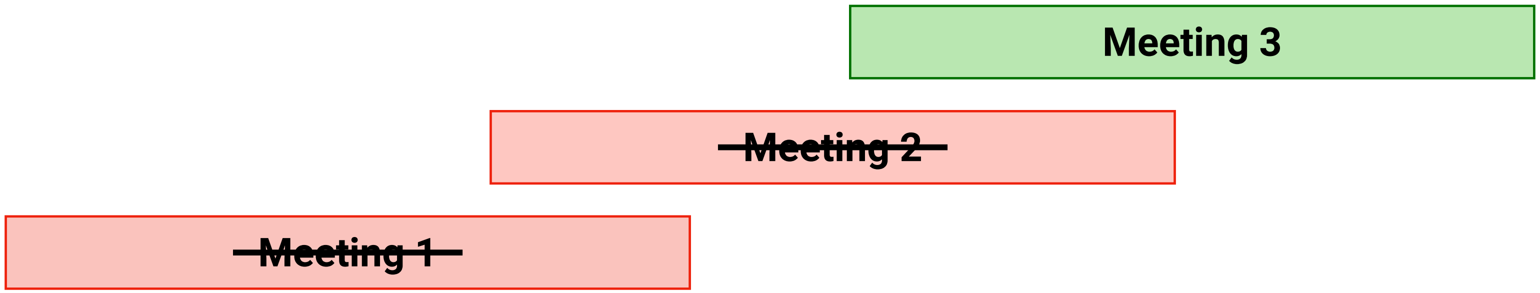
> *Add a new property based test which ensures this invariant:* ***for every auto rejected meeting, there should be at least one higher priority meeting that overlaps with this meeting and is accepted*** *by the current user.*



“For This” → “For All”

- Defining what must always hold true
- Understanding counter examples

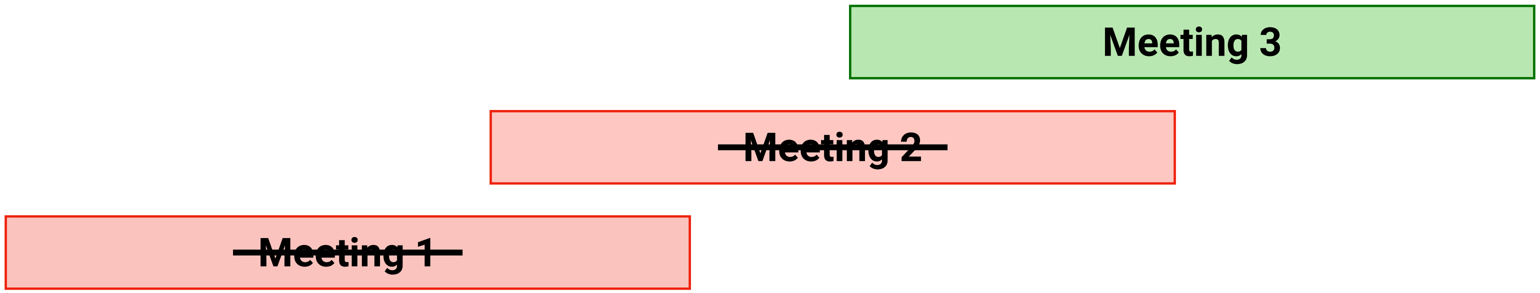
> *Add a new property based test which ensures this invariant:* ***for every auto rejected meeting, there should be at least one higher priority meeting that overlaps with this meeting and is accepted*** *by the current user.*



“For This” → “For All”

- Defining what must always hold true
- Understanding counter examples

> *Add a new property based test which ensures this invariant:* ***for every auto rejected meeting, there should be at least one higher priority meeting that overlaps with this meeting and is accepted*** *by the current user.*



No overlapping accepted meeting

Counter Examples are Transformative

Counter Examples are Transformative

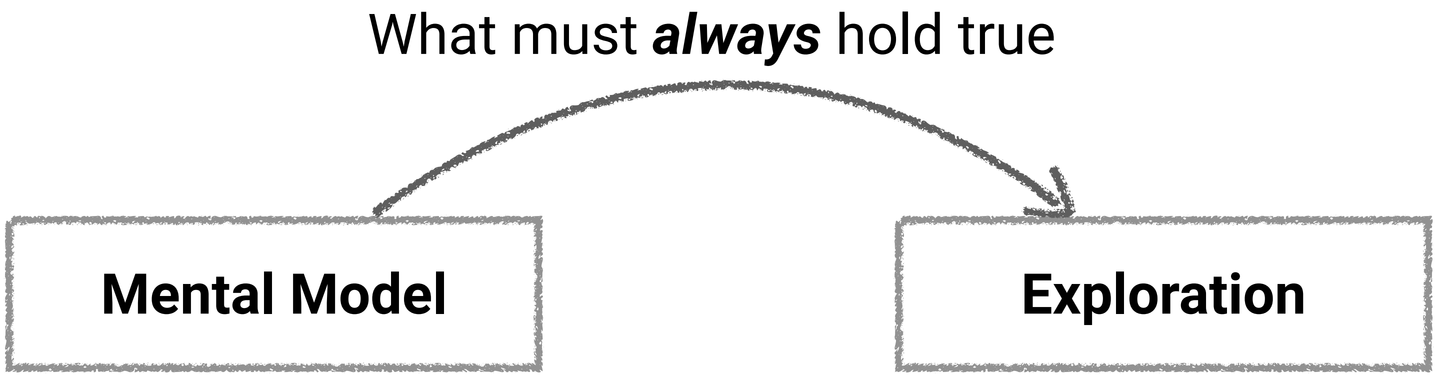
Mental Model

Counter Examples are Transformative

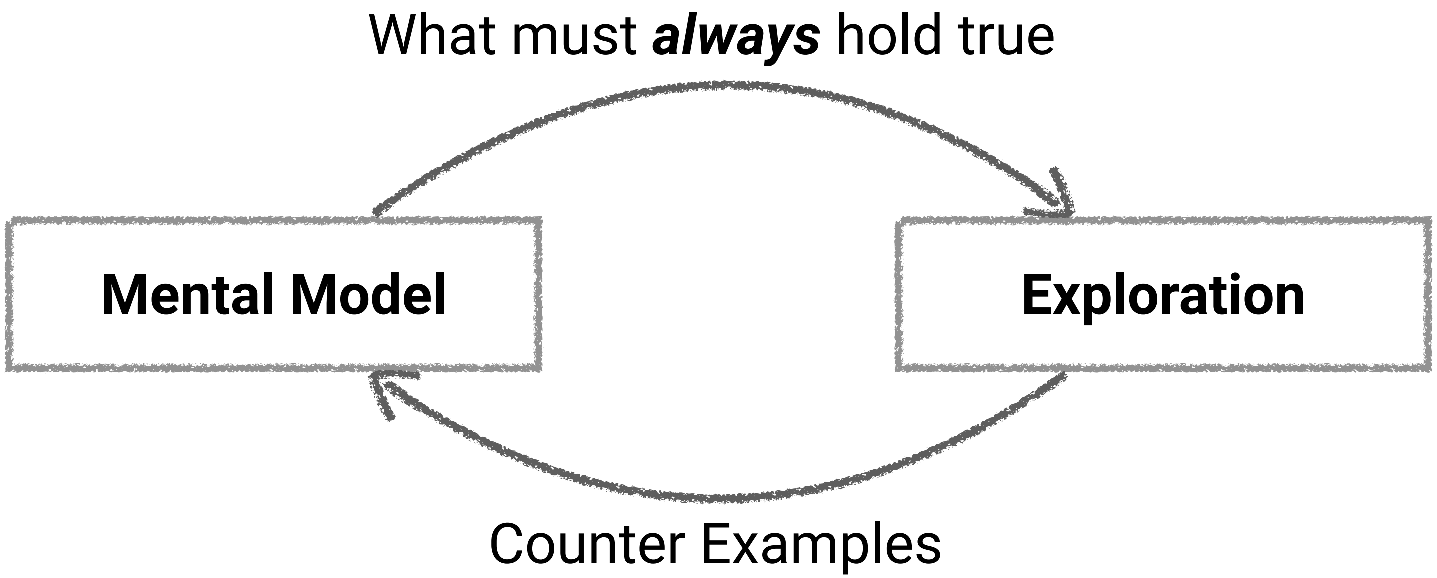
What must ***always*** hold true

Mental Model

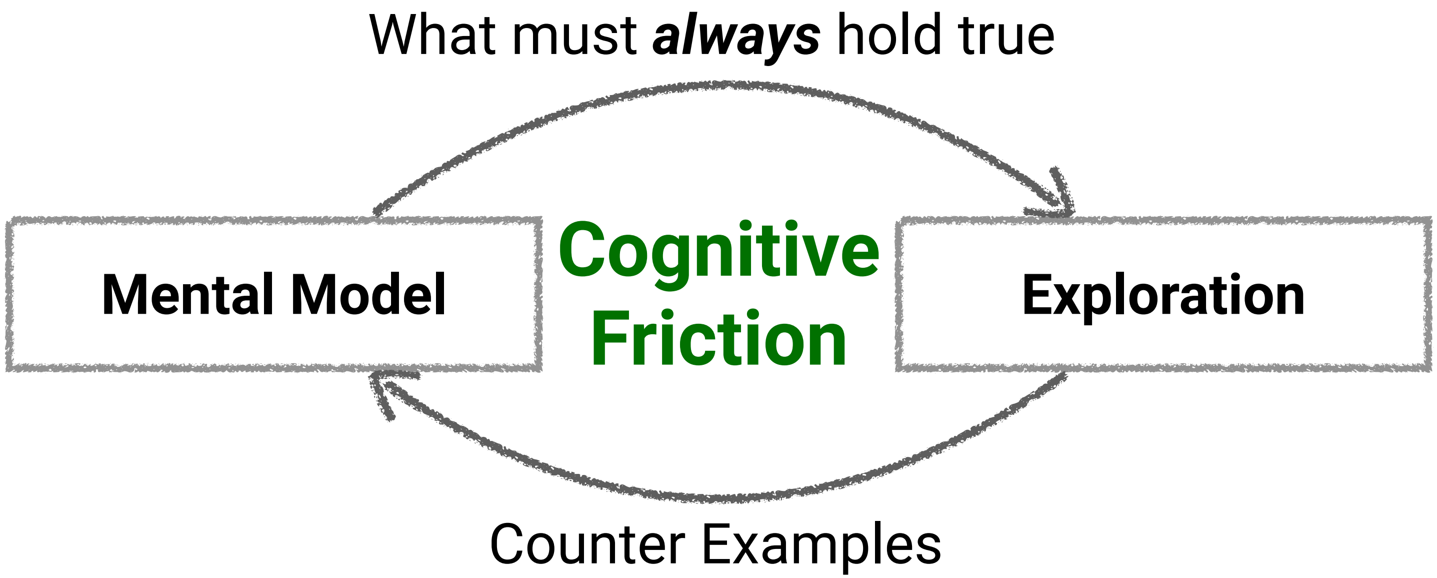
Counter Examples are Transformative



Counter Examples are Transformative



Counter Examples are Transformative



Everything’s Changed

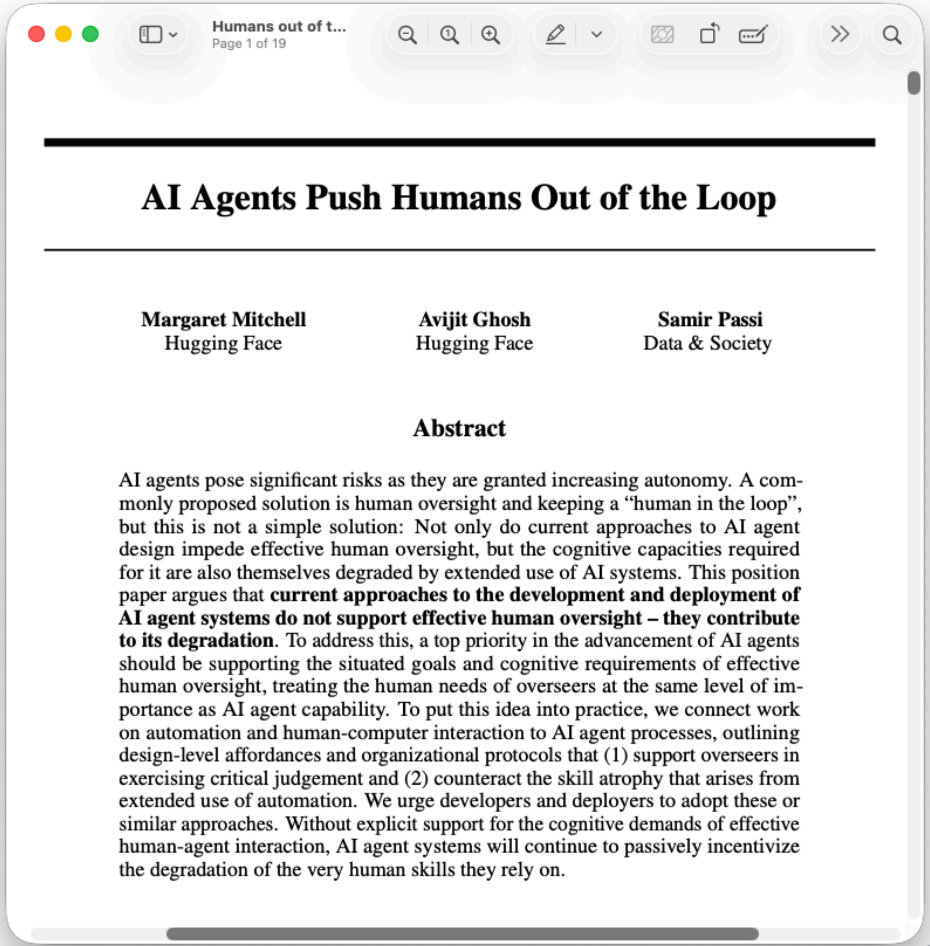
Passive Overseeing is Not Enough

Passive Overseeing is Not Enough

- LLMs code faster than we can read
  - Quality is not guaranteed
  - Just being “in the loop” doesn’t work

Passive Overseeing is Not Enough

- LLMs code faster than we can read
  - Quality is not guaranteed
  - Just being “in the loop” doesn’t work



AI Agents Push Humans Out of the Loop

Margaret Mitchell Hugging Face
Avijit Ghosh Hugging Face
Samir Passi Data & Society

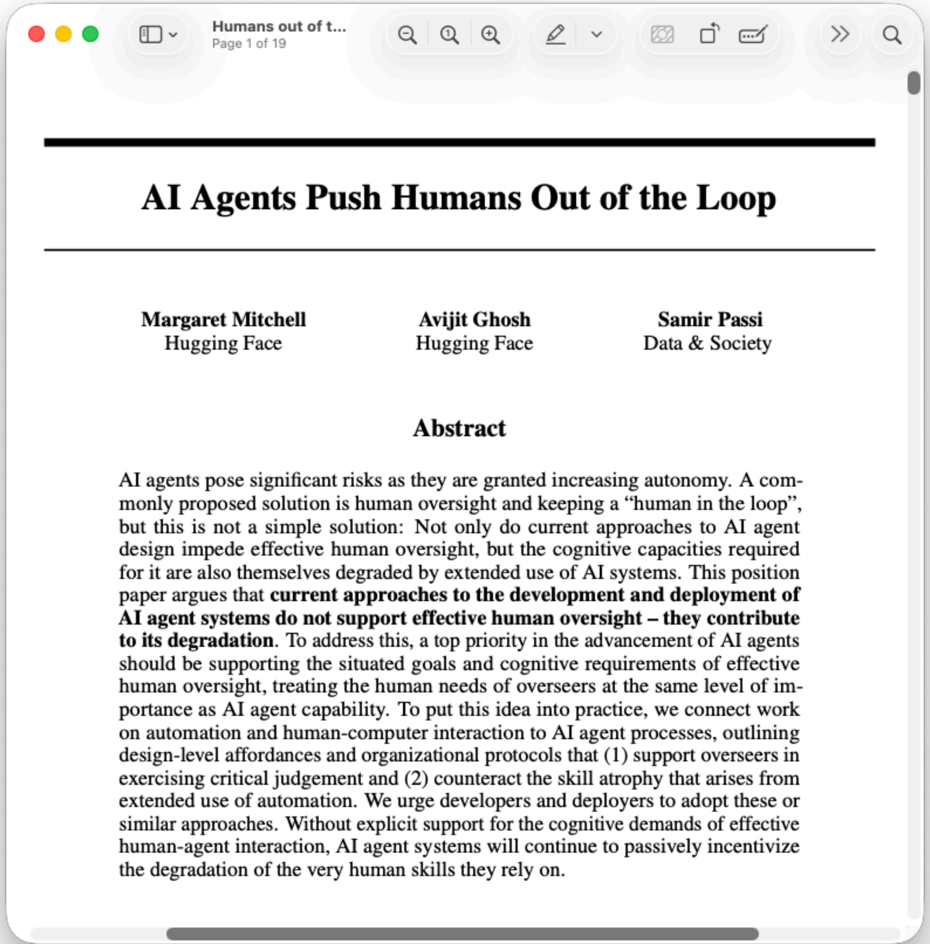
Abstract

AI agents pose significant risks as they are granted increasing autonomy. A commonly proposed solution is human oversight and keeping a “human in the loop”, but this is not a simple solution: Not only do current approaches to AI agent design impede effective human oversight, but the cognitive capacities required for it are also themselves degraded by extended use of AI systems. This position paper argues that **current approaches to the development and deployment of AI agent systems do not support effective human oversight – they contribute to its degradation**. To address this, a top priority in the advancement of AI agents should be supporting the situated goals and cognitive requirements of effective human oversight, treating the human needs of overseers at the same level of importance as AI agent capability. To put this idea into practice, we connect work on automation and human-computer interaction to AI agent processes, outlining design-level affordances and organizational protocols that (1) support overseers in exercising critical judgement and (2) counteract the skill atrophy that arises from extended use of automation. We urge developers and deployers to adopt these or similar approaches. Without explicit support for the cognitive demands of effective human-agent interaction, AI agent systems will continue to passively incentivize the degradation of the very human skills they rely on.

Source: https://arxiv.org/abs/2608.23642

Passive Overseeing is Not Enough

- LLMs code faster than we can read
  - Quality is not guaranteed
  - Just being “in the loop” doesn’t work
- Strategic cognitive friction
  - Embed human comprehension into software engineering



AI Agents Push Humans Out of the Loop

Margaret Mitchell
Hugging Face

Avijit Ghosh
Hugging Face

Samir Passi
Data & Society

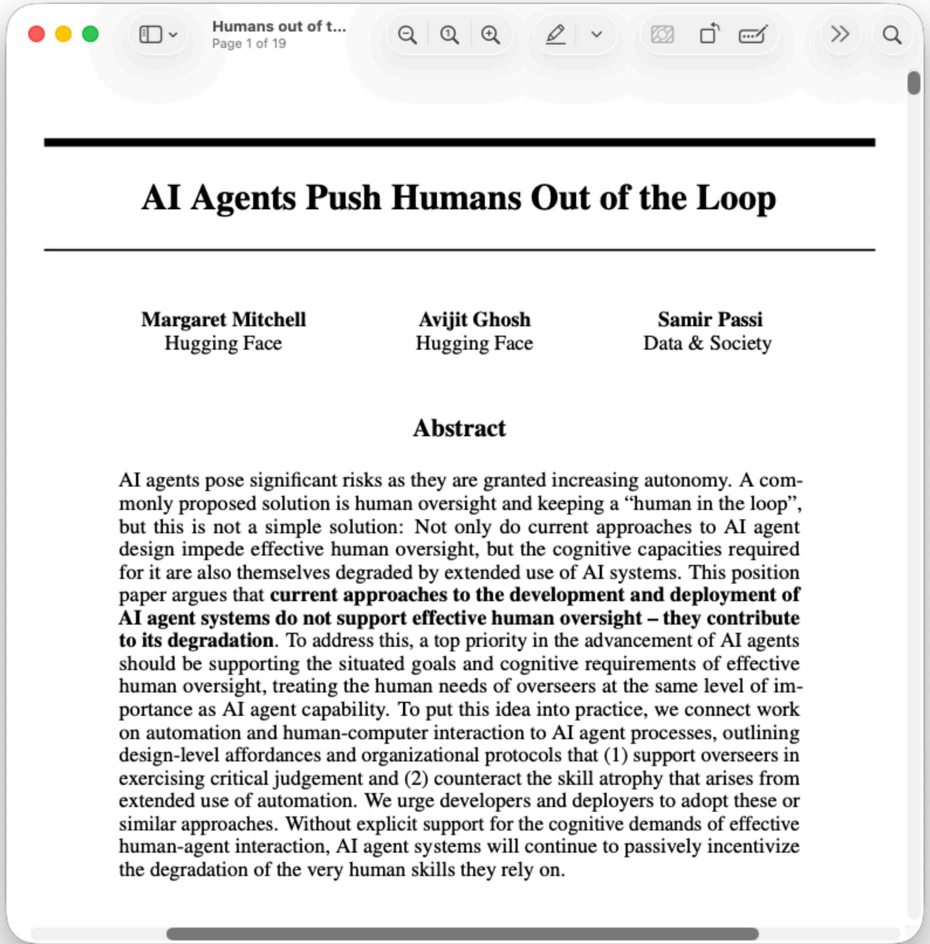
Abstract

AI agents pose significant risks as they are granted increasing autonomy. A commonly proposed solution is human oversight and keeping a “human in the loop”, but this is not a simple solution: Not only do current approaches to AI agent design impede effective human oversight, but the cognitive capacities required for it are also themselves degraded by extended use of AI systems. This position paper argues that **current approaches to the development and deployment of AI agent systems do not support effective human oversight – they contribute to its degradation**. To address this, a top priority in the advancement of AI agents should be supporting the situated goals and cognitive requirements of effective human oversight, treating the human needs of overseers at the same level of importance as AI agent capability. To put this idea into practice, we connect work on automation and human-computer interaction to AI agent processes, outlining design-level affordances and organizational protocols that (1) support overseers in exercising critical judgement and (2) counteract the skill atrophy that arises from extended use of automation. We urge developers and deployers to adopt these or similar approaches. Without explicit support for the cognitive demands of effective human-agent interaction, AI agent systems will continue to passively incentivize the degradation of the very human skills they rely on.

Source: https://arxiv.org/abs/2608.23642

Passive Overseeing is Not Enough

- LLMs code faster than we can read
  - Quality is not guaranteed
  - Just being "in the loop" doesn't work
- Strategic cognitive friction
  - Embed human comprehension into software engineering
- Exploratory testing brings cognitive friction back



AI Agents Push Humans Out of the Loop

Margaret Mitchell Hugging Face

Avijit Ghosh Hugging Face

Samir Passi Data & Society

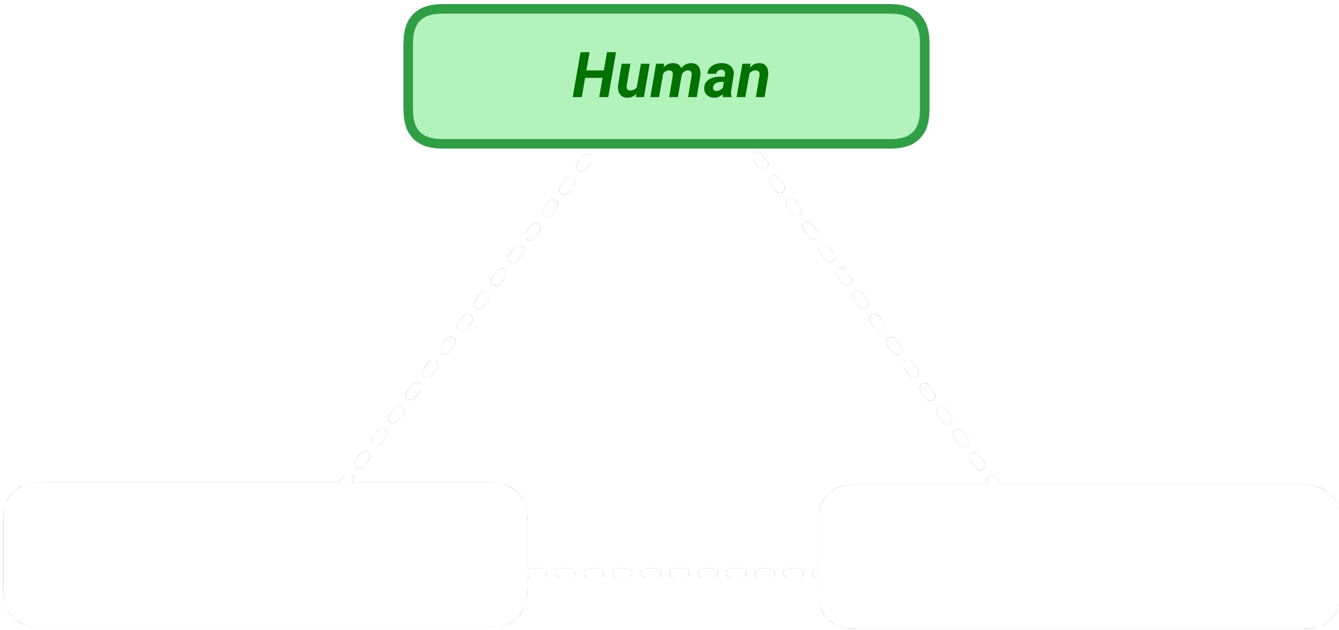
Abstract

AI agents pose significant risks as they are granted increasing autonomy. A commonly proposed solution is human oversight and keeping a "human in the loop", but this is not a simple solution: Not only do current approaches to AI agent design impede effective human oversight, but the cognitive capacities required for it are also themselves degraded by extended use of AI systems. This position paper argues that **current approaches to the development and deployment of AI agent systems do not support effective human oversight – they contribute to its degradation**. To address this, a top priority in the advancement of AI agents should be supporting the situated goals and cognitive requirements of effective human oversight, treating the human needs of overseers at the same level of importance as AI agent capability. To put this idea into practice, we connect work on automation and human-computer interaction to AI agent processes, outlining design-level affordances and organizational protocols that (1) support overseers in exercising critical judgement and (2) counteract the skill atrophy that arises from extended use of automation. We urge developers and deployers to adopt these or similar approaches. Without explicit support for the cognitive demands of effective human-agent interaction, AI agent systems will continue to passively incentivize the degradation of the very human skills they rely on.

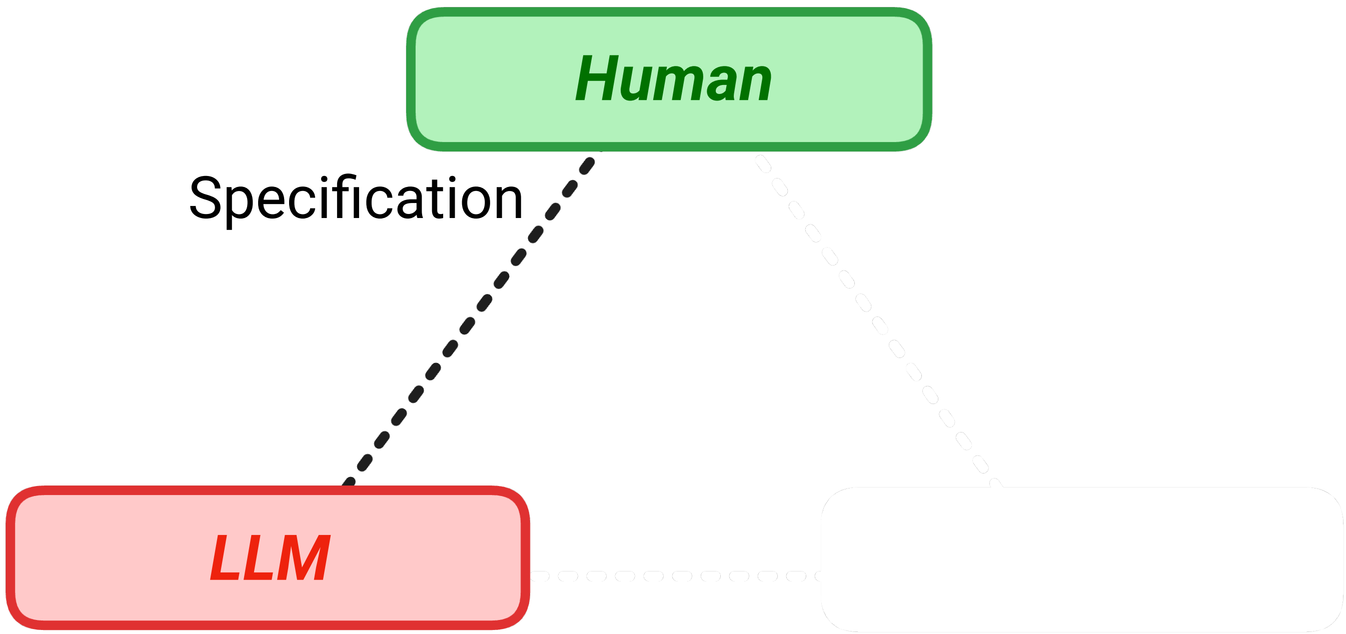
Source: https://arxiv.org/abs/2608.23642

Division of Cognitive Labour

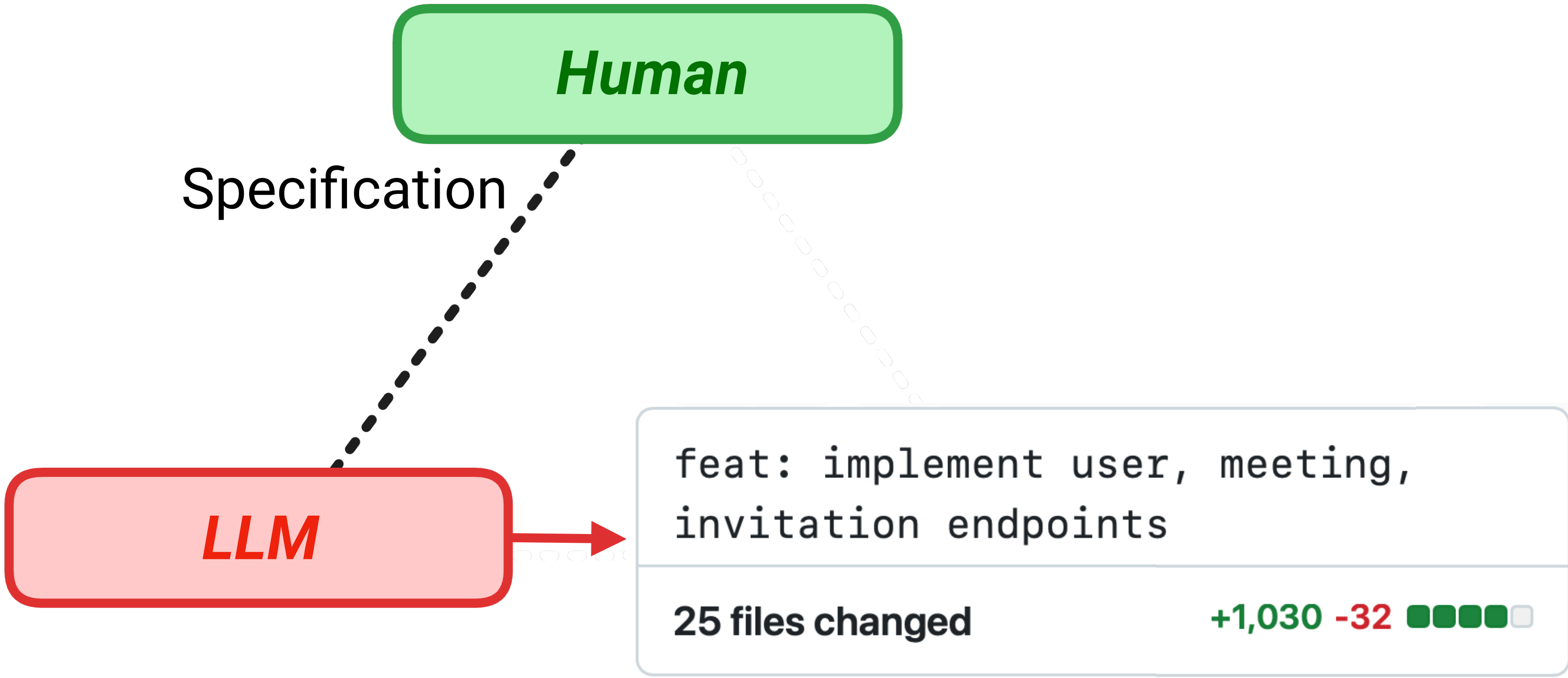
Division of Cognitive Labour



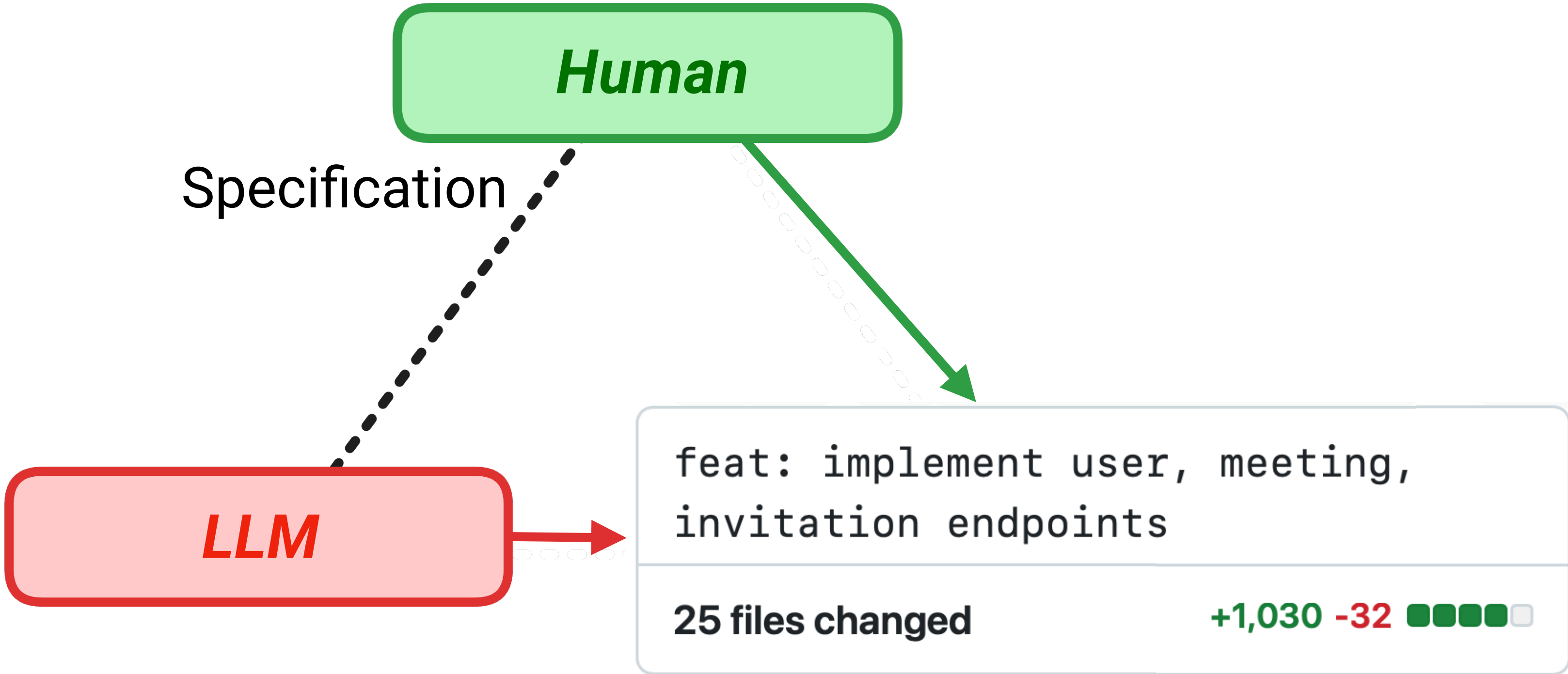
Division of Cognitive Labour



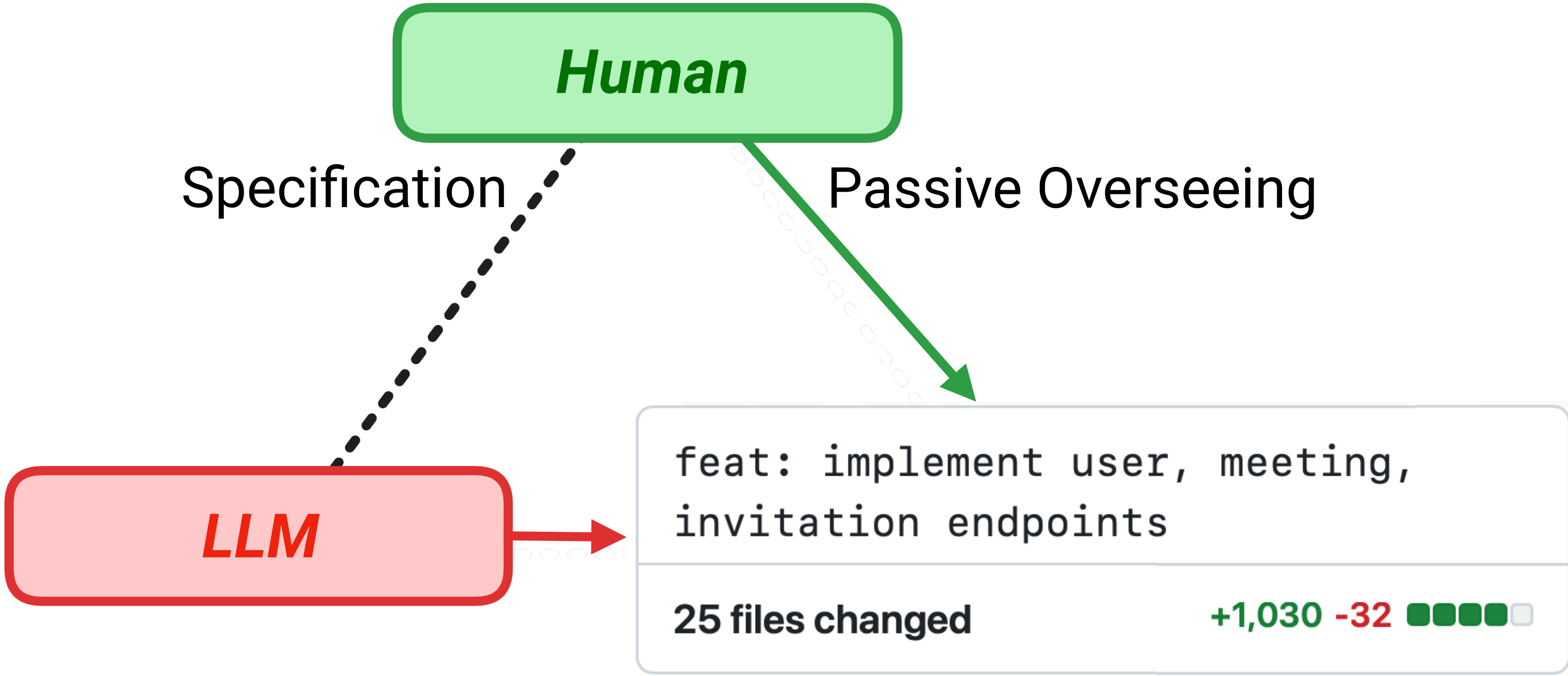
Division of Cognitive Labour



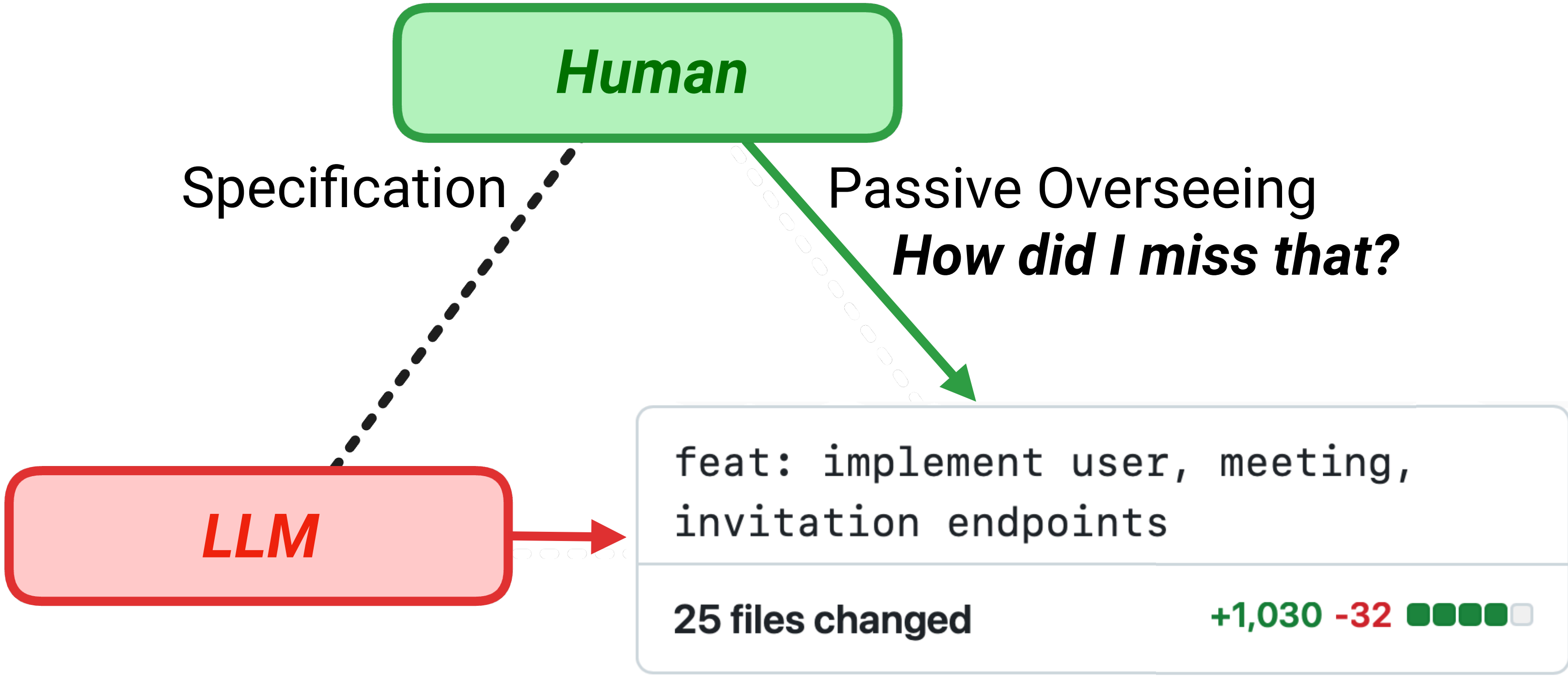
Division of Cognitive Labour



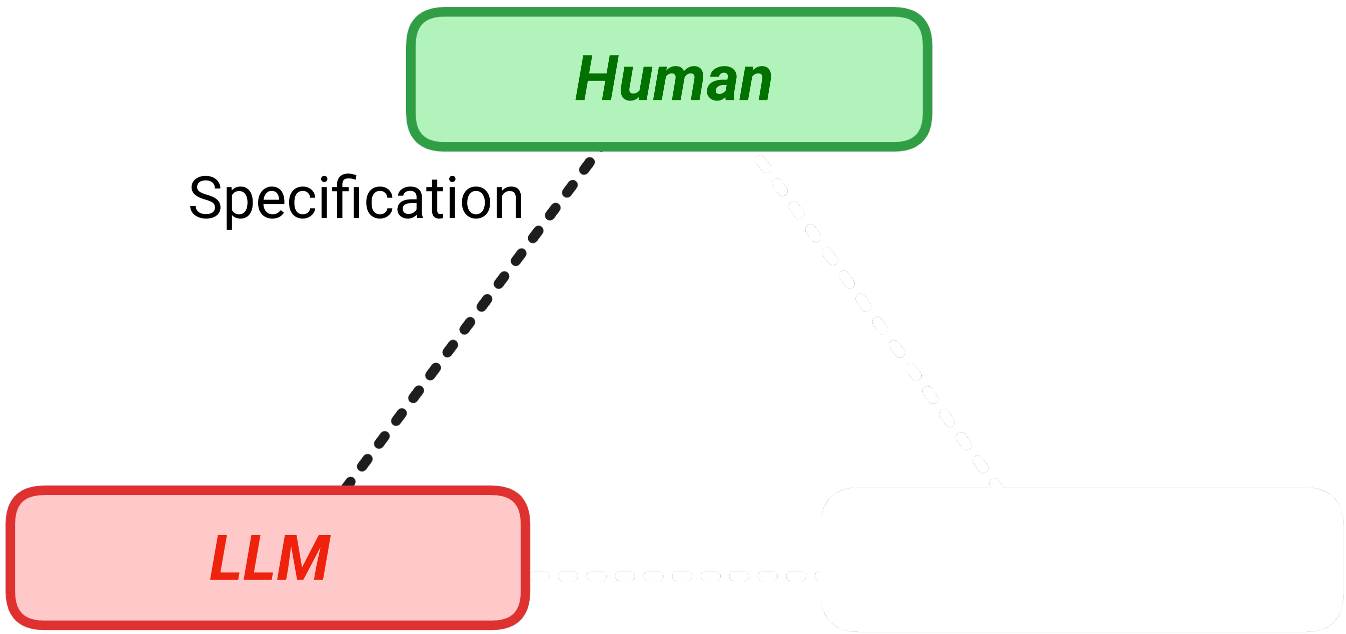
Division of Cognitive Labour



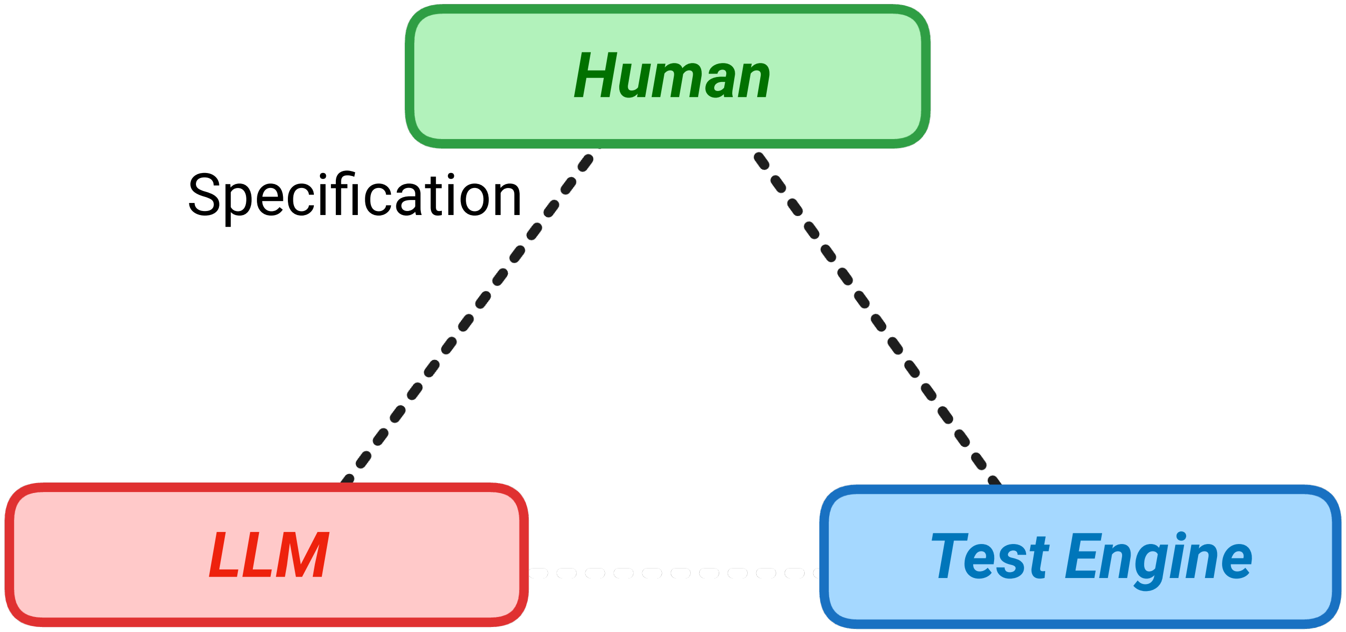
Division of Cognitive Labour



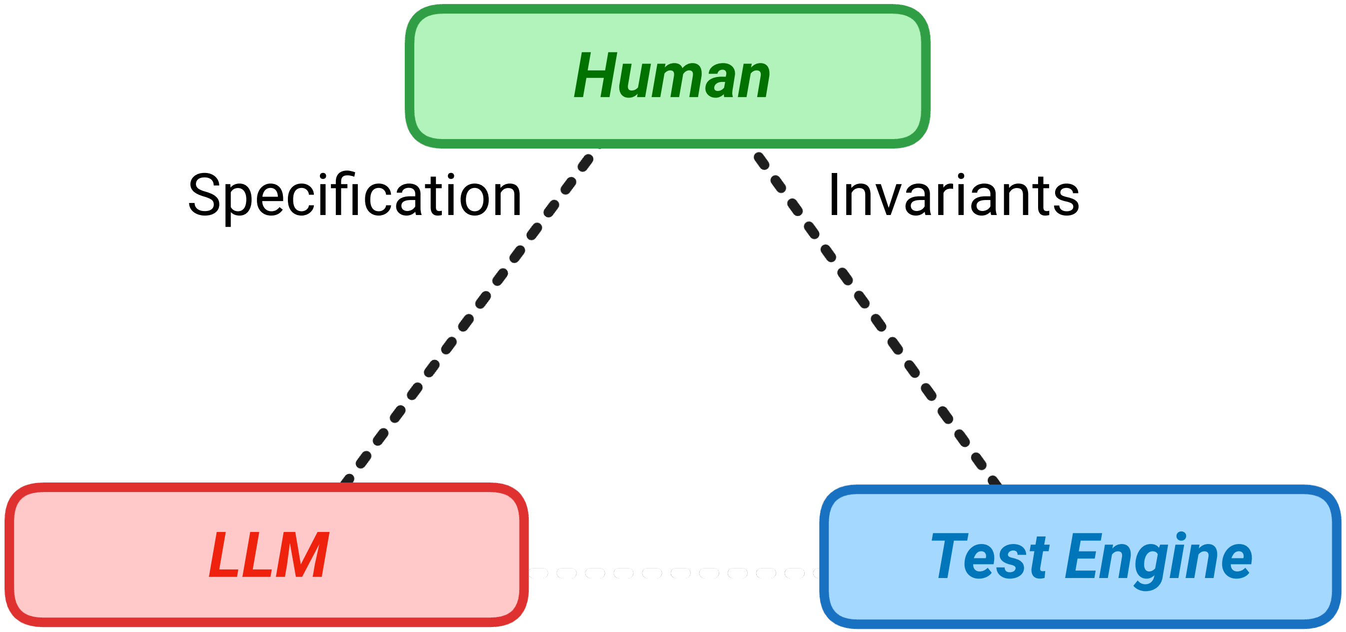
Division of Cognitive Labour



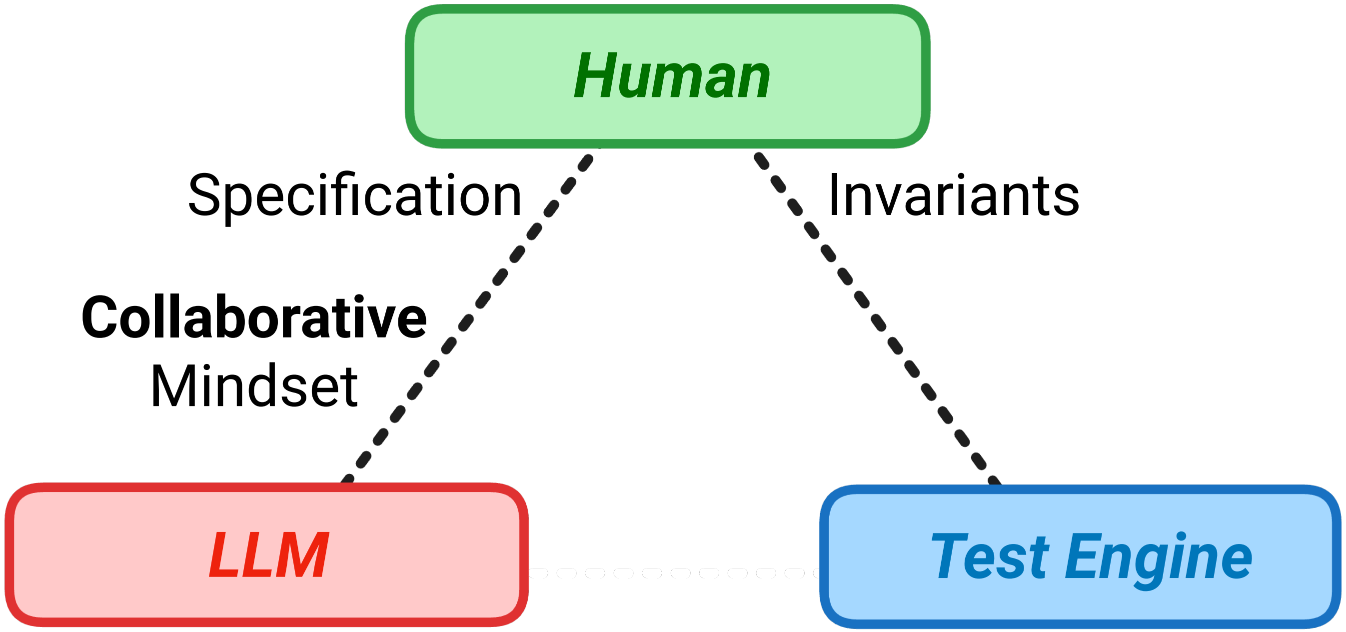
Division of Cognitive Labour



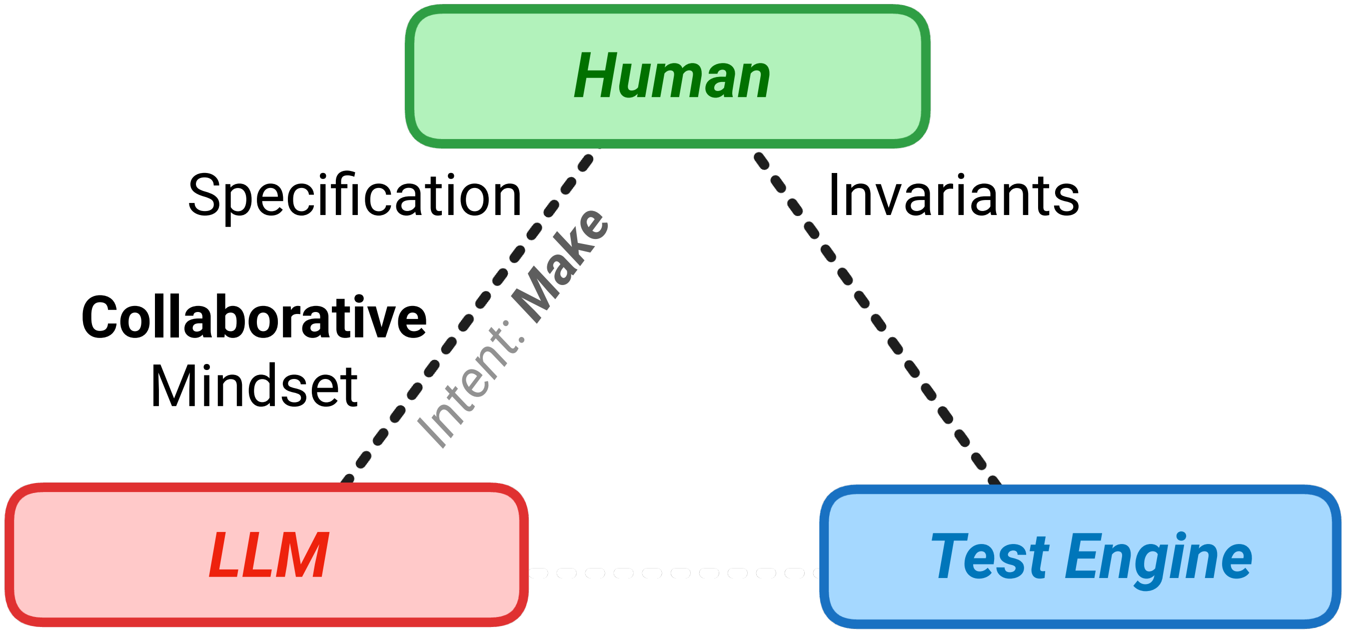
Division of Cognitive Labour



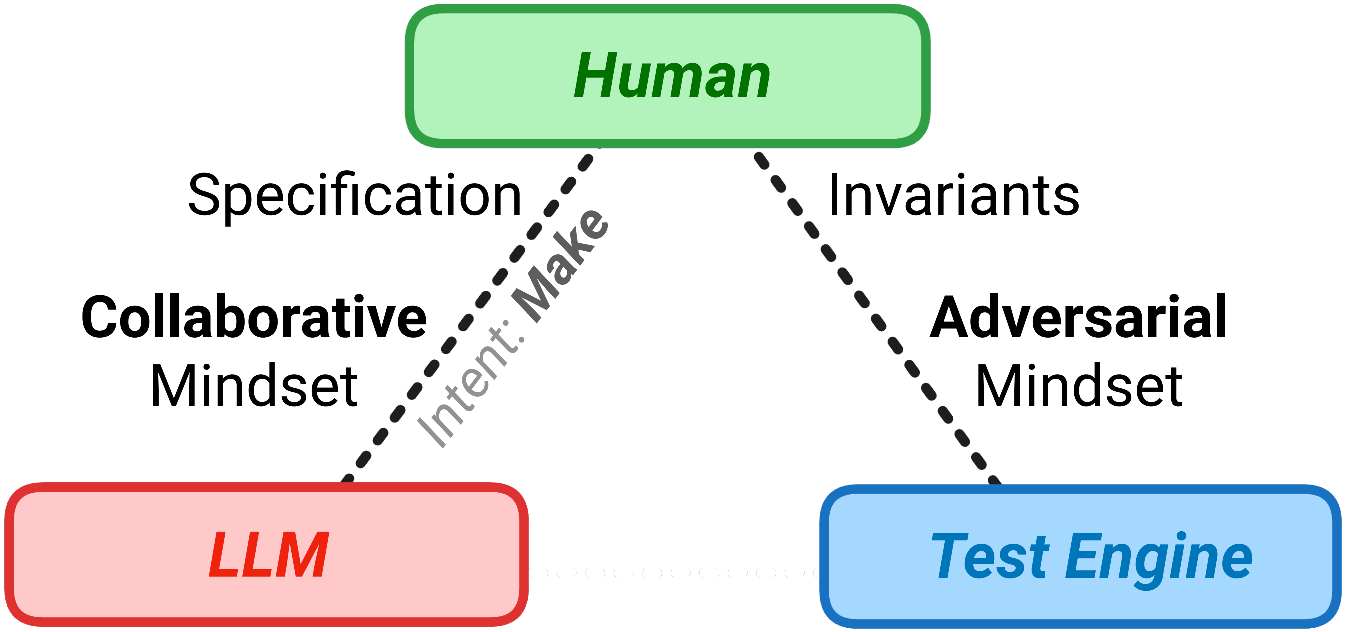
Division of Cognitive Labour



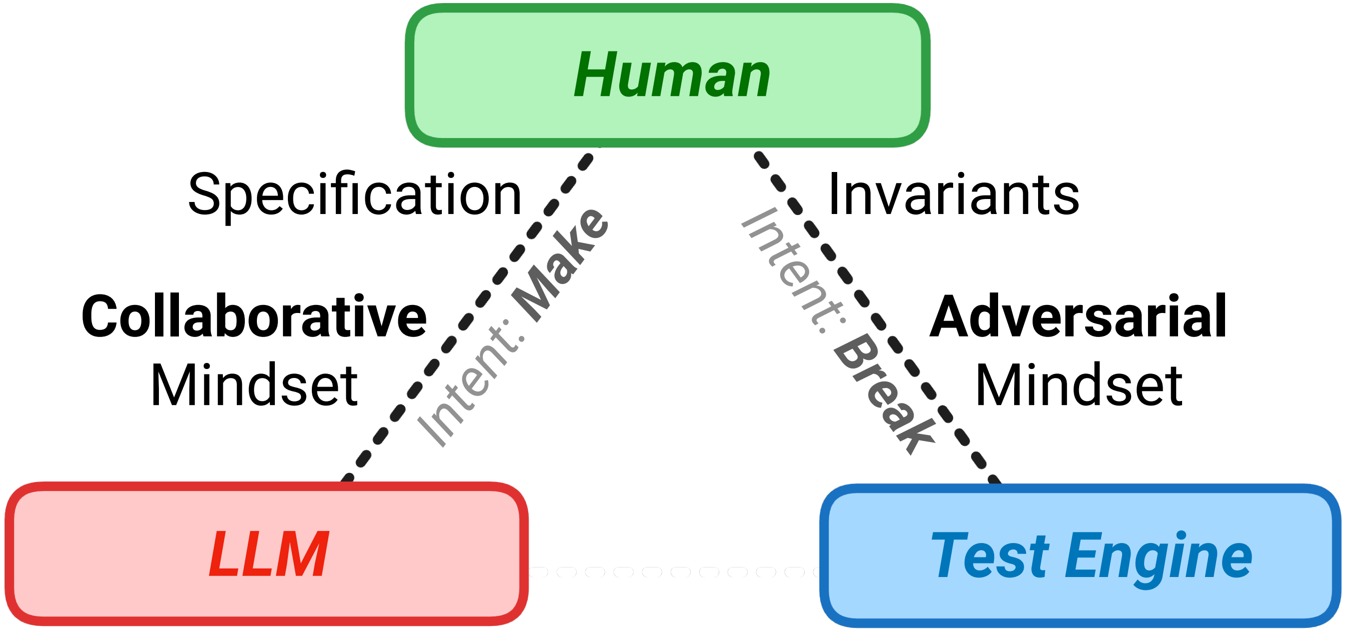
Division of Cognitive Labour



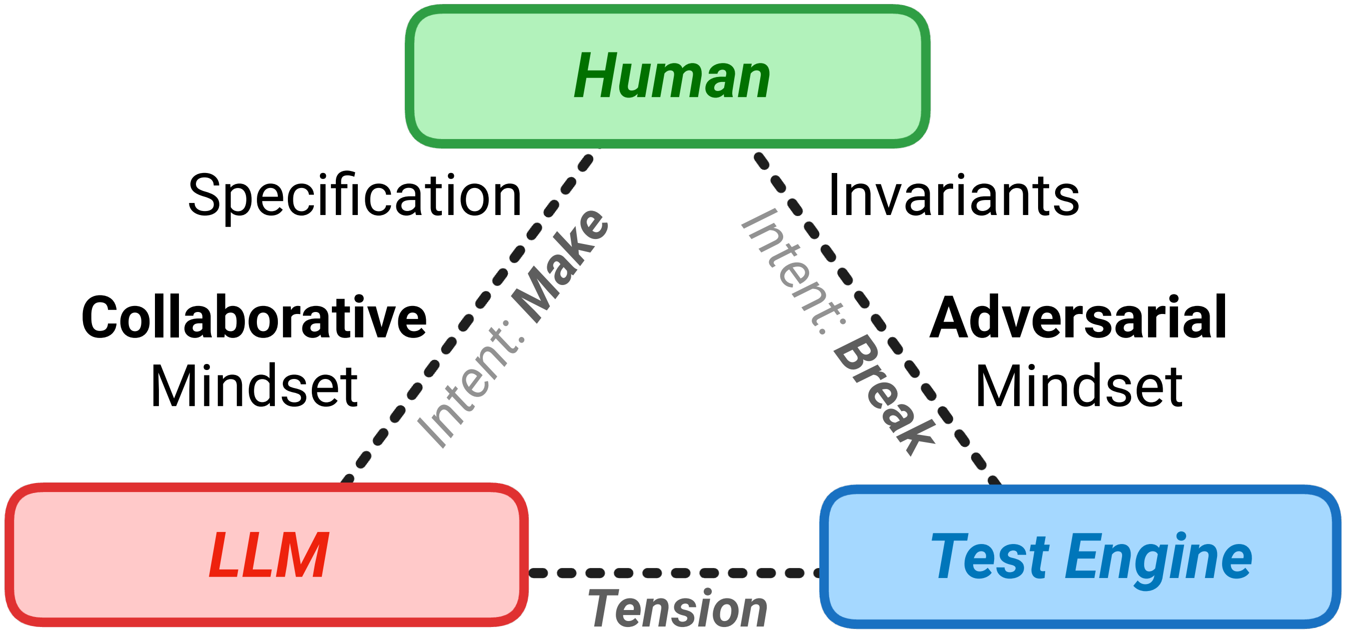
Division of Cognitive Labour



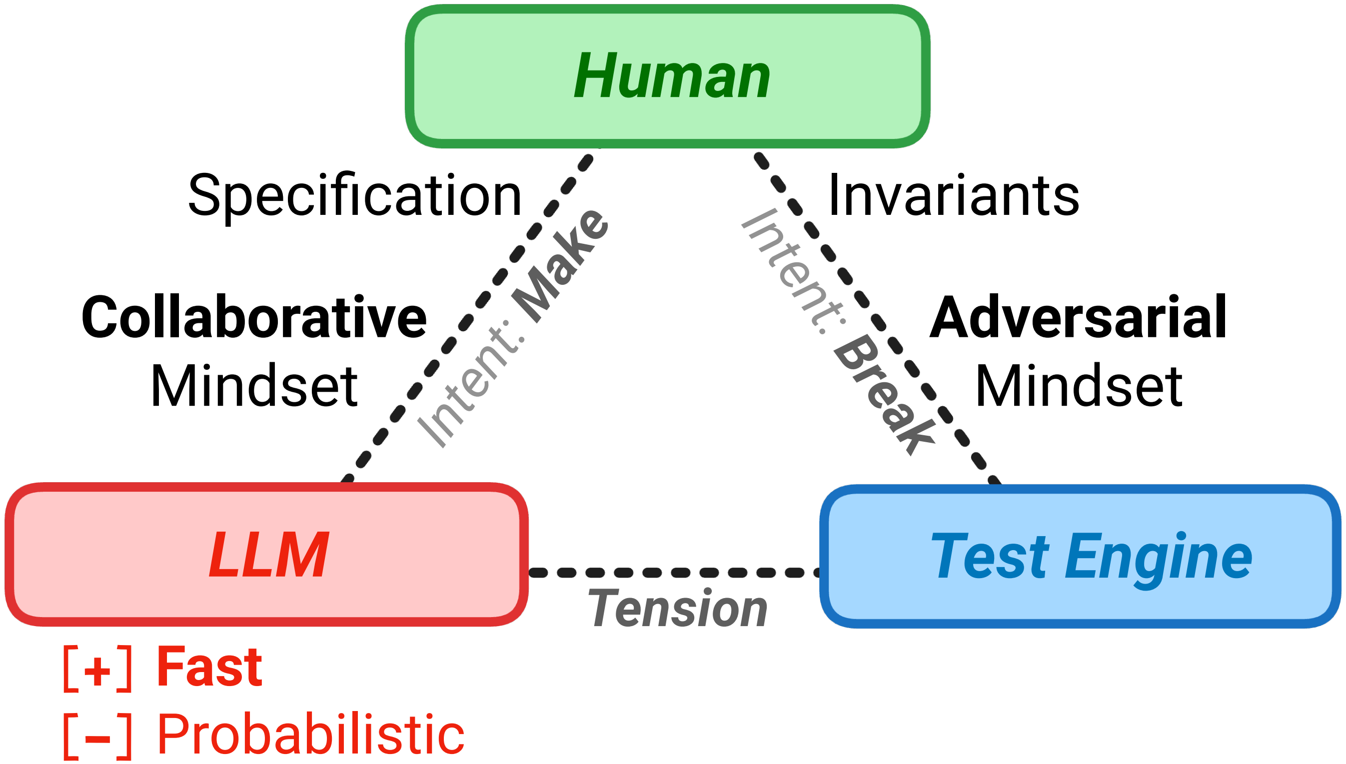
Division of Cognitive Labour



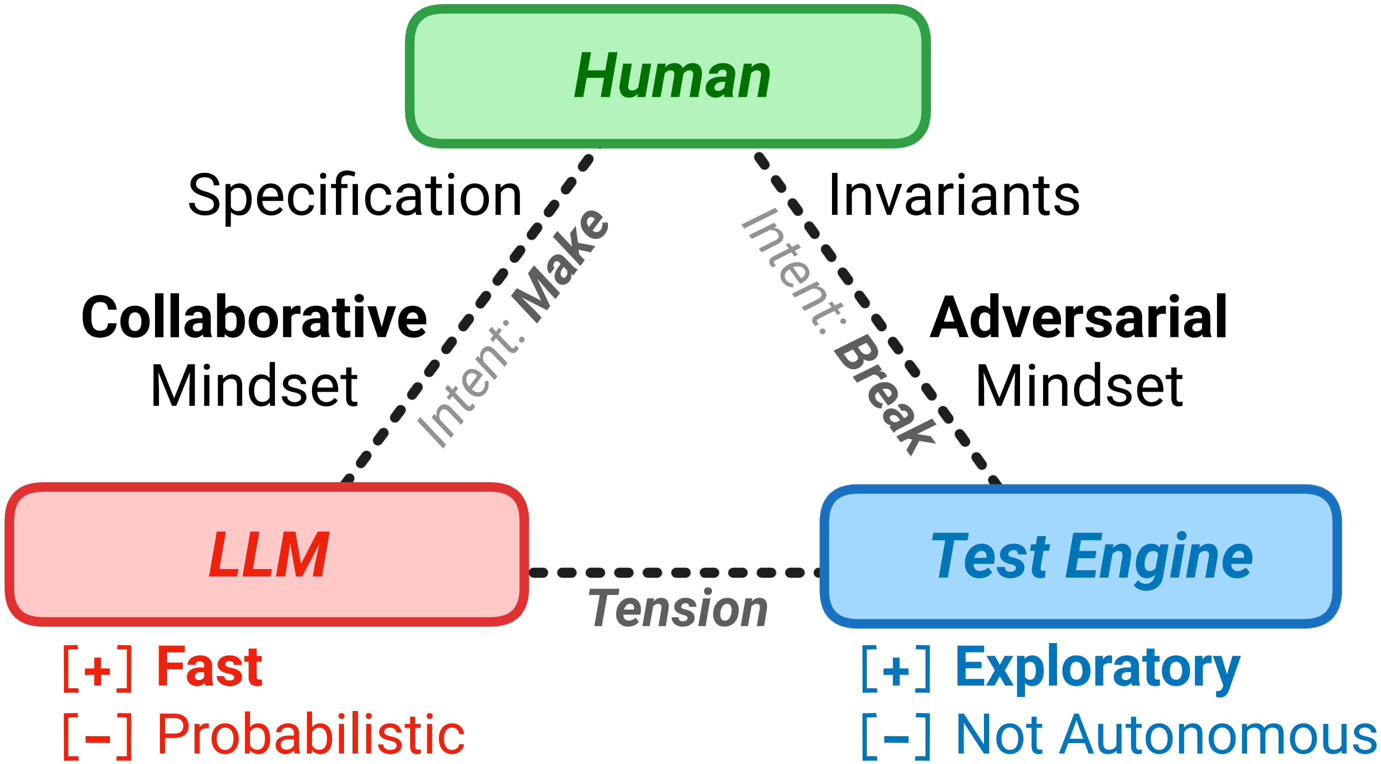
Division of Cognitive Labour



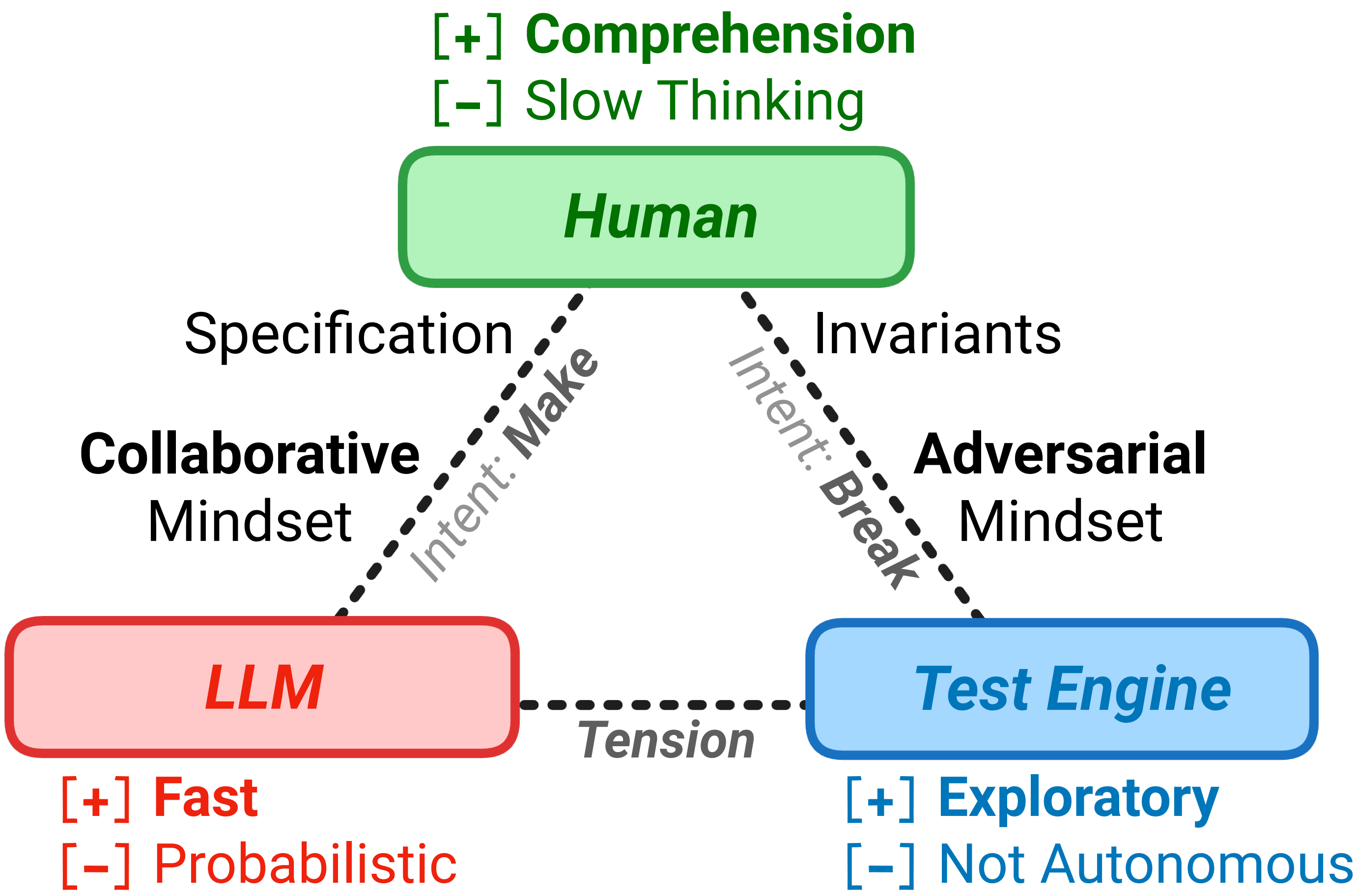
Division of Cognitive Labour



Division of Cognitive Labour



Division of Cognitive Labour



Thank You!

- mourjo.me
- github.com/mourjo/prompt-meetings
- mourjo.me/slides/2026-09-11-signals.pdf

linkedin.com/in/mourjo